概述

计算机系统基础

任课教师：
龚奕利
yiligong@whu.edu.cn

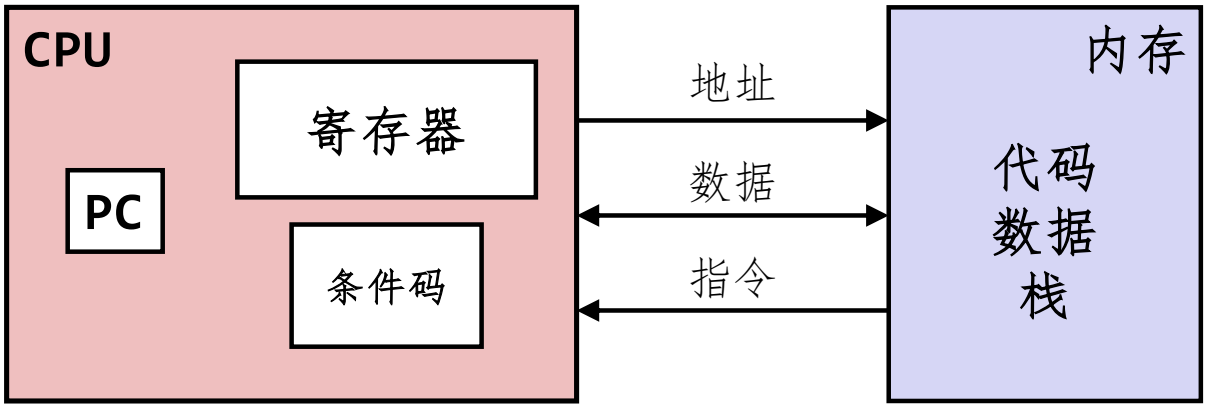
概述：本节内容

- Intel 处理器的历史与架构
- C 代码，汇编代码，机器级代码
- 汇编语言基础：寄存器，操作数，move 数据传送指令
- 算术和逻辑操作

一些定义

- 架构（指令集体系结构，ISA）：理解或书写汇编代码、机器级代码应该掌握的处理器设计的知识
  - 例如：指令集的规范，寄存器
- 微体系结构：架构的实现
  - 例如：高速缓存的大小、核的频率
- 代码格式：
  - 机器代码：处理器执行的字节级程序
  - 汇编代码：机器代码的文本表示
- ISA举例：
  - Intel：x86，IA32，Itanium，x86-64
  - ARM：Apple M-series，几乎应用在所有的手机上
  - MIPS，RISC-V
  - Loongson：LoongArch

汇编代码&机器代码之观

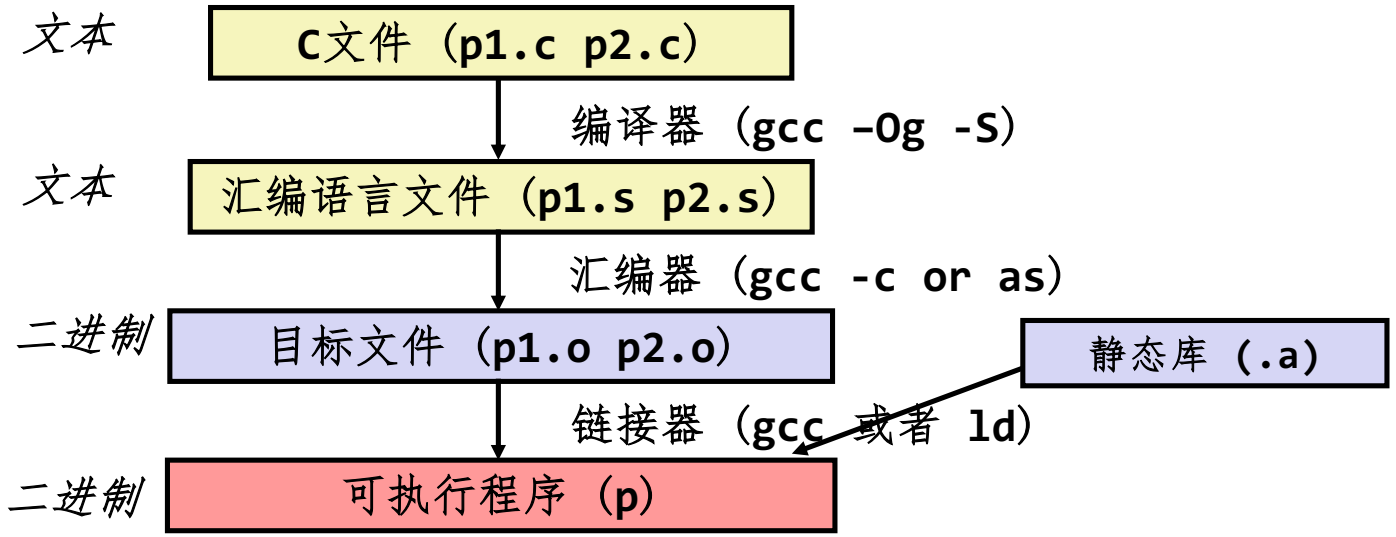


一个程序员应该知道：

- **PC：程序计数器**
  - 下一条指令的地址
  - 称作“RIP”（x86-64）
- **寄存器文件**
  - 程序频繁使用的数据
- **条件码**
  - 储存最近一次算术或逻辑运算的结果
  - 用于条件分支语句
- **内存**
  - 以字节为单位的数组
  - 代码和用户数据
  - 为程序创建栈帧

C 文件是如何转变为可执行程序的？

- 代码文件： **p1.c p2.c**
- 使用如下命令编译： **gcc –Og p1.c p2.c -o p**
  - 使用最基本的优化选项（**-Og**）
  - 得到的二进制文件存放在 **p** 中



编译生成汇编语言

C代码(sum.c)

```
long plus(long x, long y);

void sumstore(long x, long y,
              long *dest)
{
    long t = plus(x, y);
    *dest = t;
}
```

生成的x86-64汇编语言文件

```
sumstore:
    pushq   %rbx
    movq    %rdx, %rbx
    call    plus
    movq    %rax, (%rbx)
    popq    %rbx
    ret
```

使用命令

gcc -Og -S sum.c

生成文件 **sum.s**

*注意：由于不同版本的gcc或不同的编译器设置，在不同的机器（**Linux，Mac OS-X**）上会得到非常不同的结果。*

计算机系统基础-深入理解计算机系统（第三版）

汇编语言特点：操作

- 对存储在寄存器或内存上的数据进行算术操作
- 在内存和寄存器之间传送数据
  - 从内存向寄存器加载数据
  - 向内存存储寄存器的数据
- 控制的转移
  - 无条件跳转到过程或从过程返回
  - 条件分支语句

目标代码

sumstore的代码

```
0x0400595:
    0x53
    0x48
    0x89
    0xd3
    0xe8
    0xf2
    0xff
    0xff
    0xff
    0xff
    0x48
    0x89
    0x03
    0x5b
    0xc3
```

- 一共14个字节
- 每一个指令有1,3或5字节
- 从地址0x0400595开始

- 汇编器
  - 将 .s 文件汇编为 .o 文件
  - 每条指令的二进制编码
  - 几近完整的可执行代码
  - 还缺少不同文件之间的链接
- 链接器
  - 解析文件之间的引用
  - 与静态运行时库结合
    - 例如 **malloc, printf** 的代码
  - 一些库是*动态链接*的
    - 链接发生在程序开始执行时

机器指令示例

```
*dest = t;
```

```
movq %rax, (%rbx)
```

```
0x40059e:  48 89 d3
```

- C 代码
  - 将 t 的值存放在 dest 指向的地方
- 汇编语言
  - 将8字节的值存储在内存中
    - 在 x86-64 术语中称为*四字*
  - 操作数：

    t: 寄存器 **%rax**

    dest: 寄存器 **%rbx**

    ***dest:** 内存 **M[%rbx]**
- 目标代码
  - 字节指令
  - 存储在地址 **0x40059e** 处

反汇编目标代码

反汇编后

```
0000000000400595 <sumstore>:
  400595:  53                 push   %rbx
  400596:  48 89 d3           mov    %rdx,%rbx
  400599:  e8 f2 ff ff ff     callq  400590 <plus>
  40059e:  48 89 03           mov    %rax,(%rbx)
  4005a1:  5b                 pop    %rbx
  4005a2:  c3                 retq
```

- 反汇编器

  objdump -d sum

  - 检查目标代码的有用工具
  - 分析指令序列的位模式
  - 得到目标代码大致对应的汇编代码
  - 可被用于 .out（完全可执行文件）和 .o 文件

另一种反汇编的方式

目标代码

```
0x0400595:
    0x53
    0x48
    0x89
    0xd3
    0xe8
    0xf2
    0xff
    0xff
    0xff
    0x48
    0x89
    0x03
    0x5b
    0xc3
```

反汇编后

```
Dump of assembler code for function sumstore:
 0x0000000000400595 <+0>: push   %rbx
 0x0000000000400596 <+1>: mov    %rdx,%rbx
 0x0000000000400599 <+4>: callq  0x400590 <plus>
 0x000000000040059e <+9>: mov    %rax,(%rbx)
 0x00000000004005a1 <+12>:pop    %rbx
 0x00000000004005a2 <+13>:retq
```

- 在 **gdb** 调试器中

  gdb sum

  disassemble sumstore

  - 反汇编过程

  x/14xb sumstore

  - 检查从 sumstore 开始的14个字节

什么是可以反汇编的？

```
% objdump -d WINWORD.EXE

WINWORD.EXE:   file format pei-i386

No symbols in "WINWORD.EXE".
Disassembly of section .text:

30001000 <.text>:
30001000:
30001001:
30001003:
30001005:
3000100a:
```

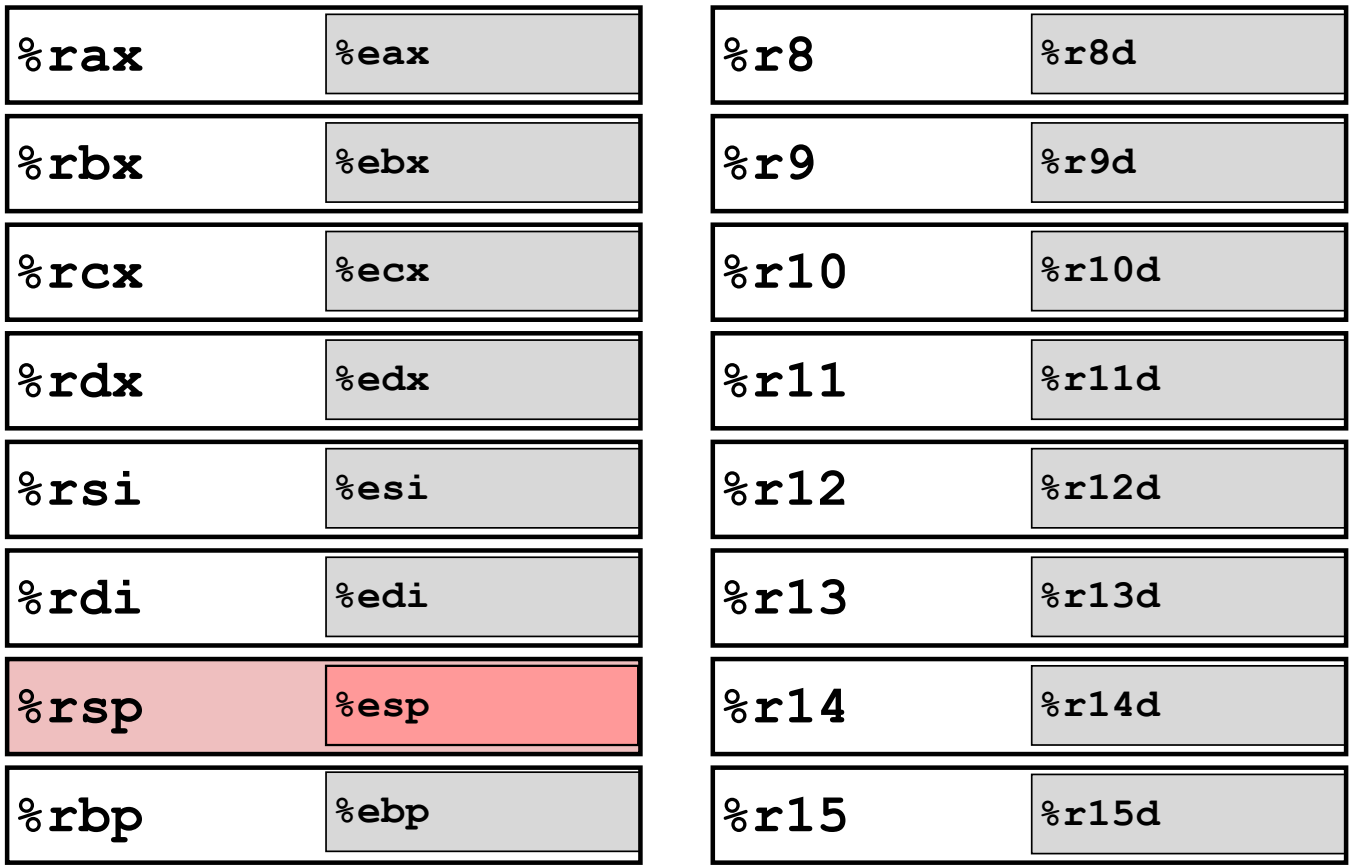
微软最终用户许可协议禁止反向工程

- 任何可以被解释为可执行代码的文件
- 反汇编器检查字节内容并重构汇编源代码

本节内容：

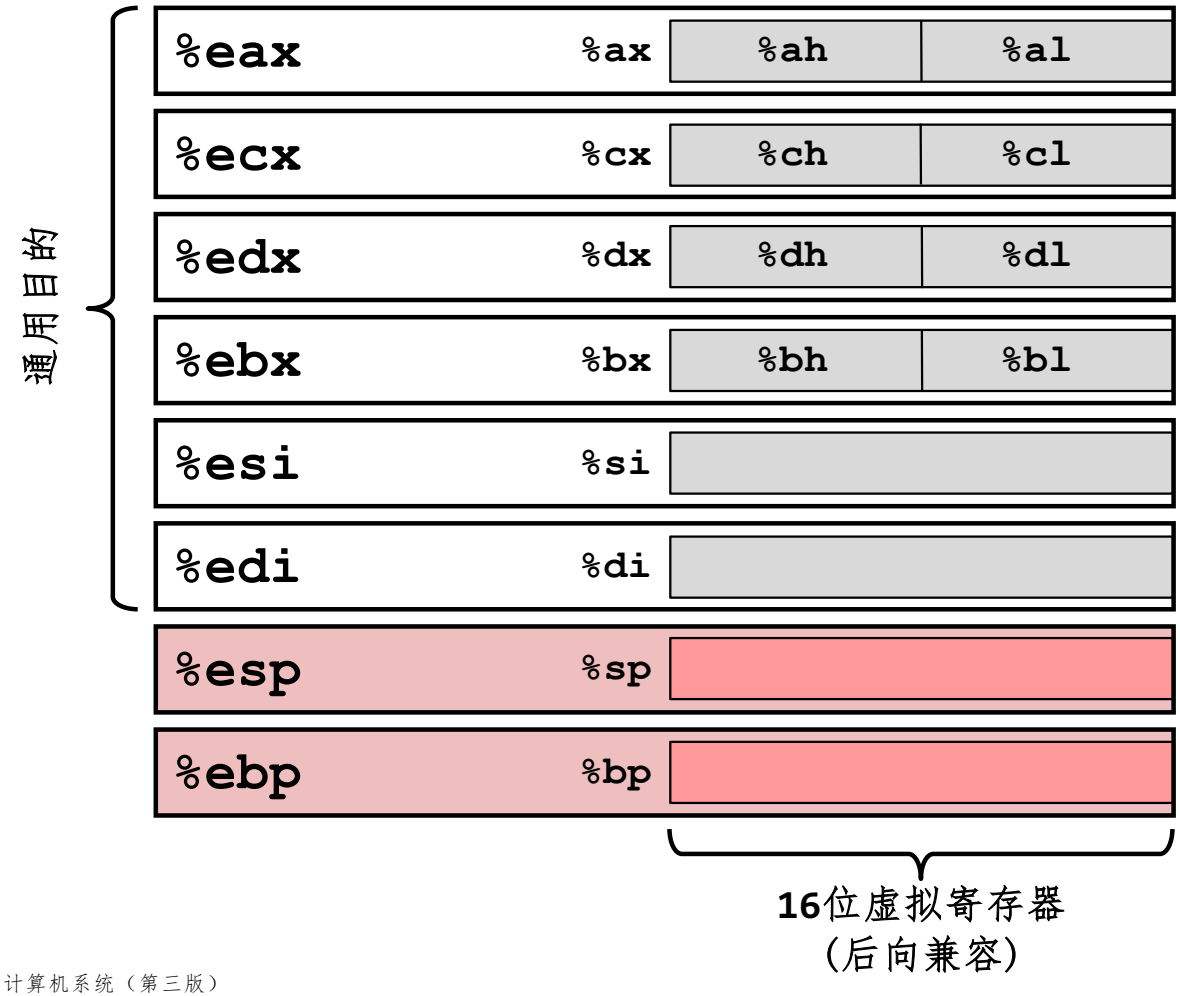
- Intel 处理器的历史与架构
- C 代码，汇编代码，机器级代码
- 汇编语言基础：寄存器，操作数，move 数据传送指令
- 算术和逻辑操作

x86-64整型寄存器



- 也可以只引用低1，2 或 4字节

一些历史：IA32寄存器



起源
（大部分已经淘汰）

累加 accumulate

计数器 counter

数据 data

基址 base

源索引 sourceindex

目的索引 destination index

栈指针 stack pointer

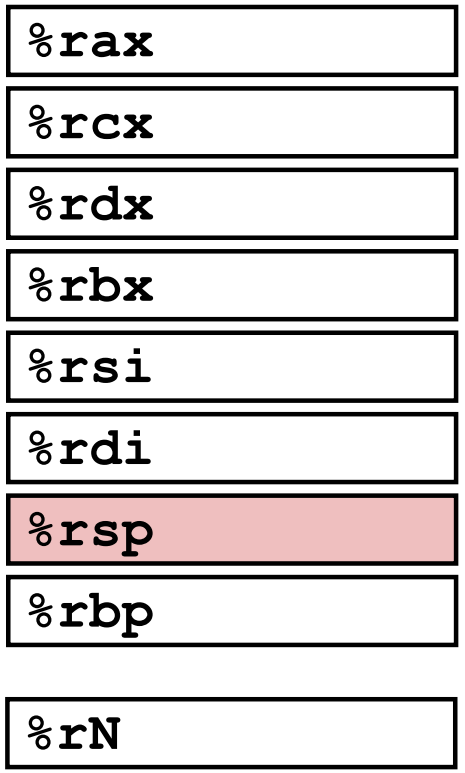
基址指针 base pointer

传送数据

- 传送数据

  movq *Source*, *Dest*

- 操作数类型
  - *立即数：*常整型数据
    - 例如：**$0x400, $-533**
    - 类似于 C 常量，只是多了前缀 **$**
    - 用1，2或4字节编码
  - *寄存器：*16个整型寄存器中的一个
    - 例如：**%rax, %r13**
    - 但 **%rsp** 保留为特殊用途
    - 其它的寄存器对个别指令有特殊用途
  - *内存：*内存中从寄存器指向的地址开始的8个连续字节
    - 最简单的例子：**(%rax)**
    - 也有一些其它的寻址方式



movq 操作数组合示例

	Source	Dest	Src,Dest	对应的 C 代码
movq	立即数	Reg	`movq $0x4,%rax`	`temp = 0x4;`
		Mem	`movq $-147,(%rax)`	`*p = -147;`
	寄存器	Reg	`movq %rax,%rdx`	`temp2 = temp1;`
		Mem	`movq %rax,(%rdx)`	`*p = temp;`
	内存	Reg	`movq (%rax),%rdx`	`temp = *p;`

单个指令无法实现数据的内存到内存传送

简单寻址模式

- 通常 (R) Mem[Reg[R]]
  - 寄存器 R 表示内存地址
  - 类似于 C 中的解引用

```
movq (%rcx),%rax
```

- 偏移量 D(R) Mem[Reg[R]+D]
  - 寄存器 R 表示一段内存区间的起始地址
  - 常偏移量 D 表示偏移量的大小

```
movq 8(%rbp),%rdx
```

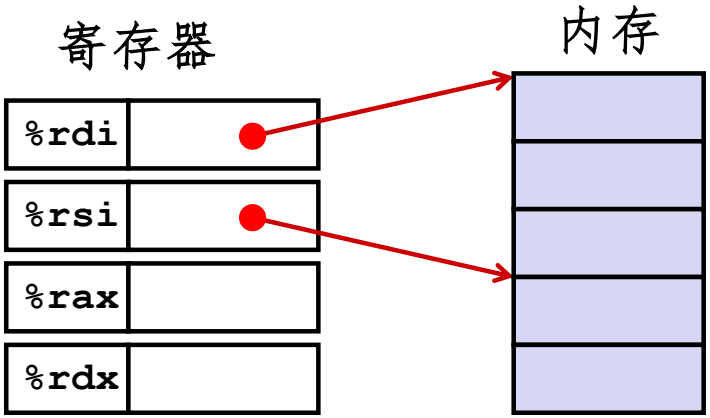
示例

```
void swap(long *xp, long *yp)
{
  long t0 = *xp;
  long t1 = *yp;
  *xp = t1;
  *yp = t0;
}
```

```
swap:
    movq    (%rdi), %rax
    movq    (%rsi), %rdx
    movq    %rdx, (%rdi)
    movq    %rax, (%rsi)
    ret
```

再来一个示例：理解函数swap()

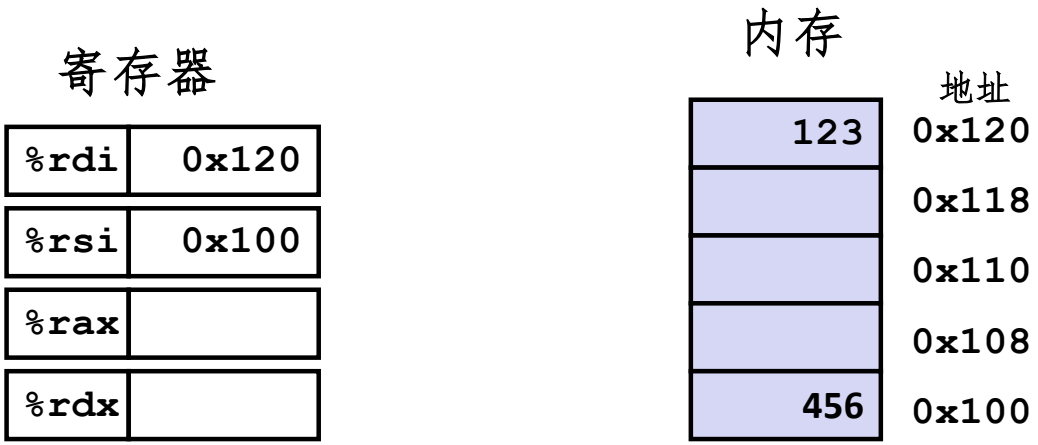
```
void swap(long *xp, long *yp)
{
  long t0 = *xp;
  long t1 = *yp;
  *xp = t1;
  *yp = t0;
}
```



寄存器	值
%rdi	xp
%rsi	yp
%rax	t0
%rdx	t1

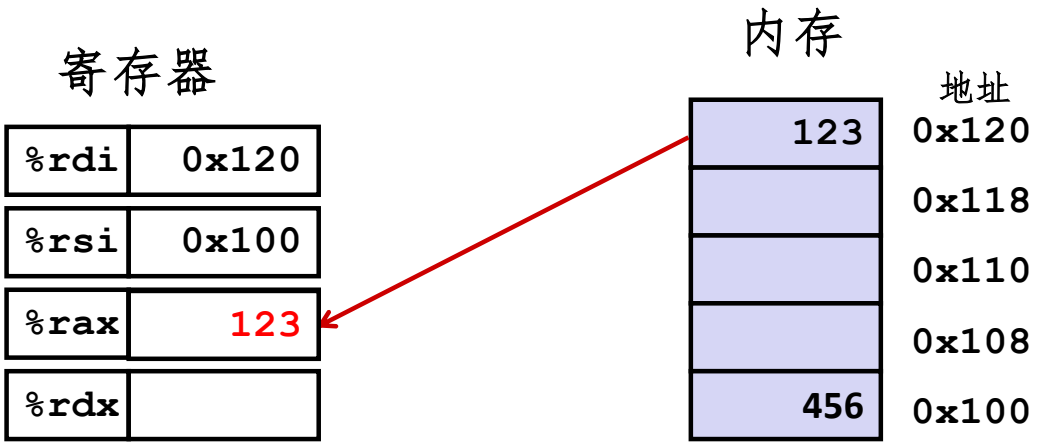
```
swap:
    movq    (%rdi), %rax  # t0 = *xp
    movq    (%rsi), %rdx  # t1 = *yp
    movq    %rdx, (%rdi)  # *xp = t1
    movq    %rax, (%rsi)  # *yp = t0
    ret
```

理解函数swap()



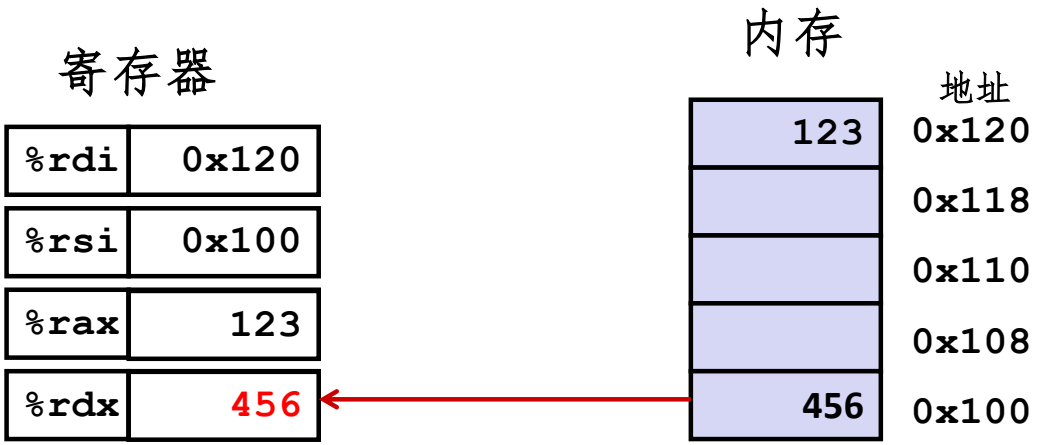

```
swap:
   movq    (%rdi), %rax  # t0 = *xp
   movq    (%rsi), %rdx  # t1 = *yp
   movq    %rdx, (%rdi)  # *xp = t1
   movq    %rax, (%rsi)  # *yp = t0
   ret
```

理解函数swap()



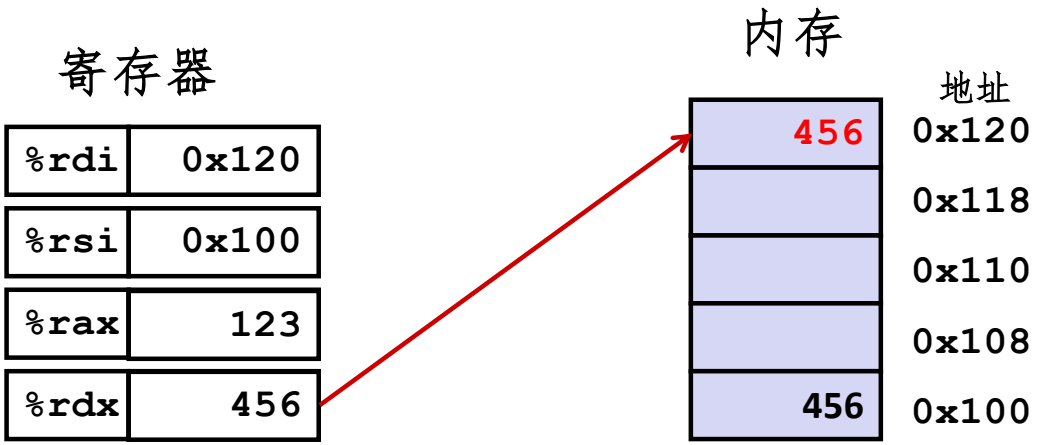

```
swap:
    movq    (%rdi), %rax  # t0 = *xp
    movq    (%rsi), %rdx  # t1 = *yp
    movq    %rdx, (%rdi)  # *xp = t1
    movq    %rax, (%rsi)  # *yp = t0
    ret
```

理解函数swap()



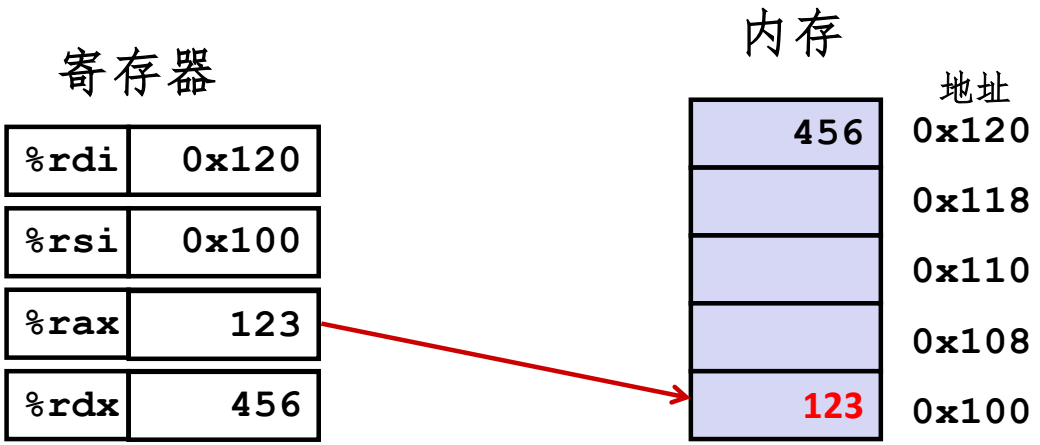

```
swap:
    movq    (%rdi), %rax  # t0 = *xp
    movq    (%rsi), %rdx  # t1 = *yp
    movq    %rdx, (%rdi)  # *xp = t1
    movq    %rax, (%rsi)  # *yp = t0
    ret
```

理解函数swap()




```
swap:
    movq    (%rdi), %rax  # t0 = *xp
    movq    (%rsi), %rdx  # t1 = *yp
    movq    %rdx, (%rdi)  # *xp = t1
    movq    %rax, (%rsi)  # *yp = t0
    ret
```

理解函数swap()




```
swap:
    movq    (%rdi), %rax  # t0 = *xp
    movq    (%rsi), %rdx  # t1 = *yp
    movq    %rdx, (%rdi)  # *xp = t1
    movq    %rax, (%rsi)  # *yp = t0
    ret
```

简单内存寻址模式

- 通常 (R) Mem[Reg[R]]
  - 寄存器R表示内存地址
  - 类似于C中的解引用

```
movq (%rcx),%rax
```

- 偏移量 D(R) Mem[Reg[R]+D]
  - 寄存器 R 表示一段内存区间的起始地址
  - 常偏移量 D 表示偏移量的大小

```
movq 8(%rbp),%rdx
```

完整寻址模式

- 最通用的形式

  D(Rb,Ri,S)　　Mem[Reg[Rb]+S*Reg[Ri]+ D]

  - D:　常偏移量，1、2或4字节
  - Rb:　基址寄存器，16个整型寄存器中的任意一个
  - Ri:　索引寄存器，除 **%rsp** 外的任一寄存器
  - S:　伸缩值，1，2，4或8（*为什么是这些数？*）

- 特殊情况

  (Rb,Ri)　　Mem[Reg[Rb]+Reg[Ri]]

  D(Rb,Ri)　　Mem[Reg[Rb]+Reg[Ri]+D]

  (Rb,Ri,S)　　Mem[Reg[Rb]+S*Reg[Ri]]

地址计算示例

%rdx	0xf000
%rcx	0x0100

表达式	地址计算	地址
0x8(%rdx)		
(%rdx,%rcx)		
(%rdx,%rcx,4)		
0x80(,%rdx,2)		

地址计算示例

%rdx	0xf000
%rcx	0x0100

表达式	地址计算	地址
0x8(%rdx)	0xf000 + 0x8	0xf008
(%rdx,%rcx)	0xf000 + 0x100	0xf100
(%rdx,%rcx,4)	0xf000 + 4*0x100	0xf400
0x80(,%rdx,2)	2*0xf000 + 0x80	0x1e080

本节内容

- Intel 处理器的历史与架构
- C 代码，汇编代码，机器级代码
- 汇编语言基础：寄存器，操作数，move 数据传送指令
- 算术和逻辑操作

地址计算指令

- **leaq Src, Dst**
  - Src是寻址模式表达式
  - Dst目的地址表达式

- 用途
  - 计算地址但不发生内存引用
    - 例如，p = &x[i];
  - 计算形如 x + k*y 的算术表达式
    - k = 1, 2, 4或8
- 例如

```
long m12(long x)
{
  return x*12;
}
```

编译器转换到汇编语言

```
leaq (%rdi,%rdi,2), %rax # t <- x+x*2
salq $2, %rax            # return t<<2
```

一些算术操作

- 二元操作

```
          格式          计算结果
addq      Src,Dest      Dest = Dest + Src
subq      Src,Dest      Dest = Dest – Src
imulq     Src,Dest      Dest = Dest * Src
salq      Src,Dest      Dest = Dest << Src   也称为：shlq
sarq      Src,Dest      Dest = Dest >> Src   算术
shrq      Src,Dest      Dest = Dest >> Src   逻辑
xorq      Src,Dest      Dest = Dest ^ Src
andq      Src,Dest      Dest = Dest & Src
orq       Src,Dest      Dest = Dest | Src
```

P133 3.5.5 特殊的算术操作		
指令	效果	描述
imulq S	R[%rdx]: R[%rax] ← S * R[%rdx]	有符号全乘法
mulq S	R[%rdx]: R[%rax] ← S * R[%rdx]	无符号全乘法

- 请注意参数的顺序
- 无符号整型和有符号整型之间没有区别（为什么？）

一些算术操作

- 一元操作

```
incq    Dest      Dest = Dest + 1
decq    Dest      Dest = Dest – 1
negq    Dest      Dest = – Dest
notq    Dest      Dest = ~ Dest
```

- 更多的指令见课本教材

算术表达式示例

```
long arith
(long x, long y, long z)
{
  long t1 = x+y;
  long t2 = z+t1;
  long t3 = x+4;
  long t4 = y * 48;
  long t5 = t3 + t4;
  long rval = t2 * t5;
  return rval;
}
```

```
arith:
  leaq    (%rdi,%rsi), %rax
  addq    %rdx, %rax
  leaq    (%rsi,%rsi,2), %rdx
  salq    $4, %rdx
  leaq    4(%rdi,%rdx), %rcx
  imulq   %rcx, %rax
  ret
```

有趣的指令

- **leaq**: 加载有效地址
- **salq**: 移位
- **imulq**: 乘法
  - 但是只使用了一次

理解算术表达式示例

```
long arith
(long x, long y, long z)
{
  long t1 = x+y;
  long t2 = z+t1;
  long t3 = x+4;
  long t4 = y * 48;
  long t5 = t3 + t4;
  long rval = t2 * t5;
  return rval;
}
```

```
arith:
   leaq    (%rdi,%rsi), %rax    # t1
   addq    %rdx, %rax           # t2
   leaq    (%rsi,%rsi,2), %rdx
   salq    $4, %rdx             # t4
   leaq    4(%rdi,%rdx), %rcx   # t5
   imulq   %rcx, %rax           # rval
   ret
```

寄存器	用途
%rdi	参数 x
%rsi	参数 y
%rdx	参数 z
%rax	t1, t2, rval
%rdx	t4
%rcx	t5

程序的机器级表示I：总结

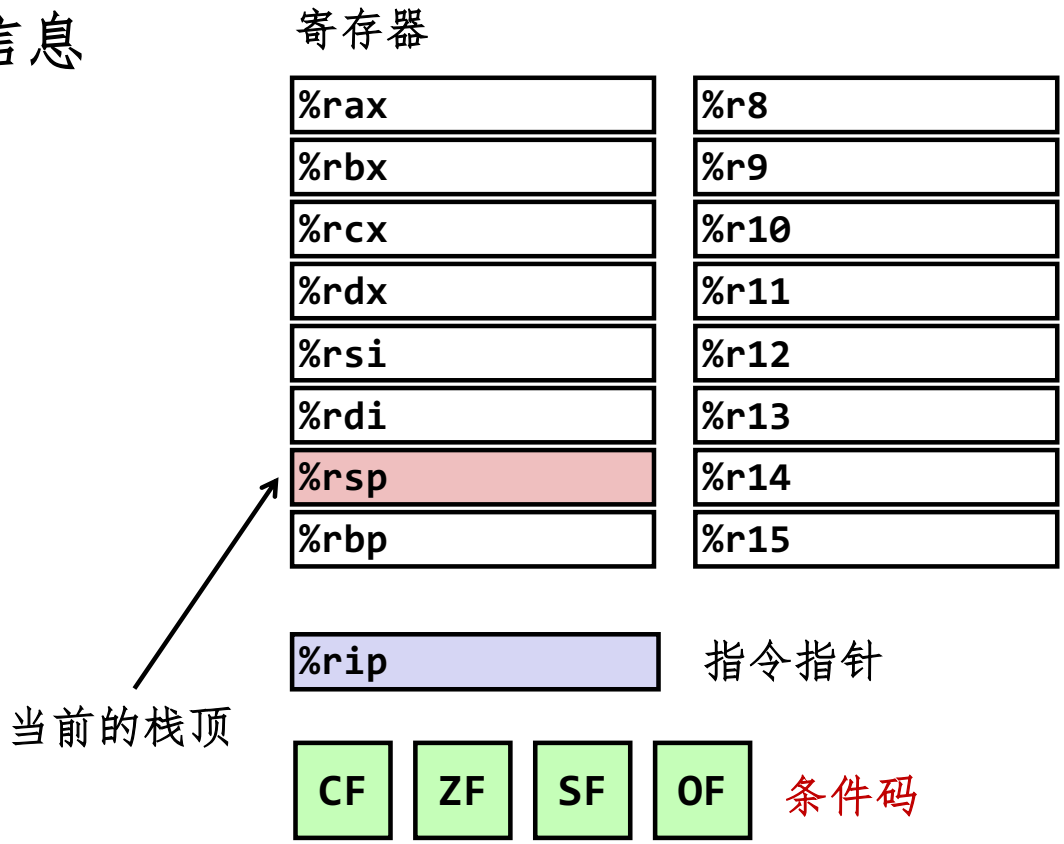
- Intel 处理器的历史与架构
  - 不断演进的设计产生了很多奇怪之处
- C 代码，汇编代码，机器级代码
  - 新的可见状态：程序计数器，寄存器...
  - 编译器需要将语句、表达式和过程转换成低级的指令序列
- 汇编语言基础：寄存器，操作数，move 数据传送指令
  - x86-64 的move 指令囊括了各类型的数据传送形式
- 算术和逻辑操作
  - C 编译器会分辨出不同的指令组合，并执行计算

控制：本节内容

- 控制：条件码
- 条件分支
- 循环
- Switch 语句

处理器状态（x86-64，部分）

- 当前正在执行的程序的信息
  - 临时数据
    (%rax, …)
  - 运行时栈的位置
    (%rsp)
  - 当前代码控制点的位置
    (%rip, …)
  - 最近一次测试的状态
    (CF, ZF, SF, OF)



条件码（隐式设置）

- 单比特位寄存器
  - CF　进位标志（无符号）　SF　符号标志（有符号）
  - ZF　零标志　OF　溢出标志（有符号）

- 算术操作隐式设置（可理解为副作用）

  例如：addq Src, Dest ↔ t = a+b

  设置CF：如果最高位产生进位（无符号溢出）

  设置ZF：如果t == 0

  设置SF：如果t < 0（有符号）

  设置OF：如果补码溢出（有符号，正溢出或负溢出）

  `(a>0 && b>0 && t<0) || (a<0 && b<0 && t>=0)`

- leaq 指令并不会设置条件码

CF：进位标志

- 在 x86 机器中，对减法操作，是将 CF 当作借位标志（borrow flag）来设置的
  - 当计算 a-b 且 a<b 时，必须进行借位
  - 如果 a≥b，该位被清零
- 带借位减法指令(SBB)会计算 a–b–C = a–(b+C)
- 不带借位减法指令(SUB)计算时，会视借位被清零
- System/360, 6502, MSP430, COP8, ARM and PowerPC 处理器使用其他的约定方式
  - 它会直接使用等式 –x = (not x)+1 (i.e. 不会存储进位的反转值)，以 a+(not b)+1 方式来计算 a–b，并依据此加法设置该位。

条件码（显式设置：Compare）

- 使用 **Compare** 指令显式设置
  - `cmpq Src2, Src1`
  - `cmpq b,a` 就像执行 `a-b` 但不改变目的寄存器的值

  - 设置CF：如果最高位发生进位 （用于无符号比较）
  - 设置ZF：如果 `a == b`
  - 设置SF：如果 `(a-b) < 0`（有符号）
  - 设置OF：如果补码（有符号）溢出

```
(a>0 && b<0 && (a-b)<0) || (a<0 && b>0 && (a-b)>0)
```

条件码（显式设置：Test）

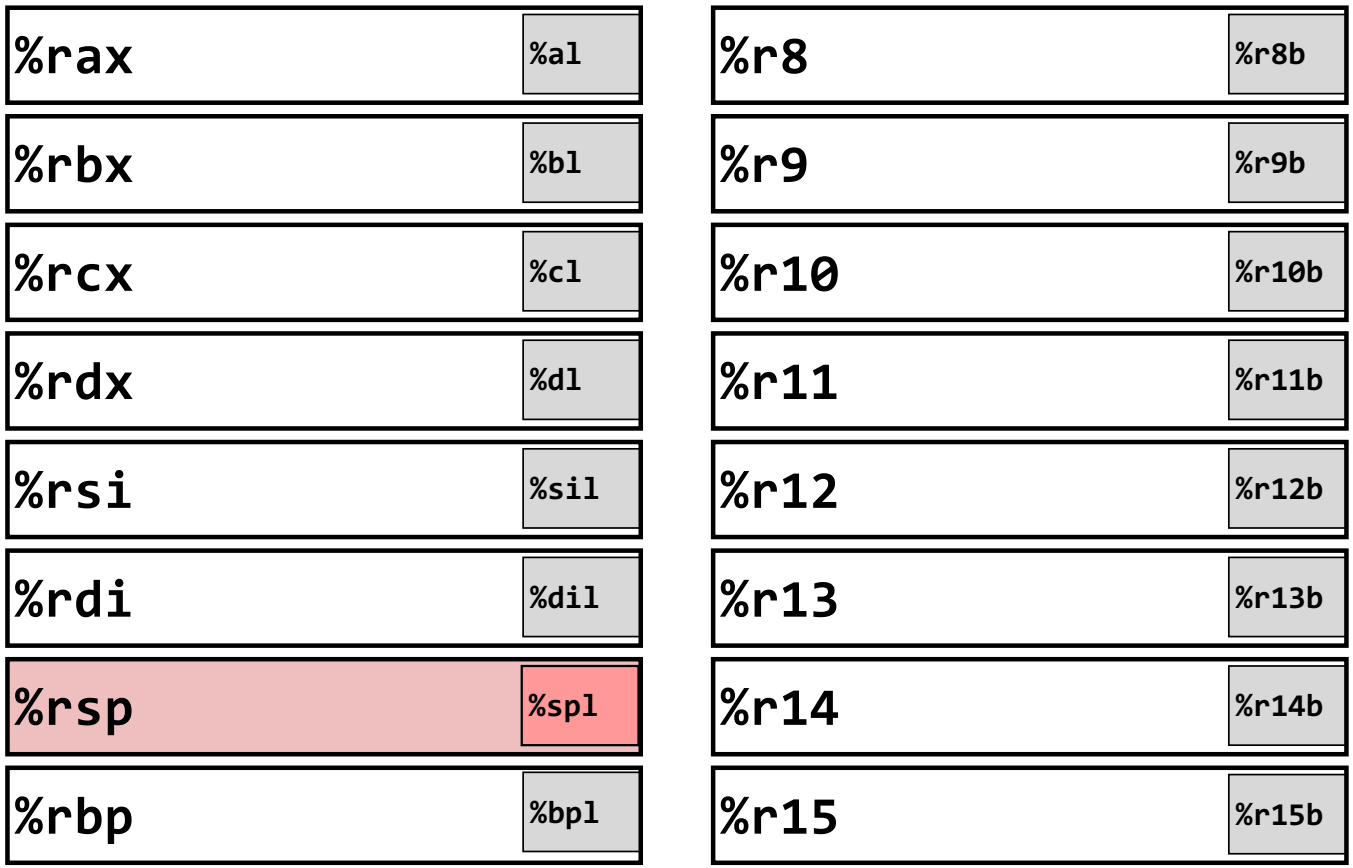
- 使用 Test 语句显示设置
  - testq Src2, Src1
    - testq b,a 类似于执行 a&b 但不改变目的寄存器的值
  - 基于 Src1 & Src2 的值设置条件码
  - 当一个操作数为掩码时会很有用
  - 设置ZF：当 a&b == 0
  - 设置SF：当 a&b < 0

访问条件码

- **SetX** 指令
  - 根据条件码不同组合的结果设置目的寄存器的最低字节为0或1
  - 并不改变剩下的7个字节

SetX	条件	含义
sete	ZF	相等/为零
setne	~ZF	不相等/不为零
sets	SF	负数
setns	~SF	非负数
setl	(SF^OF)	小于（有符号）
setle	(SF^OF)|ZF	小于等于（有符号）
setg	~(SF^OF)&~ZF	大于（有符号）
setge	~(SF^OF)	大于等于（有符号）
seta	~CF&~ZF	大于（无符号）
setb	CF	小于（无符号）

x86-64 整型寄存器



- 可以引用低位字节

读取条件码（续）

- SetX 指令
  - 根据条件码的组合设置单个字节的值
- 某个可寻址的字节寄存器
  - 并不改变其它字节
  - 比较典型的是用 movzbl 指令完成整个操作
    - 32位的指令会同时将高32位清零

```
int gt (long x, long y)
{
  return x > y;
}
```

寄存器	值
%rdi	变量 x
%rsi	变量 y
%rax	返回值

```
    cmpq    %rsi, %rdi    # Compare x:y
    setg    %al           # Set when >
    movzbl %al, %eax      # Zero rest of %rax
    ret
```

本节内容

- 控制：条件码
- 条件分支
- 循环
- Switch 语句

跳转语句

- **jX 指令**
  - 根据条件码跳转到代码的不同部分

jX	条件	含义
jmp	1	无条件
je	ZF	相等/为零
jne	~ZF	不相等/不为零
js	SF	负数
jns	~SF	非负数
jg	~(SF^OF)&~ZF	大于（有符号）
jge	~(SF^OF)	大于等于（有符号）
jl	(SF^OF)	小于（有符号）
jle	(SF^OF)|ZF	小于等于（有符号）
ja	~CF&~ZF	大于（无符号）
jb	CF	小于（无符号）

条件分支语句（旧式风格）

- 通常情况

```
linux> gcc -Og -S -fno-if-conversion control.c
```

```
long absdiff(long x, long y)
{
  long result;
  if (x > y)
    result = x-y;
  else
    result = y-x;
  return result;
}
```

```
absdiff:
    cmpq    %rsi, %rdi   # x:y
    jle     .L4
    movq    %rdi, %rax
    subq    %rsi, %rax
    ret
.L4:                     # x <= y
    movq    %rsi, %rax
    subq    %rdi, %rax
    ret
```

寄存器	值
%rdi	变量 x
%rsi	变量 y
%rax	返回值

Goto 语句表达

- C 允许 goto 语句
- 跳转到标号指定的位置

```
long absdiff
  (long x, long y)
{
    long result;
    if (x > y)
        result = x-y;
    else
        result = y-x;
    return result;
}
```

```
long absdiff_j
  (long x, long y)
{
    long result;
    int ntest = x <= y;
    if (ntest) goto Else;
    result = x-y;
    goto Done;
 Else:
    result = y-x;
 Done:
    return result;
}
```

一般条件表达式的翻译（使用分支语句）

C 代码

```
val = Test ? Then_Expr : Else_Expr;
```

```
val = x>y ? x-y : y-x;
```

Goto 版本

```
  ntest = !Test;
  if (ntest) goto Else;
  val = Then_Expr;
  goto Done;
Else:
  val = Else_Expr;
Done:
  . . .
```

- 为 then & else 开辟各自的代码区域
- 执行适当的那部分代码

条件传送

- 条件传送指令
  - 类似于：
    `if (Test) Dest ← Src`
  - 1995年后的 x86 处理器支持
  - GCC会尝试使用
    - 但只会在确认安全时使用
- 原因
  - 指令通过流水线时，分支指令会中断指令流
  - 条件传送指令不要求控制转移

C代码

```
val = Test
    ? Then_Expr
    : Else_Expr;
```

Goto版本

```
  result = Then_Expr;
  eval = Else_Expr;
  nt = !Test;
  if (nt) result = eval;
  return result;
```

条件传送示例

```
long absdiff
  (long x, long y)
{
    long result;
    if (x > y)
        result = x-y;
    else
        result = y-x;
    return result;
}
```

```
absdiff:
    movq    %rdi, %rax  # x
    subq    %rsi, %rax  # result = x-y
    movq    %rsi, %rdx
    subq    %rdi, %rdx  # eval = y-x
    cmpq    %rsi, %rdi  # x:y
    cmovle  %rdx, %rax  # if <=, result = eval
    ret
```

寄存器	值
%rdi	变量 x
%rsi	变量 y
%rax	返回值

书写适合用条件传送实现的代码

```
1. void minmax1(long a[], long b[], long n) {
2.     long i;
3.     for (i = 0; i < n; i++) {
4.         if (a[i] > b[i]) {
5.             long t = a[i];
6.             a[i] = b[i];
7.             b[i] = t;
8.         }
9.     }
10.}
```

- 对于随机数据，**CPE 13.5**
- 对于可预测数据，**CPE 2.5–3.5**，表示预测错误惩罚大约在 **20** 个周期

书写适合用条件传送实现的代码

```
1. void minmax1(long a[], long b[], long n) {
2.     long i;
3.     for (i = 0; i < n; i++) {
4.         long min = a[i] < b[i] ? a[i] : b[i];
5.         long max = a[i] < b[i] ? b[i] : a[i];
6.         a[i] = min;
7.         b[i] = max;
8.     }
9. }
```

- **CPE** 约为 **4.0**，无论数据是任意的还是可预测的

不适合条件传送的情况

计算昂贵

```
val = Test(x) ? Hard1(x) : Hard2(x);
```

- 两个值都被计算
- 只有当计算很简单的时候才有好处

计算有风险

```
val = p ? *p : 0;
```

- 所有的值都被计算
- 可能产生意想不到的效果

计算产生副作用

```
val = x > 0 ? x*=7 : x+=3;
```

- 所有的值都被计算
- 必须是无副作用的才能使用

本节内容

- 控制：条件码
- 条件分支
- 循环
- Switch 语句

“Do-While” 循环示例

C代码

```
long pcount_do(unsigned long x) {
    long result = 0;
    do {
        result += x & 0x1;
        x >>= 1;
    } while (x);
    return result;
}
```

Goto版本

```
long pcount_goto(unsigned long x) {
    long result = 0;
loop:
    result += x & 0x1;
    x >>= 1;
    if(x) goto loop;
    return result;
}
```

- 计算变量 x 中 1 的数量
- 使用条件分支语句继续循环或退出循环

“Do-While” 循环的编译

Goto版本

```
long pcount_goto(unsigned long x) {
    long result = 0;
loop:
    result += x & 0x1;
    x >>= 1;
    if(x) goto loop;
    return result;
}
```

```
    movl    $0, %eax        #  result = 0
.L2:                        #  loop:
    movq    %rdi, %rdx
    andl    $1, %edx        #  t = x & 0x1
    addq    %rdx, %rax      #  result += t
    shrq    $1, %rdi        #  x >>= 1
    jne     .L2             #  if (x) goto loop
    rep;
    ret
```

寄存器	值
%rdi	参数 x
%rax	结果

一般 While 语句转化 #1

- “跳转到中间”的翻译
- 使用-Og 编译选项

While版本

```
while (Test)
  Body
```



Goto版本

```
  goto test;
loop:
  Body
test:
  if (Test)
    goto loop;
done:
```

While 循环示例 #1

C 代码

```
long pcount_while(unsigned long x) {
    long result = 0;
    while (x) {
        result += x & 0x1;
        x >>= 1;
    }
    return result;
}
```

跳转到中间版本

```
long pcount_goto_jtm(unsigned long x) {
    long result = 0;
    goto test;
loop:
    result += x & 0x1;
    x >>= 1;
test:
    if(x) goto loop;
    return result;
}
```

- 与该函数的 **do-while** 版本进行比较
- 第一个 **goto** 从 **test** 处开始循环

While 循环示例 #2

C代码

```
long pcount_while (unsigned long x) {
  long result = 0;
  while (x) {
    result += x & 0x1;
    x >>= 1;
  }
  return result;
}
```

Do-While版本

```
long pcount_goto_dw(unsigned long x) {
    long result = 0;
    if (!x) goto done;
loop:
    result += x & 0x1;
    x >>= 1;
    if(x) goto loop;
done:
    return result;
}
```

- 与该函数的 do-while 版本进行比较
- 最开始的条件判断控制能否进入循环

For 循环

一般形式

```
for (Init; Test; Update )
    Body
```

```
#define WSIZE 8*sizeof(int)
long pcount_for(unsigned long x)
{
    size_t i;
    long result = 0;
    for (i = 0; i < WSIZE; i++)
    {
        unsigned bit = (x >> i) & 0x1;
        result += bit;
    }
    return result;
}
```

初始化语句

```
i = 0
```

判断语句

```
i < WSIZE
```

更新语句

```
i++
```

循环主体语句

```
{
    unsigned bit = (x >> i) & 0x1;
    result += bit;
}
```

For 循环→ While 循环

For版本

```
for (Init; Test; Update )
    Body
```



While版本

```
Init;
while (Test ) {
    Body
    Update;
}
```

For-While 转化

初始化语句

```
i = 0
```

判断语句

```
i < WSIZE
```

更新语句

```
i++
```

循环主体语句

```
{
  unsigned bit = (x >> i) & 0x1;
  result += bit;
}
```

```
long pcount_for_while(unsigned long x)
{
    size_t i;
    long result = 0;
    i = 0;
    while (i < WSIZE)
    {
        unsigned bit = (x >> i) & 0x1;
        result += bit;
        i++;
    }
    return result;
}
```

For 循环 & Do-While 循环转化

C代码

```
long pcount_for(unsigned long x)
{
  size_t i;
  long result = 0;
  for (i = 0; i < WSIZE; i++)
  {
    unsigned bit =
      (x >> i) & 0x1;
    result += bit;
  }
  return result;
}
```

- 初始的判断可以被优化掉

Goto 版本

```
long pcount_for_goto_dw(unsigned long x) {
    size_t i;
    long result = 0;
    i = 0;
    if (!(i < WSIZE))                    Init
        goto done;                       !Test
loop:
    {
    unsigned bit = (x >> i) & 0x1;       Body
    result += bit;
    }
    i++;
    if (i < WSIZE)
        goto loop;                       Update
done:                                    Test
    return result;
}
```

本节内容

- 控制：条件码
- 条件分支
- 循环
- **Switch 语句**

```
long switch_eg
   (long x, long y, long z)
{
    long w = 1;
    switch(x) {
    case 1:
        w = y*z;
        break;
    case 2:
        w = y/z;
        /* Fall Through */
    case 3:
        w += z;
        break;
    case 5:
    case 6:
        w -= z;
        break;
    default:
        w = 2;
    }
    return w;
}
```

Switch 语句示例

- 多个 case 标号
  - 这里：5 & 6
- 下落的情况
  - 这里：2
- 缺失的情况
  - 这里：4

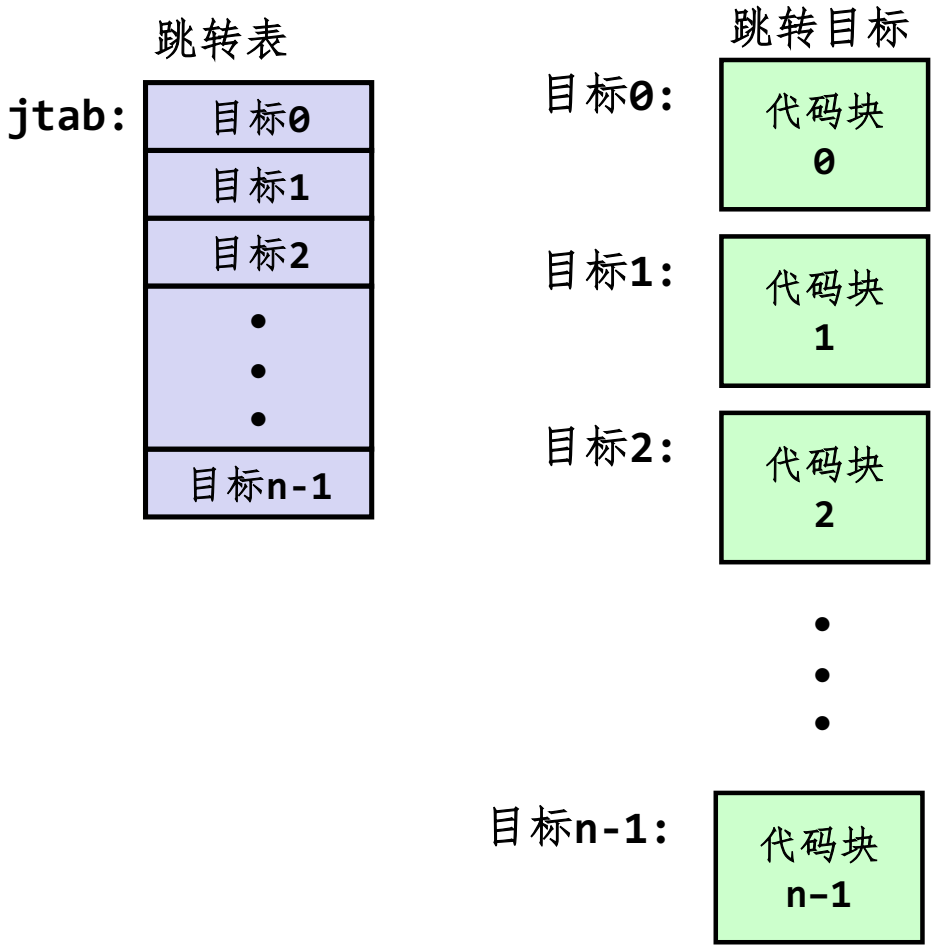
跳转表结构

Switch 形式

```
switch(x) {
  case val_0:
    Block 0
  case val_1:
    Block 1
    • • •
  case val_n-1:
    Block n-1
}
```

翻译 (扩展的 C)

```
goto *JTab[x];
```



Switch 语句示例

```
long switch_eg(long x, long y, long z)
{
    long w = 1;
    switch(x) {
      . . .
    }
    return w;
}
```

汇编代码

```
switch_eg:
    movq    %rdx, %rcx
    cmpq    $6, %rdi    # x:6
    ja      .L8
    jmp     *.L4(,%rdi,8)
```

在什么范围内的值会执行默认行为？

寄存器	值
%rdi	变量 x
%rsi	变量 y
%rdx	变量 z
%rax	返回值

注意w在这里并没有初始化

跳转语句

- **jX 指令**
  - 根据条件码跳转到代码的不同部分

jX	条件	含义
jmp	1	无条件
je	ZF	相等/为零
jne	~ZF	不相等/不为零
js	SF	负数
jns	~SF	非负数
jg	~(SF^OF)&~ZF	大于（有符号）
jge	~(SF^OF)	大于等于（有符号）
jl	(SF^OF)	小于（有符号）
jle	(SF^OF)|ZF	小于等于（有符号）
ja	~CF&~ZF	大于（无符号）
jb	CF	小于（无符号）

条件码（隐式设置）

- 单比特位寄存器
  - CF　进位标志（无符号）　SF　符号标志（有符号）
  - ZF　零标志　OF　溢出标志（有符号）

- 算术操作隐式设置（可理解为副作用）

  例如：addq Src, Dest ↔ t = a+b

  设置CF：如果最高位产生进位（无符号溢出）

  设置ZF：如果t == 0

  设置SF：如果t < 0（有符号）

  设置OF：如果补码溢出（有符号，正溢出或负溢出）

  `(a>0 && b>0 && t<0) || (a<0 && b<0 && t>=0)`

- leaq 指令并不会设置条件码

Switch 语句示例

```
long switch_eg(long x, long y, long z)
{
    long w = 1;
    switch(x) {
      . . .
    }
    return w;
}
```

准备工作：

```
switch_eg:
    movq    %rdx, %rcx
    cmpq    $6, %rdi        # x:6
    ja      .L8             # Use default
    jmp     *.L4(,%rdi,8) # goto *JTab[x]
```

间接跳转

跳转表

```
.section  .rodata
  .align 8
.L4:
  .quad   .L8  # x = 0
  .quad   .L3  # x = 1
  .quad   .L5  # x = 2
  .quad   .L9  # x = 3
  .quad   .L8  # x = 4
  .quad   .L7  # x = 5
  .quad   .L7  # x = 6
```

准备工作的汇编代码

- 表的结构
  - 每一个目标都需要8个字节
  - .L4作为基址地址

- 跳转
  - 直接跳转： **jmp .L8**
  - 跳转的目标由标号 .L8给出

  - 间接跳转：**jmp *.L4(,%rdi,8)**
  - 跳转表的起始地址：.L4
  - 间隔必须用8的倍数（地址大小是8个字节）
  - 在有效的地址 .L4 + x*8 处获取目标代码
    - 只对于 0 ≤ x ≤ 6

跳转表

```
.section .rodata
  .align 8
.L4:
  .quad  .L8 # x = 0
  .quad  .L3 # x = 1
  .quad  .L5 # x = 2
  .quad  .L9 # x = 3
  .quad  .L8 # x = 4
  .quad  .L7 # x = 5
  .quad  .L7 # x = 6
```

跳转表

跳转表

```
.section  .rodata
  .align 8
.L4:
  .quad   .L8  # x = 0
  .quad   .L3  # x = 1
  .quad   .L5  # x = 2
  .quad   .L9  # x = 3
  .quad   .L8  # x = 4
  .quad   .L7  # x = 5
  .quad   .L7  # x = 6
```

```
    switch(x) {
    case 1:       // .L3
        w = y*z;
        break;
    case 2:       // .L5
        w = y/z;
        /* Fall Through */
    case 3:       // .L9
        w += z;
        break;
    case 5:
    case 6:       // .L7
        w -= z;
        break;
    default:      // .L8
        w = 2;
    }
```

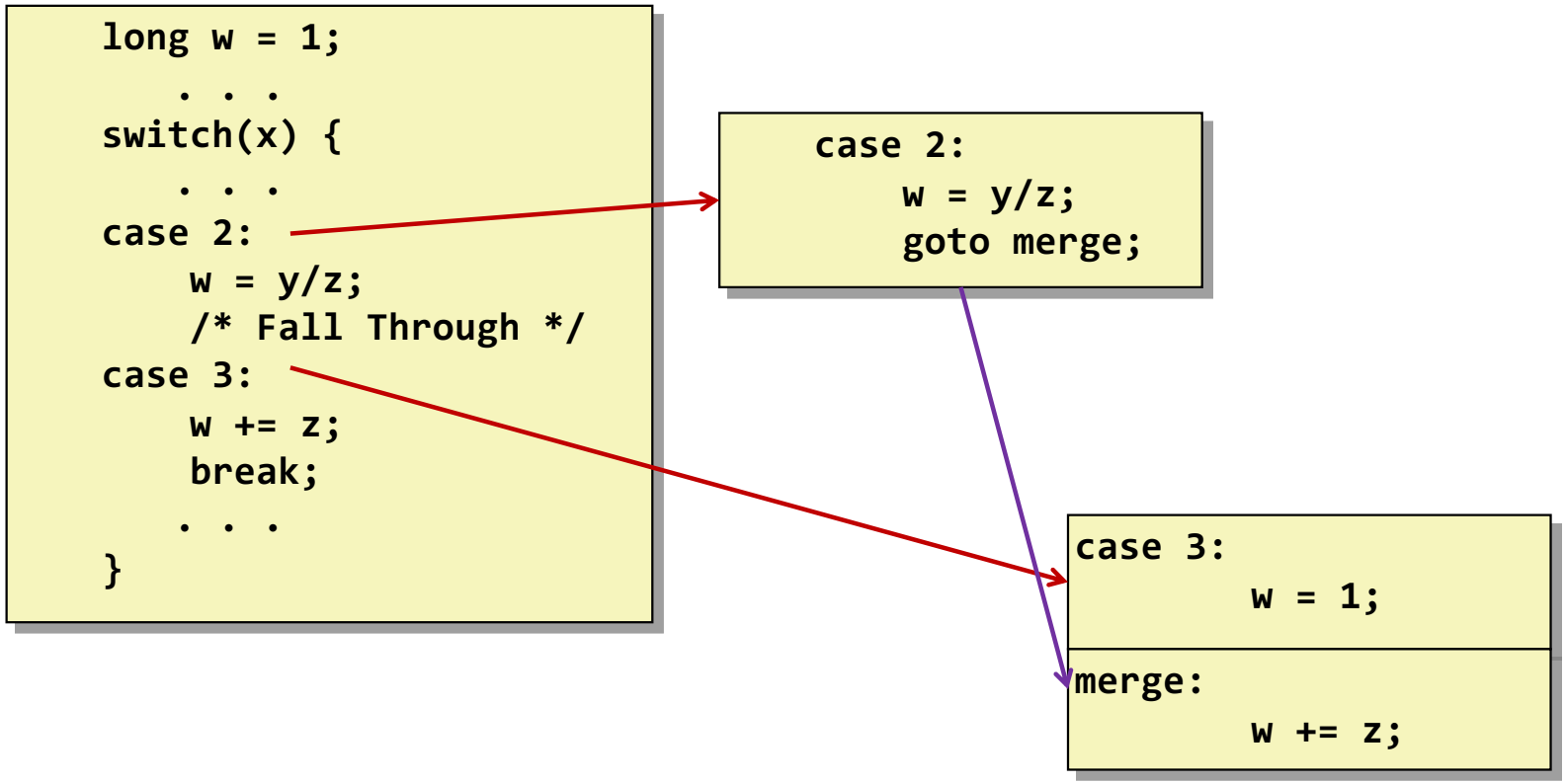
代码块 (x==1)

```
  switch(x) {
  case 1:        // .L3
        w = y*z;
        break;
    . . .
}
```

```
.L3:
   movq    %rsi, %rax  # y
   imulq   %rdx, %rax  # y*z
   ret
```

寄存器	值
%rdi	变量 x
%rsi	变量 y
%rdx	变量 z
%rax	返回值

如何解决下落的情况


long w = 1;
. . .
switch(x) {
. . .
case 2:
w = y/z;
/* Fall Through */
case 3:
w += z;
break;
. . .
}
case 2:
w = y/z;
goto merge;
case 3:
w = 1;
merge:
w += z;

代码块 (x == 2, x == 3)

```
    long w = 1;
        . . .
    switch(x) {
        . . .
    case 2:
        w = y/z;
        /* Fall Through */
    case 3:
        w += z;
        break;
        . . .
    }
```

```
.L5:                    # Case 2
   movq    %rsi, %rax
   cqto
   idivq   %rcx         #  y/z
   jmp     .L6          #  goto merge
.L9:                    # Case 3
   movl    $1, %eax     #  w = 1
.L6:                    # merge:
   addq    %rcx, %rax #  w += z
   ret
```

寄存器	值
%rdi	变量 x
%rsi	变量 y
%rdx	变量 z
%rax	返回值

代码块 (x == 5, x == 6, 默认情况)

```
  switch(x) {
    . . .
    case 5:  // .L7
    case 6:  // .L7
        w -= z;
        break;
    default: // .L8
        w = 2;
}
```

```
.L7:                    # Case 5,6
  movl  $1, %eax        #  w = 1
  subq  %rdx, %rax      #  w -= z
  ret
.L8:                    # Default:
  movl  $2, %eax        #  2
  ret
```

寄存器	值
%rdi	变量 x
%rsi	变量 y
%rdx	变量 z
%rax	返回值

总结

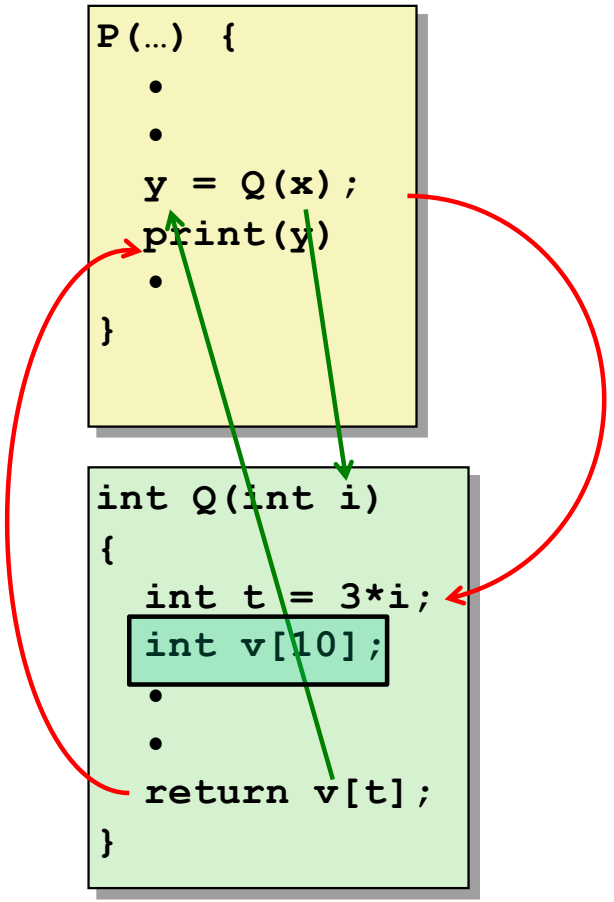
- C 控制语句
  - if-then-else
  - do-while
  - while, for
  - switch
- 汇编控制语句
  - 条件跳转
  - 条件传送
  - 间接跳转 (通过跳转表)
  - 编译器生成代码序列来实现更复杂的控制
- 标准技术
  - 循环被转换成 do-while 或跳转到中间形式
  - 较大的 switch 语句使用跳转表
  - 稀疏的 switch 语句使用决策树 (if-elseif-elseif-else)

过程：本节内容

- 过程
  - 栈的结构
  - 调用的惯例
    - 控制转移
    - 数据传送
    - 管理局部数据
  - 递归

过程调用的机制

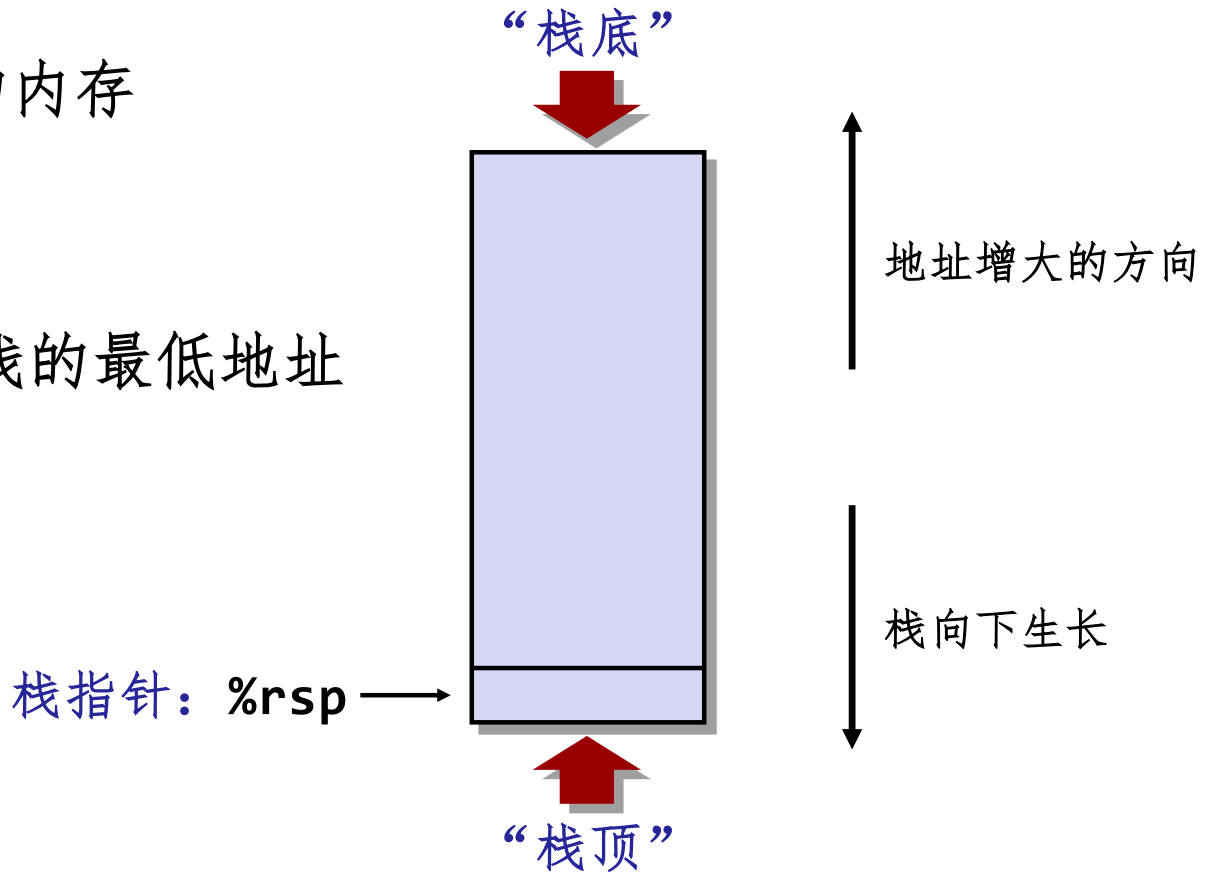
- 控制转移
  - 进入到过程代码的开始
  - 返回到返回点
- 数据传送
  - 过程参数
  - 返回值
- 内存管理
  - 程序执行时分配
  - 返回时释放
- 这些机制均由机器指令实现
- x86-64 的过程实现只使用了必须的机制



x86-64 的栈

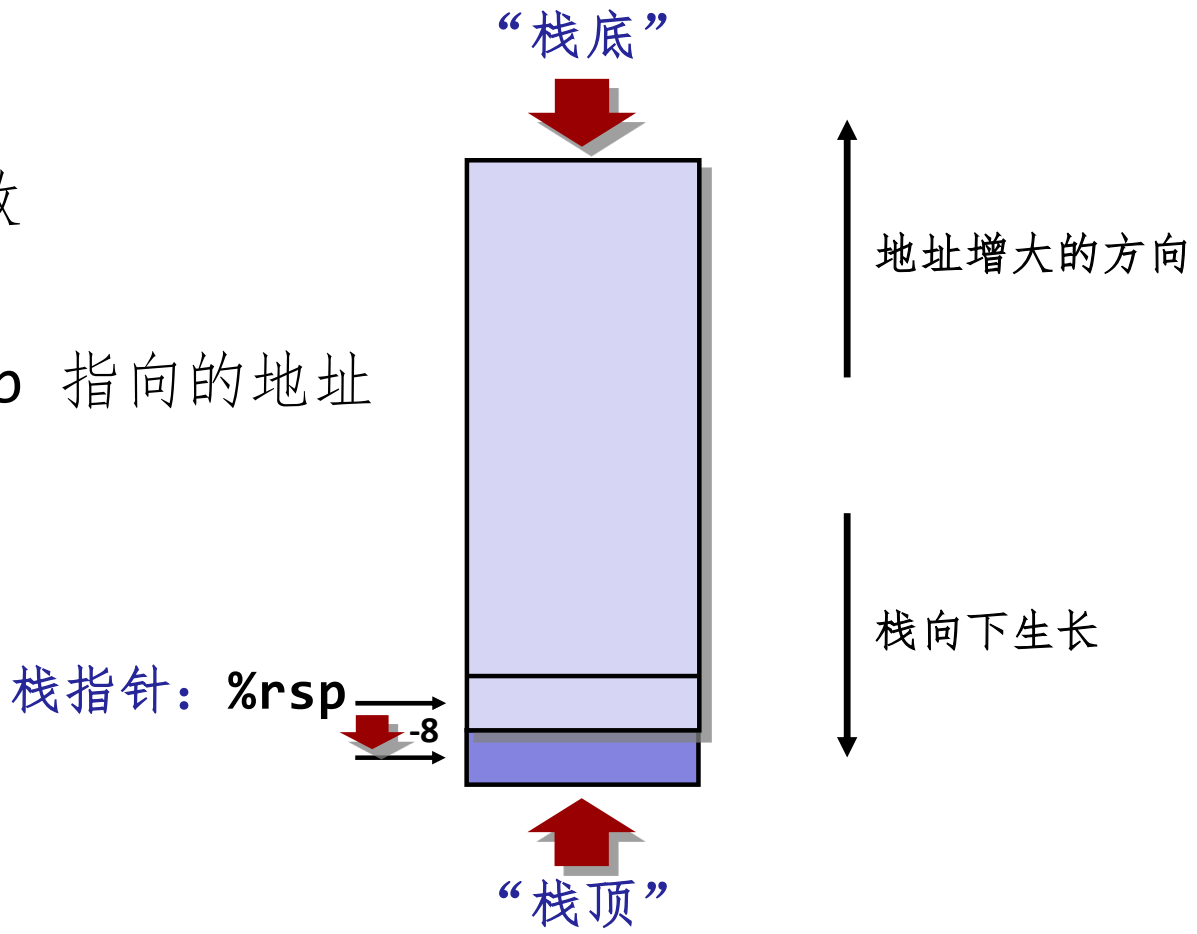
- 使用栈的规则管理的内存
- 向低地址生长

- 寄存器 %rsp 保存栈的最低地址
  - “栈顶”元素的地址



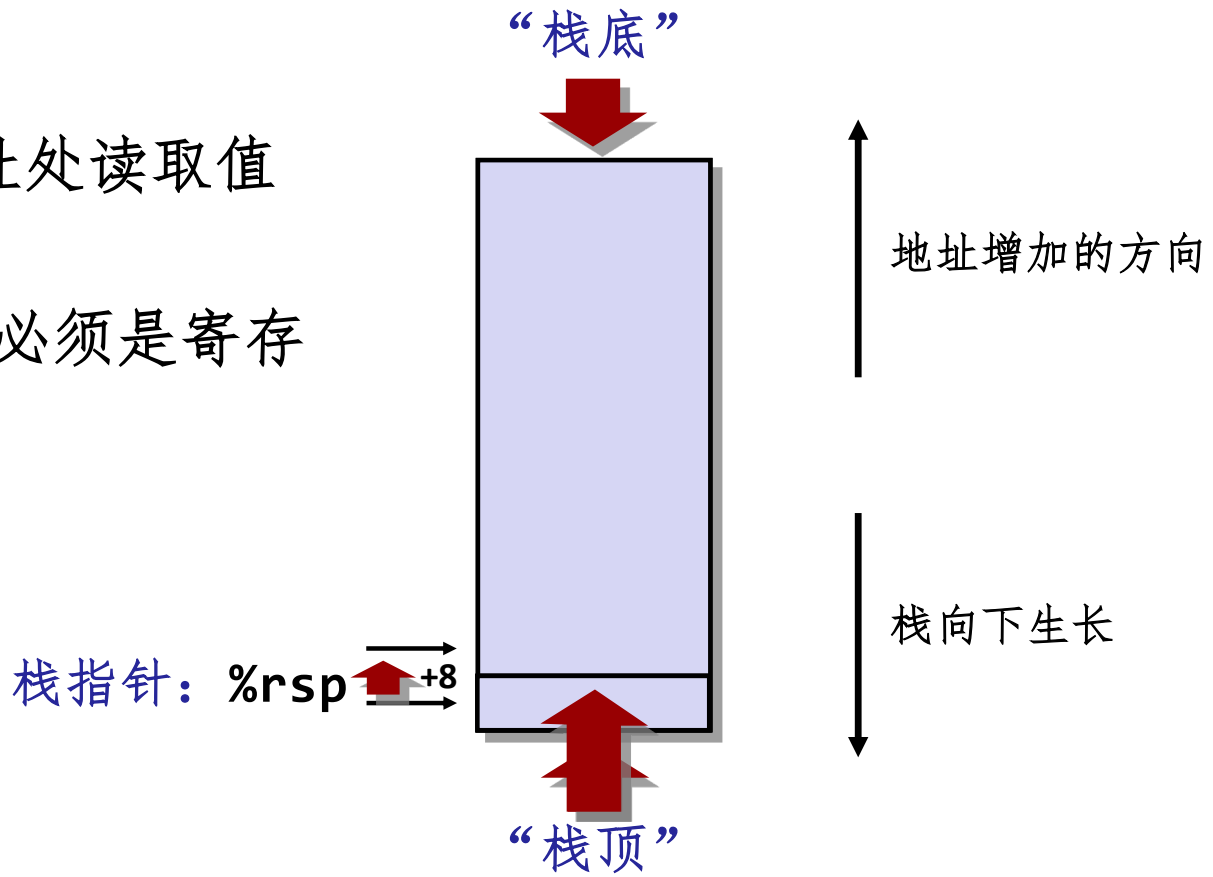
x86-64 的栈：Push

- pushq Src
  - 从 Src 获取操作数
  - %rsp 的值减8
  - 将操作数写入 %rsp 指向的地址



x86-64 的栈：Pop

- popq Dest
  - 从 %rsp 指向的地址处读取值
  - %rsp 的值加8
  - 将值存储在 Dest（必须是寄存器）中



本节内容

- 过程
  - 栈的结构
  - 调用的惯例
    - 控制转移
    - 数据传送
    - 管理局部数据
  - 递归

过程调用代码示例

```
void multstore
 (long x, long y, long *dest)
{
    long t = mult2(x, y);
    *dest = t;
}
```

```
0000000000400540 <multstore>:
  400540: push   %rbx              # Save %rbx
  400541: mov    %rdx,%rbx         # Save dest
  400544: callq  400550 <mult2>    # mult2(x,y)
  400549: mov    %rax,(%rbx)       # Save at dest
  40054c: pop    %rbx              # Restore %rbx
  40054d: retq                     # Return
```

```
long mult2
  (long a, long b)
{
  long s = a * b;
  return s;
}
```

```
0000000000400550 <mult2>:
  400550:  mov    %rdi,%rax        # a
  400553:  imul   %rsi,%rax        # a * b
  400557:  retq                    # Return
```

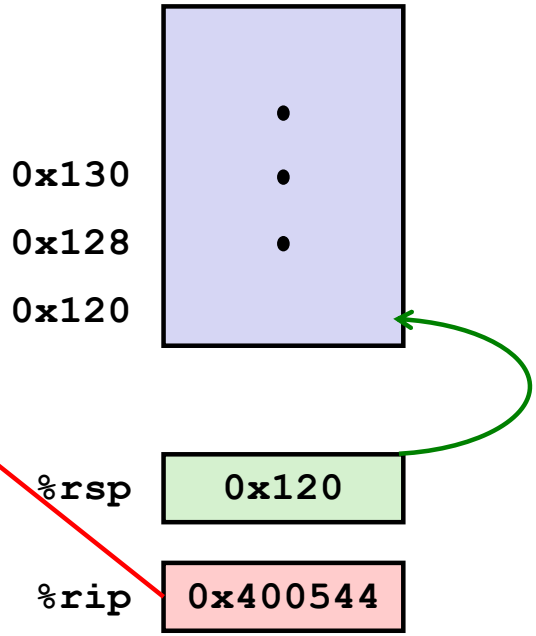
过程控制流

- 用栈支持过程调用和返回
- 过程调用：**call label**
  - 将返回地址压入栈中
  - 跳转到 label 处
- 返回地址
  - 紧接着调用后的下一条指令的地址
- 过程返回：**ret**
  - 从栈中弹出地址
  - 跳转到该地址处

控制流示例 #1

```
0000000000400540 <multstore>:
  •
  •
  400544: callq  400550 <mult2>
  400549: mov    %rax,(%rbx)
  •
  •
```

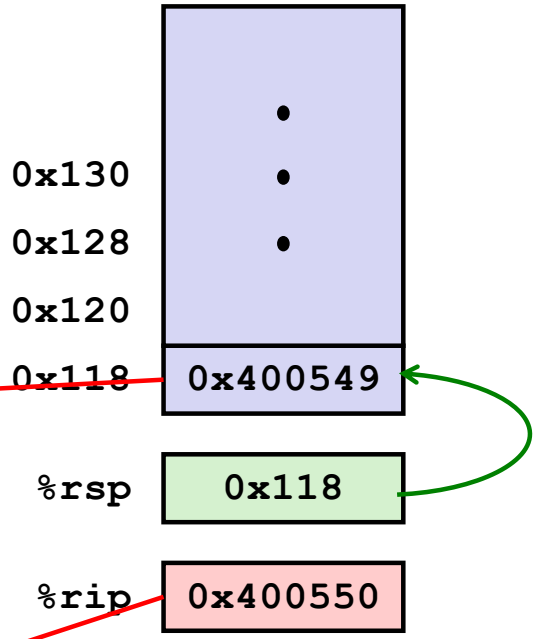
```
0000000000400550 <mult2>:
  400550:  mov    %rdi,%rax
  •
  •
  400557:  retq
```



控制流示例 #2

```
0000000000400540 <multstore>:
  •
  •
  400544: callq  400550 <mult2>
  400549: mov    %rax,(%rbx)
  •
  •
```

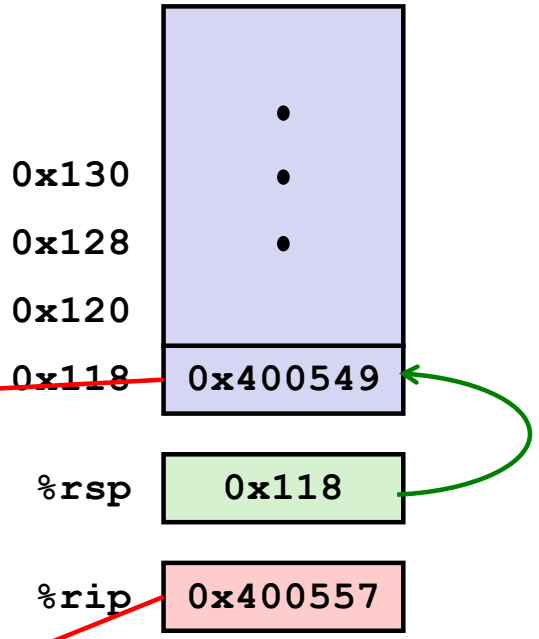
```
0000000000400550 <mult2>:
  400550:  mov    %rdi,%rax
  •
  •
  400557:  retq
```



控制流示例 #3

```
0000000000400540 <multstore>:
  •
  •
  400544: callq  400550 <mult2>
  400549: mov    %rax,(%rbx)
  •
  •
```

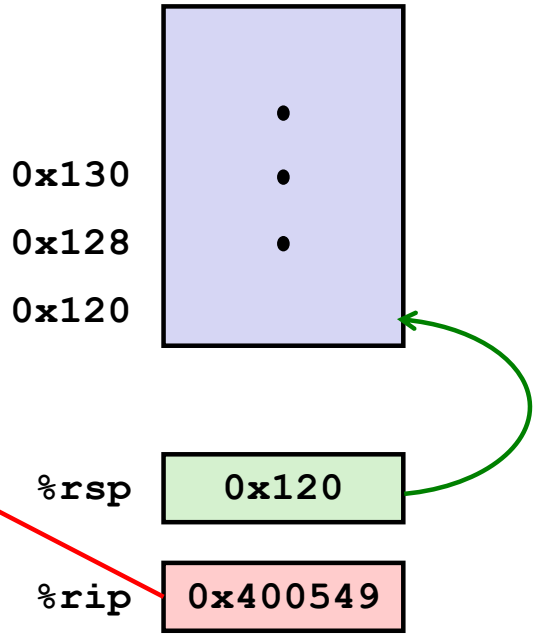
```
0000000000400550 <mult2>:
  400550:  mov    %rdi,%rax
  •
  •
  400557:  retq
```



控制流示例 #4

```
0000000000400540 <multstore>:
  •
  •
  400544: callq  400550 <mult2>
  400549: mov    %rax,(%rbx)
  •
  •
```

```
0000000000400550 <mult2>:
  400550:  mov    %rdi,%rax
  •
  •
  400557:  retq
```



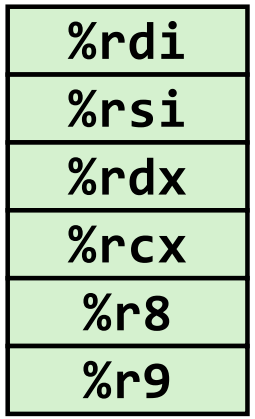
本节内容

- 过程
  - 栈的结构
  - 调用的惯例
    - 控制转移
    - 数据传送
    - 管理局部数据
  - 递归

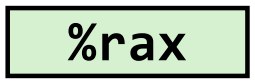
过程调用数据流

寄存器

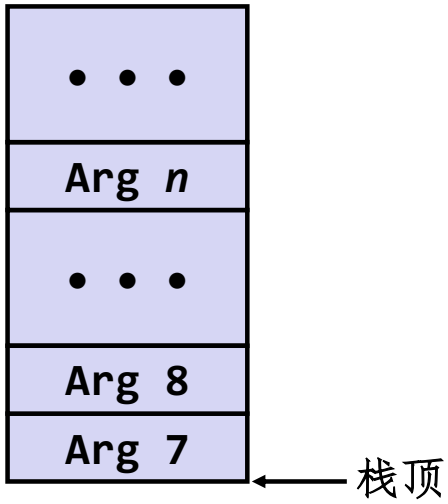
- 前6个变量



- 返回值



栈



- 通过栈传递参数时，每个数据的大小都会向上取整到8的倍数
- 只有当需要时才分配栈的空间

过程调用数据流示例

```
void multstore
 (long x, long y, long *dest)
{
    long t = mult2(x, y);
    *dest = t;
}
```

```
0000000000400540 <multstore>:
  # x in %rdi, y in %rsi, dest in %rdx
  •••
  400541: mov    %rdx,%rbx          # Save dest
  400544: callq  400550 <mult2>     # mult2(x,y)
  # t in %rax
  400549: mov    %rax,(%rbx)        # Save at dest
  •••
```

```
long mult2
  (long a, long b)
{
  long s = a * b;
  return s;
}
```

```
0000000000400550 <mult2>:
  # a in %rdi, b in %rsi
  400550:  mov    %rdi,%rax         # a
  400553:  imul   %rsi,%rax         # a * b
  # s in %rax
  400557:  retq                     # Return
```

本节内容

- 过程
  - 栈的结构
  - 调用的惯例
    - 控制转移
    - 数据传送
    - 管理局部数据
  - 递归

基于栈的语言

- 支持递归的语言
  - 例如：C, Pascal, Java
  - 代码需要是“可重入的”
    - 单个过程同时有多个实例
  - 需要空间存储每一个实例的状态
    - 参数
    - 局部变量
    - 返回地址
- 栈规则
  - 某个过程的状态只需要保存有限的时间
    - 从被调用到返回
  - 被调用者在调用者之前返回
- 栈以栈帧的形式进行分配
  - 单个过程实例的状态

斐波那契序列：

```
int fib(int n)
{
    if (n == 1 || n == 2)
        return 1;
    return fib(n-1)+fib(n-2);
}
```

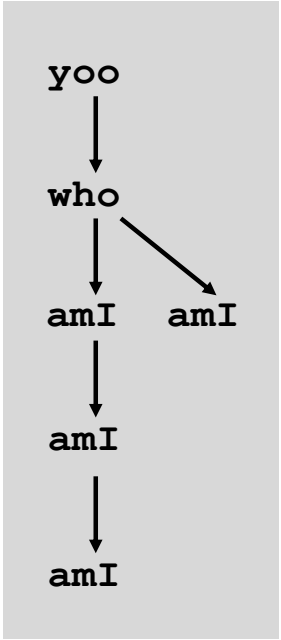
调用链示例

```
yoo(…)
{
   •
   •
   who();
   •
   •
}
```

```
who(…)
{
   • • •
   amI();
   • • •
   amI();
   • • •
}
```

```
amI(…)
{
   •
   •
   amI();
   •
   •
}
```

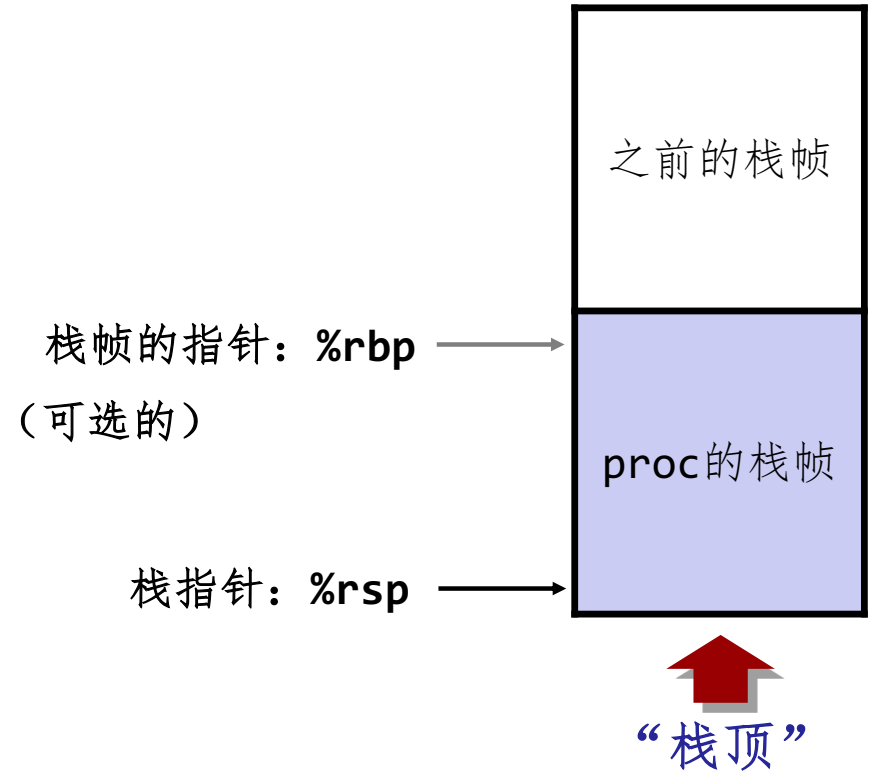
调用链



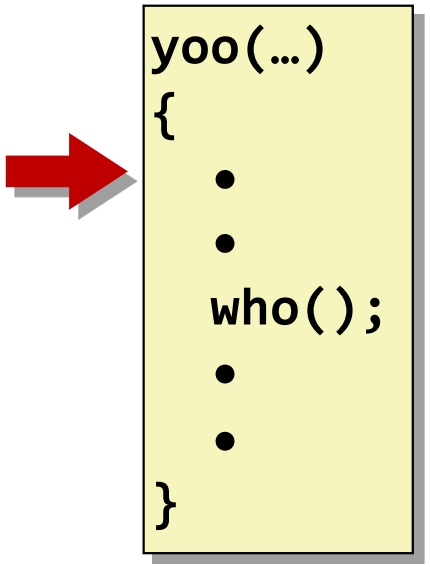
程序amI()是递归的

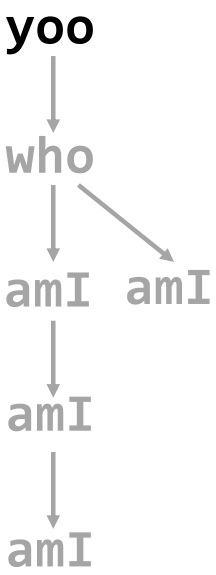
栈帧

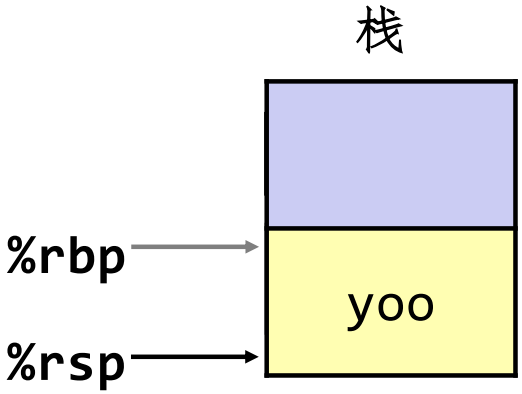
- 内容
  - 返回信息
  - 局部存储（如果需要）
  - 临时空间（如果需要）
- 管理
  - 进入过程时分配的空间
    - “准备工作” 代码
    - 包括 **call** 指令压入的数据
  - 返回时释放空间
    - “结束工作” 代码
    - 包括 **ret** 指令弹出的数据



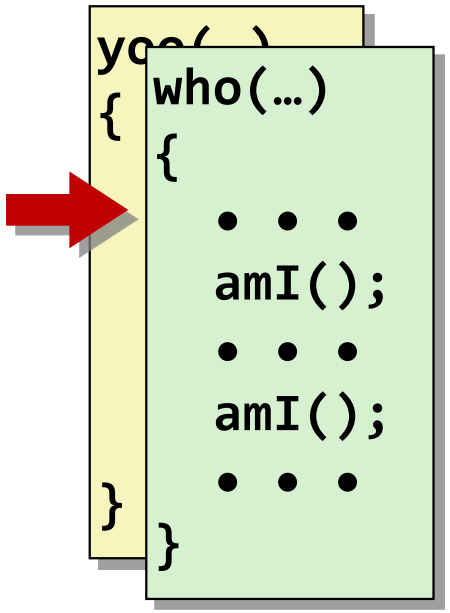
示例

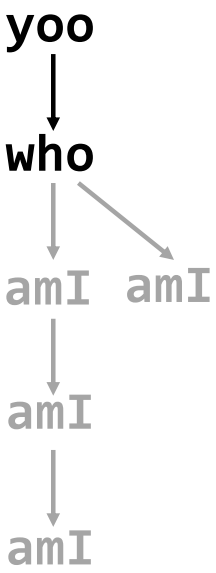


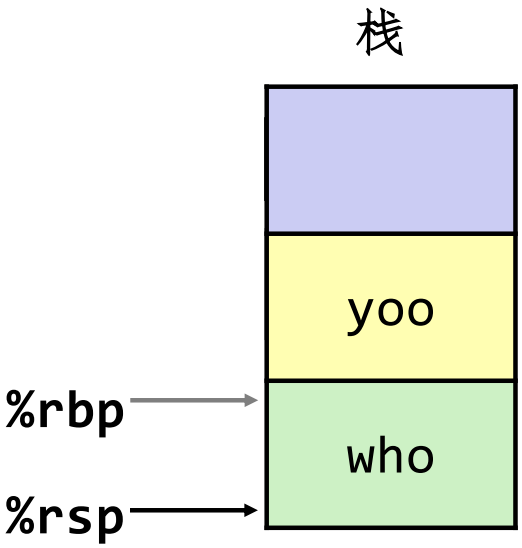




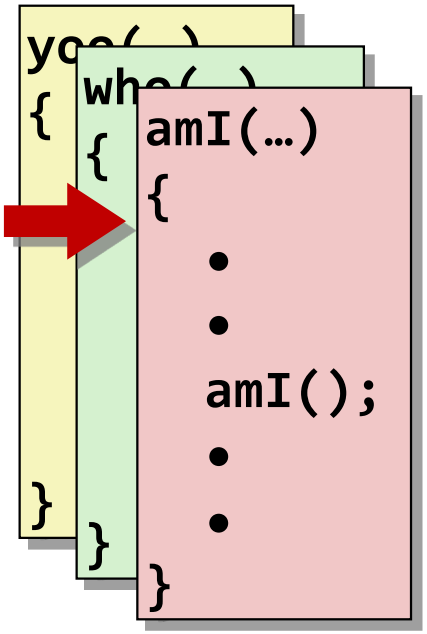
示例

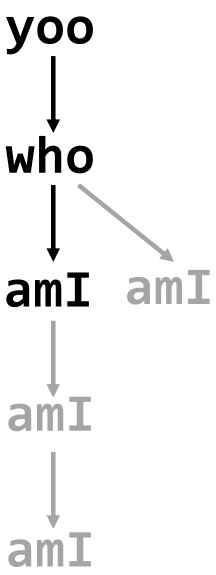


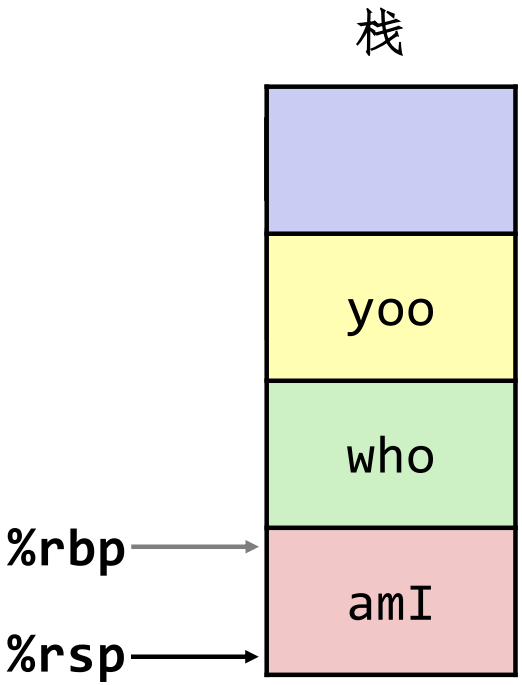




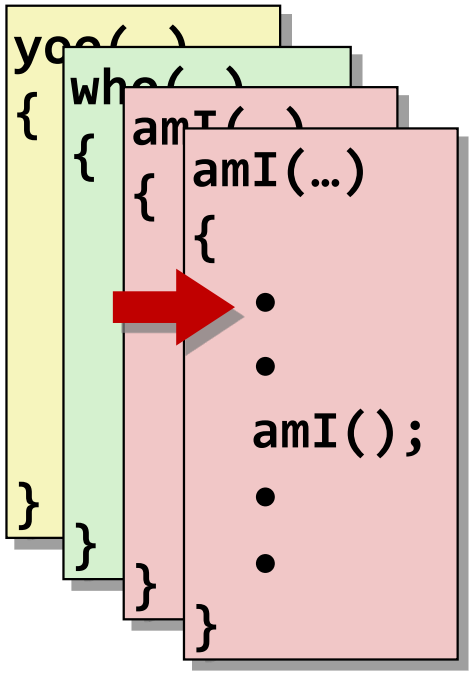
示例

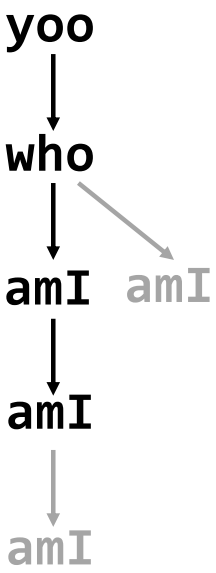


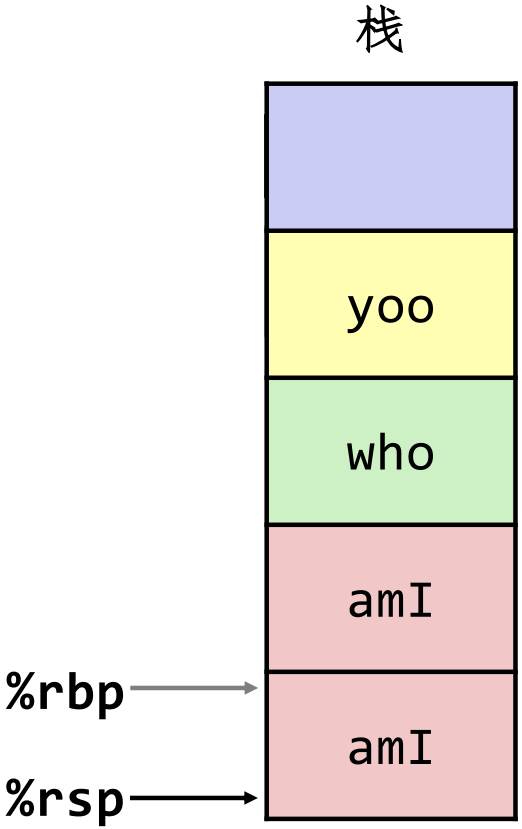




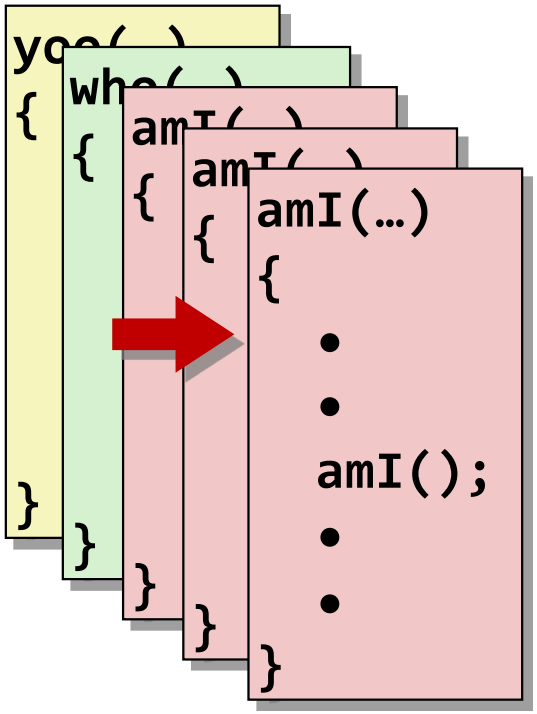
示例

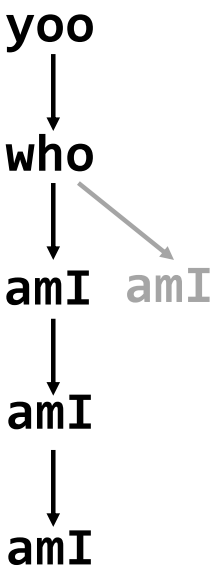


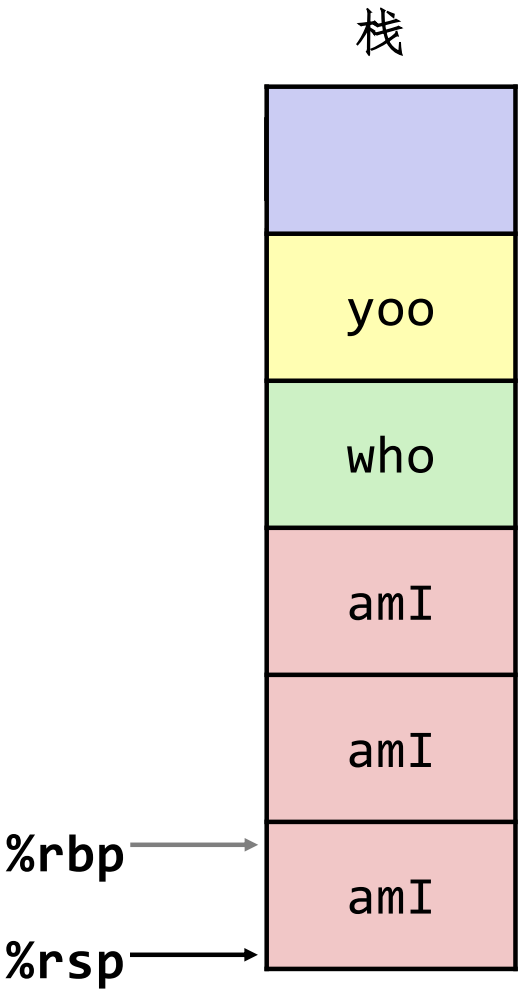




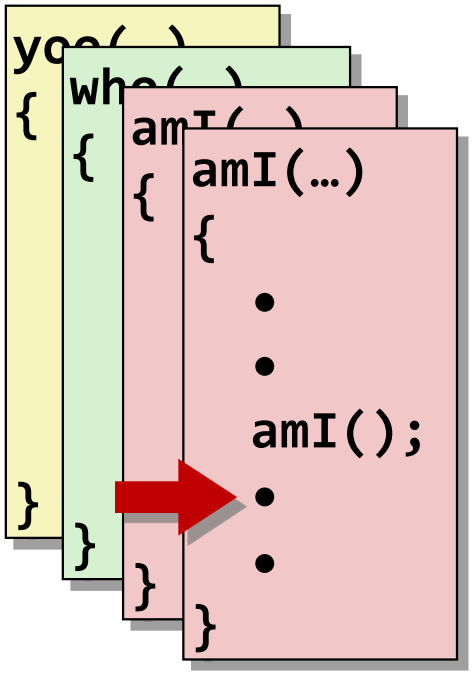
示例

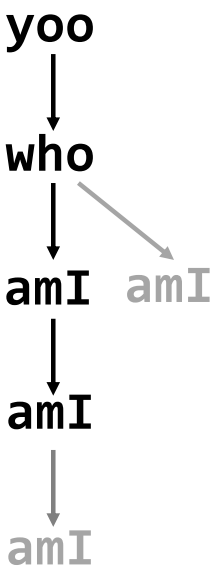


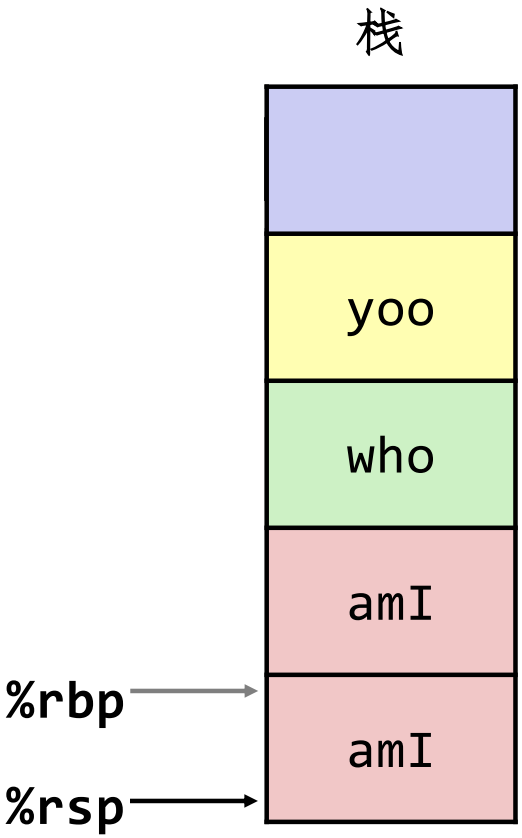




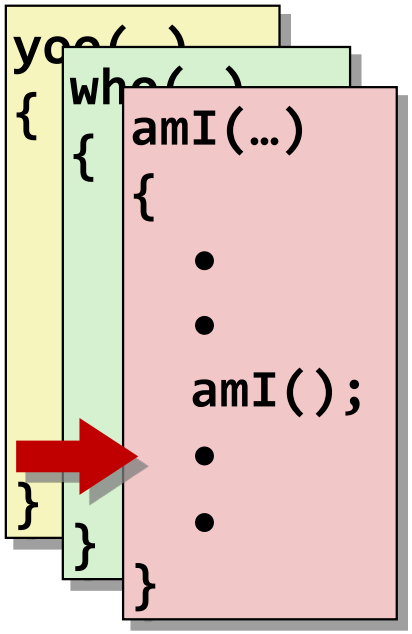
示例

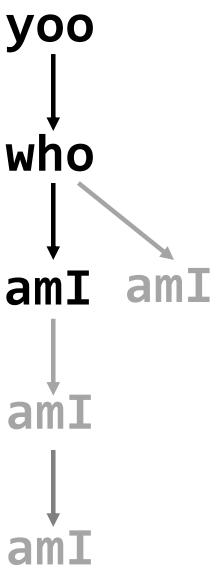


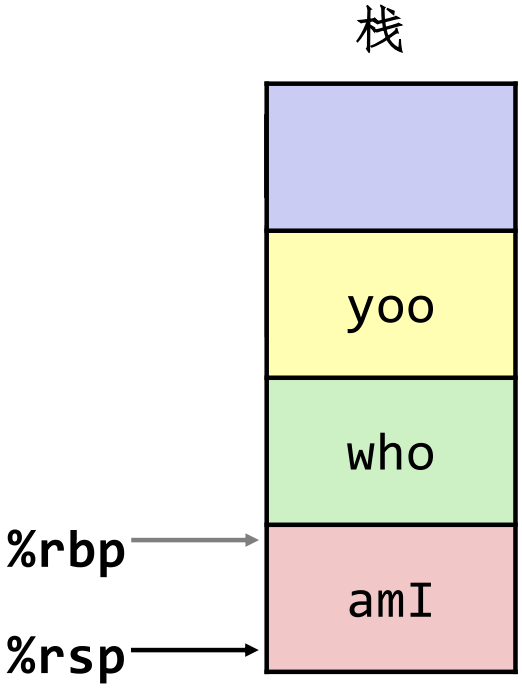




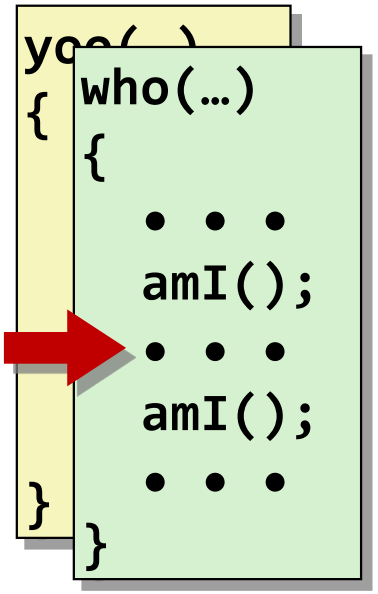
示例

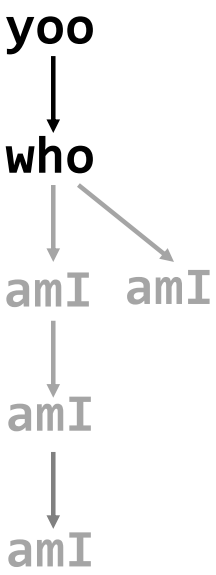


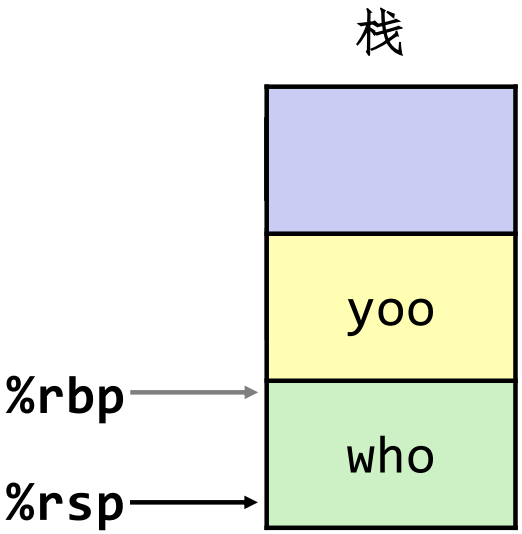




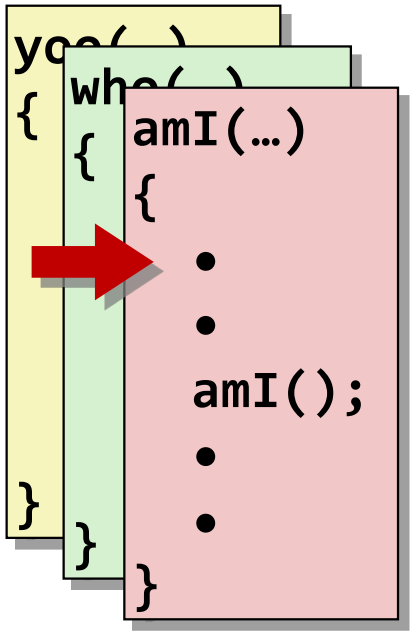
示例

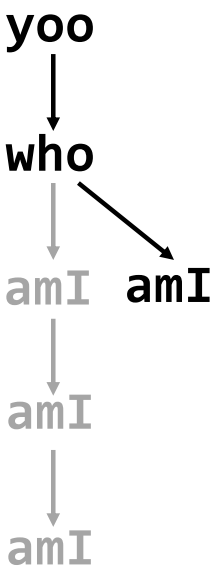


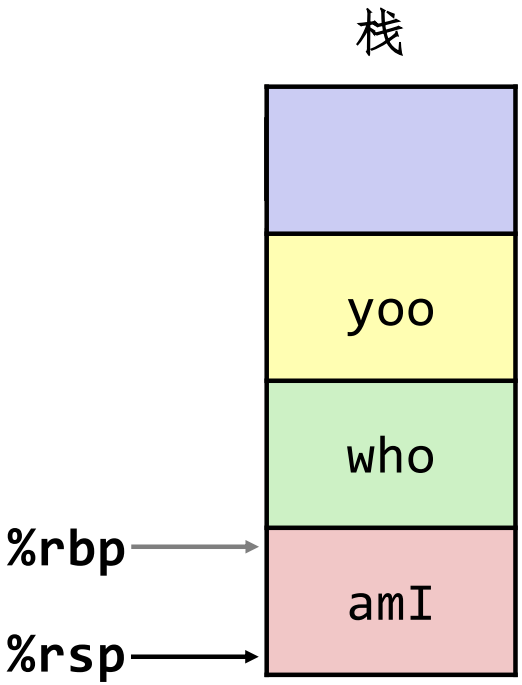




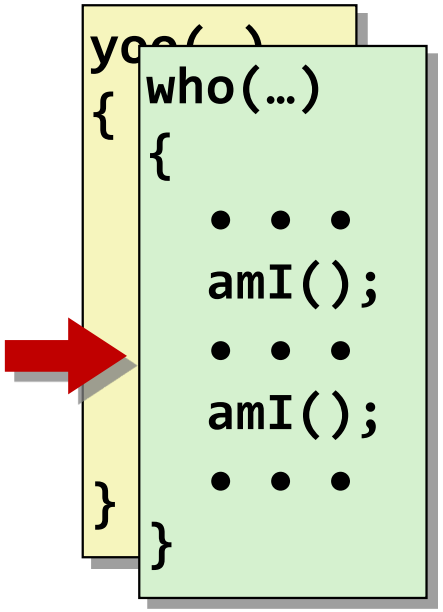
示例

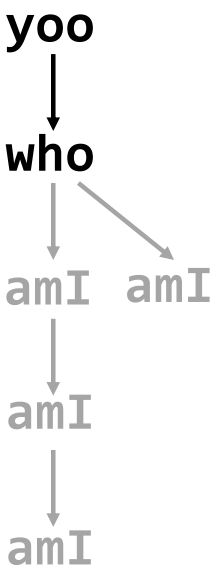


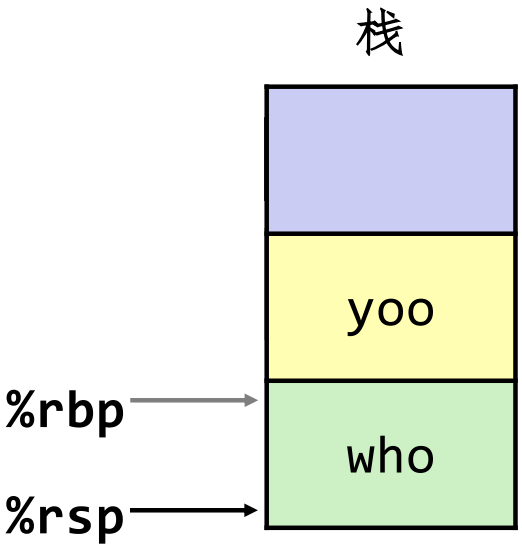




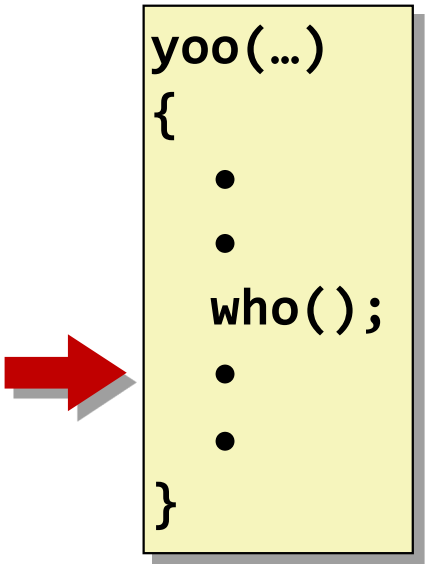
示例

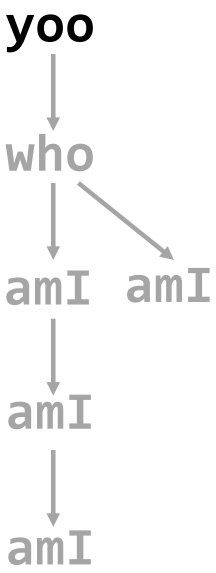


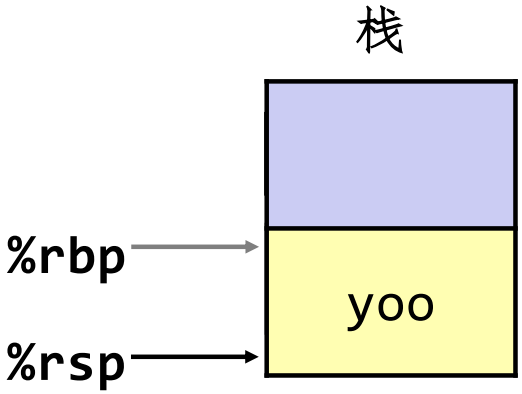




示例

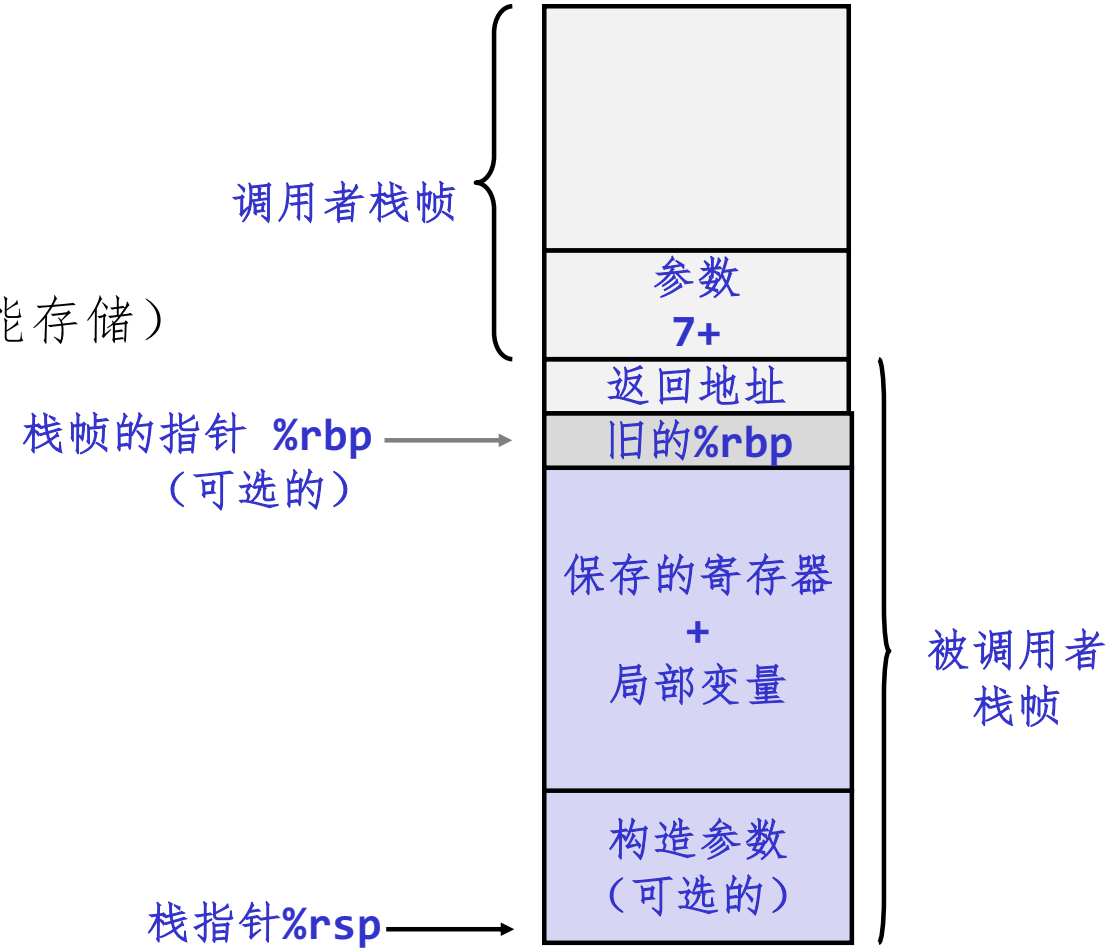






x86-64/Linux 栈帧

- 当前栈帧（从栈顶到栈底）
  - 参数构造区
    为调用的函数准备变量
  - 局部变量（如果在寄存器中不能存储）
  - 保存的寄存器
  - 旧栈帧的指针（可选择）
- 调用者的栈帧
  - 返回地址
    - 被call指令push入栈
  - 为调用函数准备的参数



示例：incr

```
long incr(long *p, long val) {
    long x = *p;
    long y = x + val;
    *p = y;
    return x;
}
```

```
incr:
  movq    (%rdi), %rax
  addq    %rax, %rsi
  movq    %rsi, (%rdi)
  ret
```

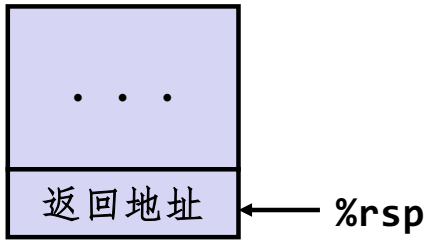
寄存器	值
%rdi	变量 p
%rsi	变量 val, y
%rax	返回值 x

示例：调用incr #1

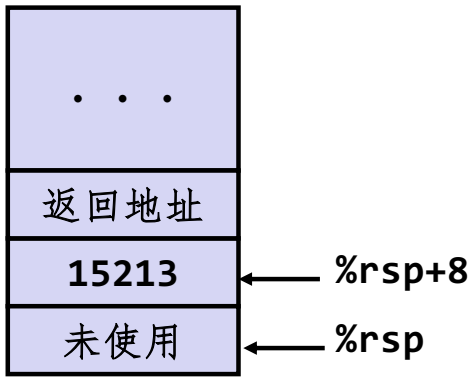
```
long call_incr() {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return v1+v2;
}
```

```
call_incr:
  subq    $16, %rsp
  movq    $15213, 8(%rsp)
  movl    $3000, %esi
  leaq    8(%rsp), %rdi
  call    incr
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

初始的栈结构



现在的栈结构

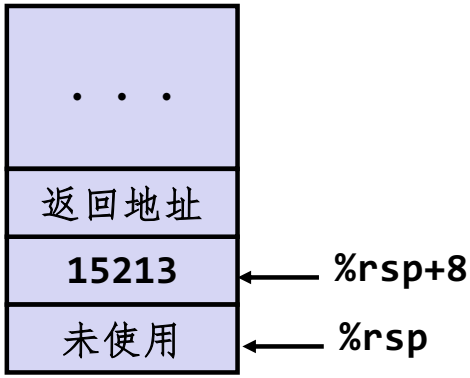


示例：调用incr #2

```
long call_incr() {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return v1+v2;
}
```

```
call_incr:
  subq    $16, %rsp
  movq    $15213, 8(%rsp)
  movl    $3000, %esi
  leaq    8(%rsp), %rdi
  call    incr
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

栈的结构



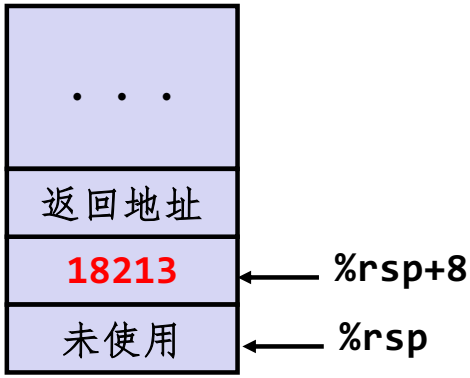
寄存器	值
%rdi	&v1
%rsi	3000

示例：调用incr #3

```
long call_incr() {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return v1+v2;
}
```

```
call_incr:
  subq    $16, %rsp
  movq    $15213, 8(%rsp)
  movl    $3000, %esi
  leaq    8(%rsp), %rdi
  call    incr
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

栈的结构



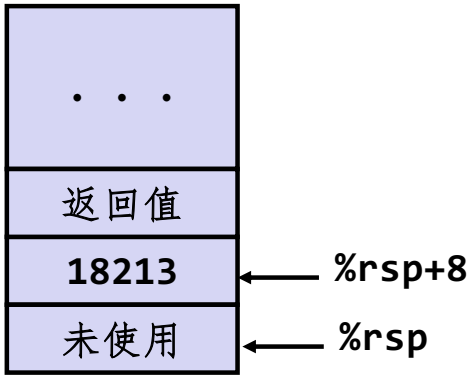
寄存器	值
%rdi	&v1
%rsi	3000

示例：调用incr #4

```
long call_incr() {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return v1+v2;
}
```

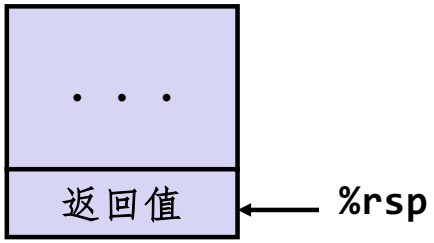
```
call_incr:
  subq    $16, %rsp
  movq    $15213, 8(%rsp)
  movl    $3000, %esi
  leaq    8(%rsp), %rdi
  call    incr
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

栈的结构



寄存器	值
%rax	返回值

更新后的栈结构

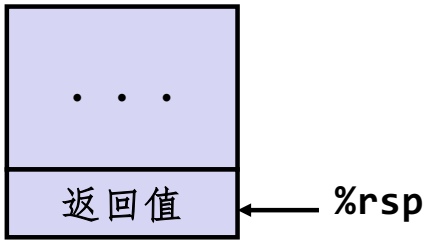


示例：调用incr #5

```
long call_incr() {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return v1+v2;
}
```

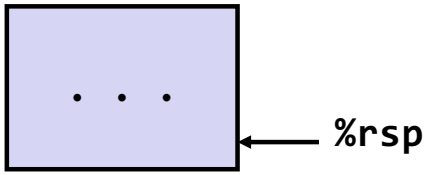
```
call_incr:
  subq    $16, %rsp
  movq    $15213, 8(%rsp)
  movl    $3000, %esi
  leaq    8(%rsp), %rdi
  call    incr
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

更新后的栈结构



寄存器	值
%rax	返回值

最后的栈结构



寄存器使用惯例

- 当程序 **yoo** 调用 **who** 时：
  - yoo 是调用者
  - who 是被调用者
- 寄存器可以用于临时存储吗？

```
yoo:
    • • •
    movq $15213, %rdx
    call who
    addq %rdx, %rax
    • • •
    ret
```

```
who:
    • • •
    subq $18213, %rdx
    • • •
    ret
```

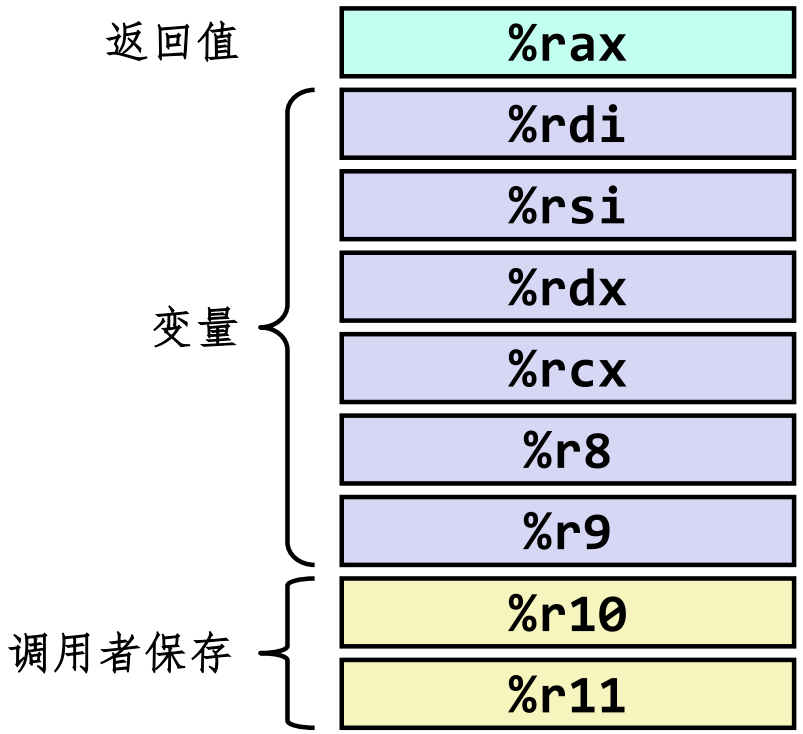
  - 寄存器 %rdx 的内容会被 who 重写
  - 这是相当麻烦的 → 我们需要采取措施！
    - 需要协调调用者和被调用者

寄存器使用惯例

- 当程序 **yoo** 调用 **who** 时：
  - yoo 是调用者
  - who 是被调用者
- 寄存器可以用于临时存储吗？
- 惯例
  - “调用者保存”
    - 函数调用前，调用者要在自己的栈帧里保存临时值
  - “被调用者保存”
    - 被调用者在使用寄存器值之前，在自己的栈帧中存储这些临时值
    - 被调用者在返回调用者之前，恢复这些值

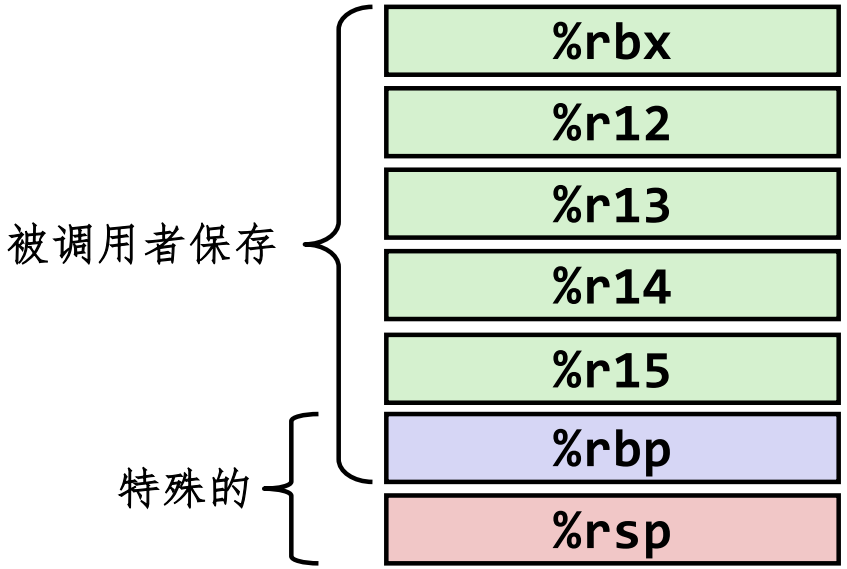
x86-64 Linux 寄存器的使用 #1

- %rax
  - 返回值
  - 调用者保存
  - 程序可以修改
- %rdi, ..., %r9
  - 变量
  - 调用者保存
  - 程序可以修改
- %r10, %r11
  - 调用者保存
  - 程序可以修改



x86-64 Linux 寄存器的保存 #2

- %rbx, %r12, %r13, %r14,%r15
  - 被调用者保存
  - 被调用者必须保存并恢复值
- %rbp
  - 被调用者保存
  - 被调用者必须保存并恢复值
  - 可以被用作帧指针
- %rsp
  - 一种特殊的被调用者保存寄存器
  - 在退出过程时恢复成原来的值

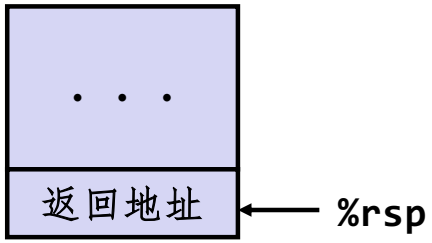


被调用者保存示例 #1

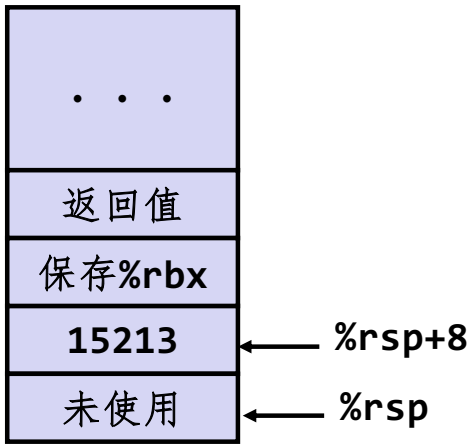
```
long call_incr2(long x) {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return x+v2;
}
```

```
call_incr2:
  pushq   %rbx
  subq    $16, %rsp
  movq    %rdi, %rbx
  movq    $15213, 8(%rsp)
  movl    $3000, %esi
  leaq    8(%rsp), %rdi
  call    incr
  addq    %rbx, %rax
  addq    $16, %rsp
  popq    %rbx
  ret
```

初始的栈结构



当前的栈结构

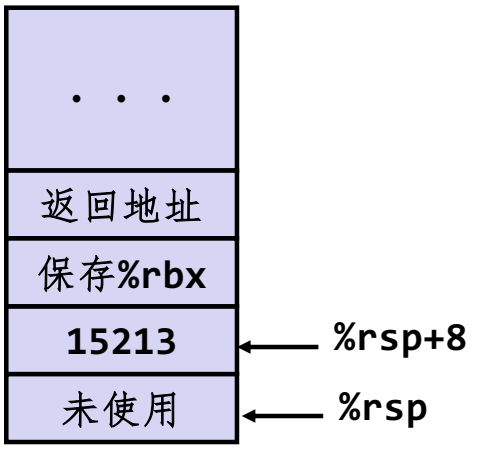


被调用者保存示例 #2

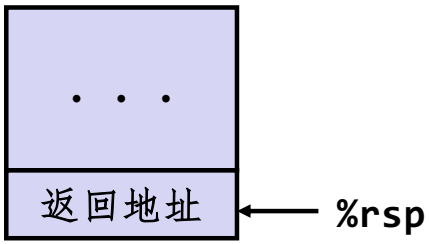
```
long call_incr2(long x) {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return x+v2;
}
```

```
call_incr2:
  pushq   %rbx
  subq    $16, %rsp
  movq    %rdi, %rbx
  movq    $15213, 8(%rsp)
  movl    $3000, %esi
  leaq    8(%rsp), %rdi
  call    incr
  addq    %rbx, %rax
  addq    $16, %rsp
  popq    %rbx
  ret
```

当前的栈结构



返回前的栈结构



本节内容

- 过程
  - 栈的结构
  - 调用的惯例
    - 控制转移
    - 数据传送
    - 管理局部数据
  - 递归

递归函数

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
  if (x == 0)
    return 0;
  else
    return (x & 1)
           + pcount_r(x >> 1);
}
```

```
pcount_r:
  movl    $0, %eax
  testq   %rdi, %rdi
  je      .L6
  pushq   %rbx
  movq    %rdi, %rbx
  andl    $1, %ebx
  shrq    %rdi
  call    pcount_r
  addq    %rbx, %rax
  popq    %rbx
.L6:
  rep; ret
```

递归函数终止情况

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
  if (x == 0)
    return 0;
  else
    return (x & 1)
           + pcount_r(x >> 1);
}
```

寄存器	值	类型
%rdi	x	参数
%rax	返回值	返回值

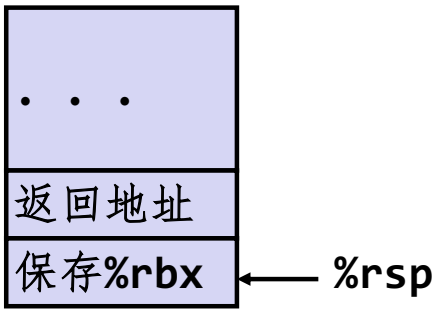
```
pcount_r:
  movl    $0, %eax
  testq   %rdi, %rdi
  je      .L6
  pushq   %rbx
  movq    %rdi, %rbx
  andl    $1, %ebx
  shrq    %rdi
  call    pcount_r
  addq    %rbx, %rax
  popq    %rbx
.L6:
  rep; ret
```

递归函数保存寄存器

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
  if (x == 0)
    return 0;
  else
    return (x & 1)
           + pcount_r(x >> 1);
}
```

寄存器	值	类型
%rdi	x	参数

```
pcount_r:
  movl    $0, %eax
  testq   %rdi, %rdi
  je      .L6
  pushq   %rbx
  movq    %rdi, %rbx
  andl    $1, %ebx
  shrq    %rdi
  call    pcount_r
  addq    %rbx, %rax
  popq    %rbx
.L6:
  rep; ret
```



递归函数调用准备

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
  if (x == 0)
    return 0;
  else
    return (x & 1)
           + pcount_r(x >> 1);
}
```

寄存器	值	类型
%rdi	x >> 1	递归参数
%rbx	x & 1	被调用者保存

```
pcount_r:
  movl    $0, %eax
  testq   %rdi, %rdi
  je      .L6
  pushq   %rbx
  movq    %rdi, %rbx
  andl    $1, %ebx
  shrq    %rdi
  call    pcount_r
  addq    %rbx, %rax
  popq    %rbx
.L6:
  rep; ret
```

递归函数调用

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
  if (x == 0)
    return 0;
  else
    return (x & 1)
           + pcount_r(x >> 1);
}
```

寄存器	值	类型
%rbx	x & 1	被调用者保存
%rax	递归调用返回值	

```
pcount_r:
  movl    $0, %eax
  testq   %rdi, %rdi
  je      .L6
  pushq   %rbx
  movq    %rdi, %rbx
  andl    $1, %ebx
  shrq    %rdi
  call    pcount_r
  addq    %rbx, %rax
  popq    %rbx
.L6:
  rep; ret
```

递归函数结果

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
  if (x == 0)
    return 0;
  else
    return (x & 1)
           + pcount_r(x >> 1);
}
```

寄存器	值	类型
%rbx	x & 1	被调用者保存
%rax	返回值	

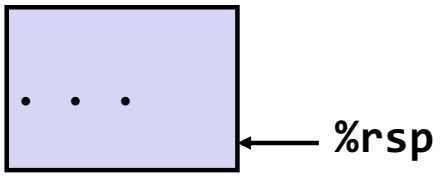
```
pcount_r:
  movl    $0, %eax
  testq   %rdi, %rdi
  je      .L6
  pushq   %rbx
  movq    %rdi, %rbx
  andl    $1, %ebx
  shrq    %rdi
  call    pcount_r
  addq    %rbx, %rax
  popq    %rbx
.L6:
  rep; ret
```

递归函数完成

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
  if (x == 0)
    return 0;
  else
    return (x & 1)
           + pcount_r(x >> 1);
}
```

寄存器	值	类型
%rax	返回值	返回值

```
pcount_r:
  movl    $0, %eax
  testq   %rdi, %rdi
  je      .L6
  pushq   %rbx
  movq    %rdi, %rbx
  andl    $1, %ebx
  shrq    %rdi
  call    pcount_r
  addq    %rbx, %rax
  popq    %rbx
.L6:
  rep; ret
```

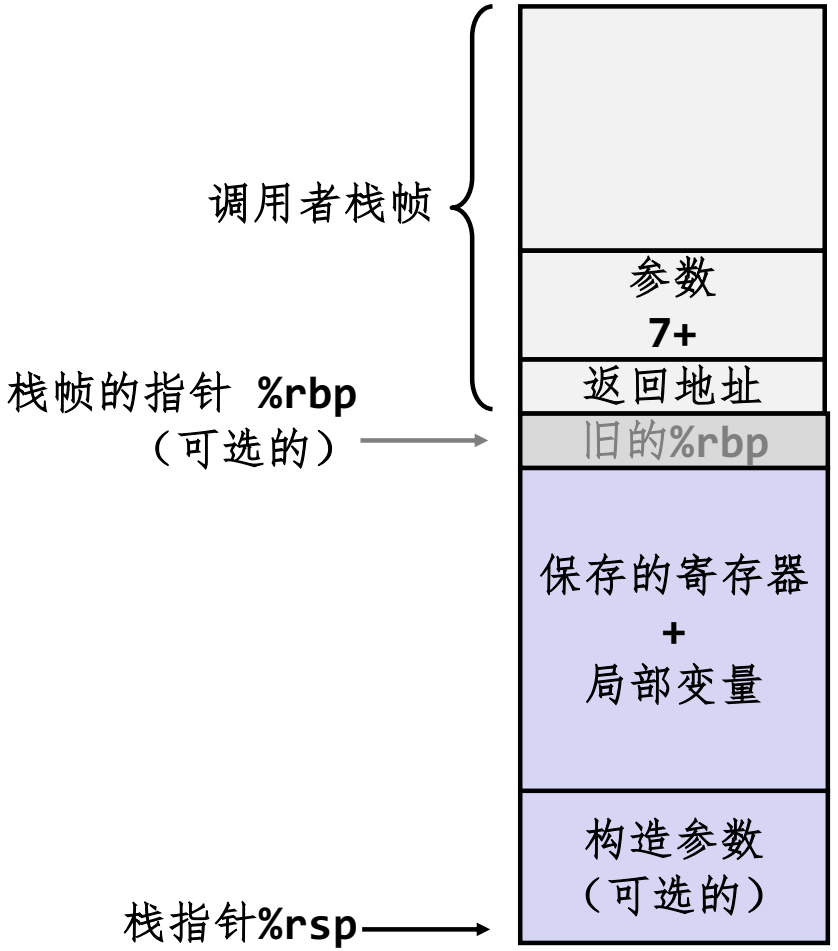


有关递归的观察

- 对递归不需要特殊处理
  - 栈帧意味着每一次函数调用都有私有的空间
    - 保存的寄存器和局部变量
    - 保存的返回地址
  - 寄存器保存惯例阻止一次函数调用篡改另一次调用的数据
    - 除非 C 代码显式地这样做（例如，后面会谈到的缓冲区溢出）
  - 栈规则与过程调用/返回模式保持一致
    - 如果 P 调用 Q，那么 Q 在 P 之前返回
    - 后进先出
- 相互递归适用
  - P调用Q；Q调用P

x86-64 过程总结

- 重点
  - 栈是保存过程调用、返回的不二之选
    - 如果 P 调用 Q，则 Q 在 P 之前返回
- 一般的调用惯例就解决了递归（相互递归）
  - 可以安全地把数据存储在局部栈帧和被调用者保存寄存器中
  - 参数置于栈顶
  - 返回值放在%rax
- 指针是值的地址
  - 值存储在栈上或为全局变量



数据：主要内容

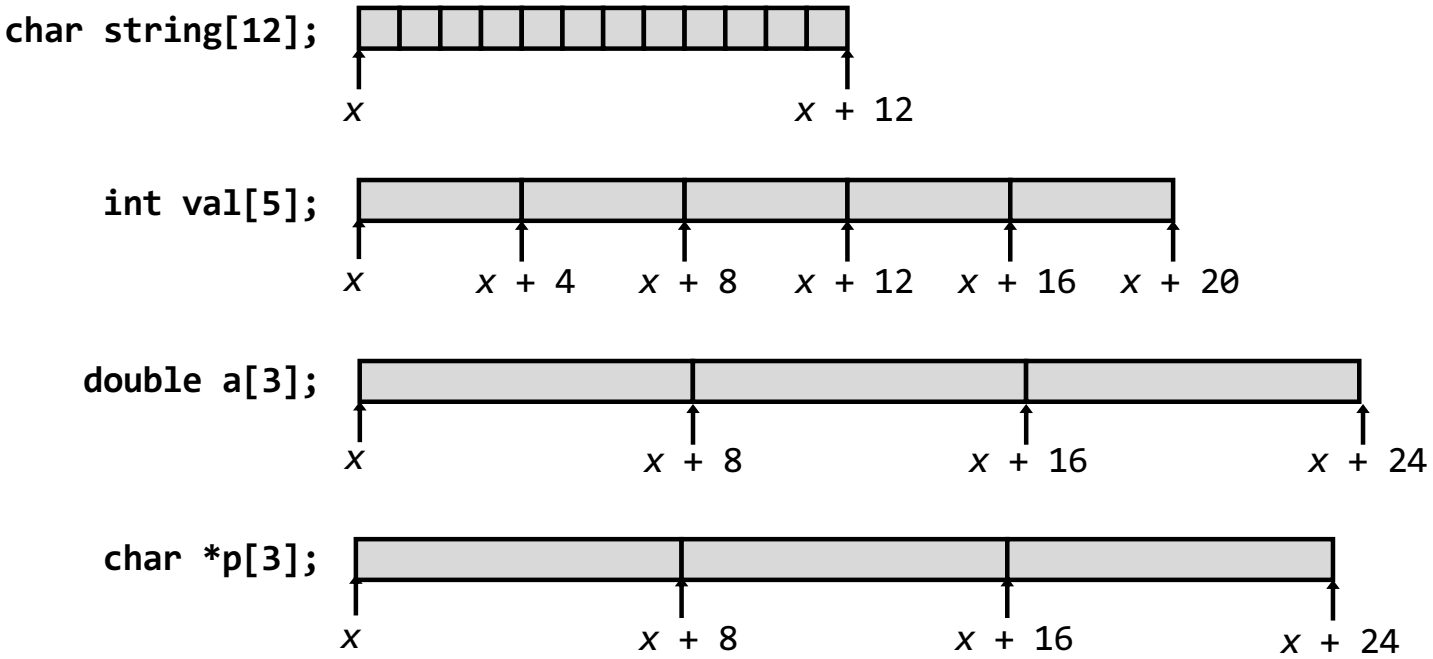
- 数组
  - 一维数组
  - 多维数组（嵌套）
  - 变长数组
- 结构体
  - 分配
  - 存取
  - 数据对齐
- 联合体

数组空间分配

- 基本原理

  T **A[***L***];**

  - 数据类型为 *T* 长度为 *L*
  - 在内存中连续地分配 *L* * **sizeof(***T***)** 字节大小的空间



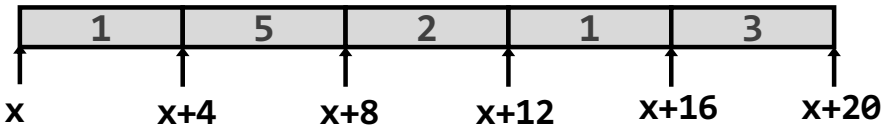
数组的访问

- 基本原则

  T **A[***L***];**

  - 数据类型为 *T* 长度为 *L*
  - 标识符 **A** 可以被用作指向数组第0个元素的指针：类型 *T**

int val[5];



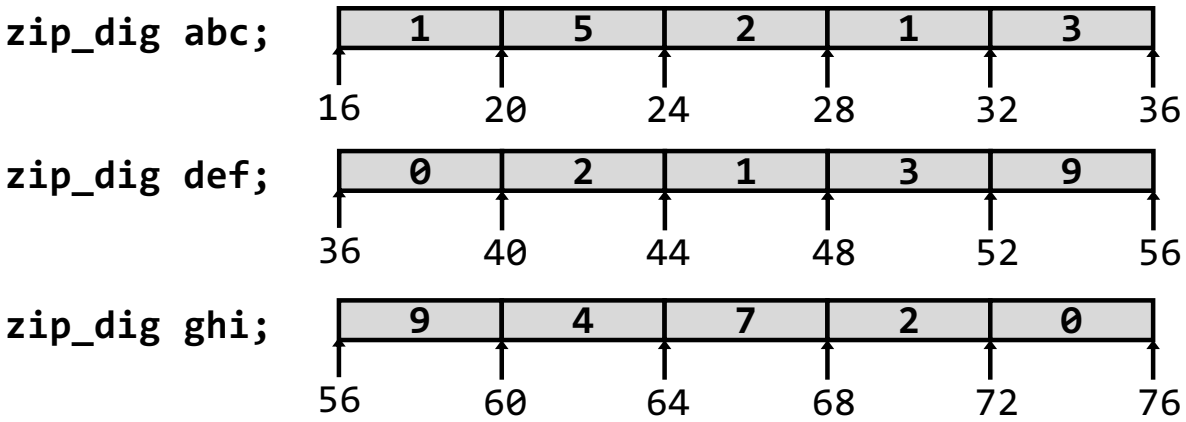
- 示例

	类型	值
val[4]	int	3
val	int *	x
val+1	int *	x + 4
&val[2]	int *	x + 8
val[5]	int	??
*(val+1)	int	5 //val[1]
val + i	int *	x + 4 * i //&val[i]

数组示例

```
#define ZLEN 5
typedef int zip_dig[ZLEN];

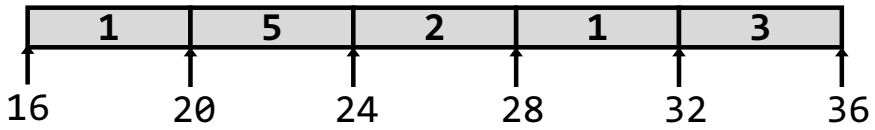
zip_dig abc = { 1, 5, 2, 1, 3 };
zip_dig def = { 0, 2, 1, 3, 9 };
zip_dig ghi = { 9, 4, 7, 2, 0 };
```



- 声明 “zip_dig abc” 等价于 “int abc[5]”
- 三个示例数组被分配到三个连续的20字节内存块中
  - 一般来说，不保证会这样分配

数组的访问示例

```
zip_dig abc;
```




```
int get_digit(zip_dig z, int digit)
{
  return z[digit];
}
```

```
  # %rdi = z
  # %rsi = digit
  movl (%rdi,%rsi,4), %eax  # z[digit]
```

- 寄存器 **%rdi** 包含数组的起始地址
- 寄存器 **%rsi** 包含数组索引
- 期望的数据位于 **%rdi + 4*%rsi**
- 使用内存引用 **(%rdi,%rsi,4)**

数组循环示例

```
void zincr(zip_dig z) {
  size_t i;
  for (i = 0; i < ZLEN; i++)
    z[i]++;
}
```

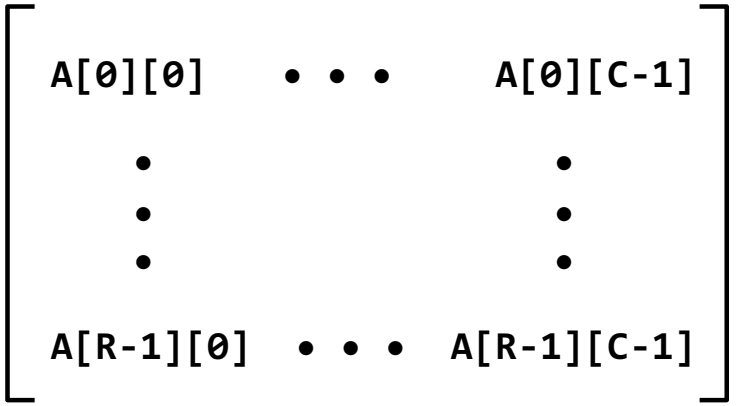
```
  # %rdi = z
  movl    $0, %eax            #   i = 0
  jmp     .L3                 #   goto middle
.L4:                          # loop:
  addl    $1, (%rdi,%rax,4)   #   z[i]++
  addq    $1, %rax            #   i++
.L3:                          # middle
  cmpq    $4, %rax            #   i:4
  jbe     .L4                 #   if <=, goto loop
  rep; ret
```

多维（嵌套）数组

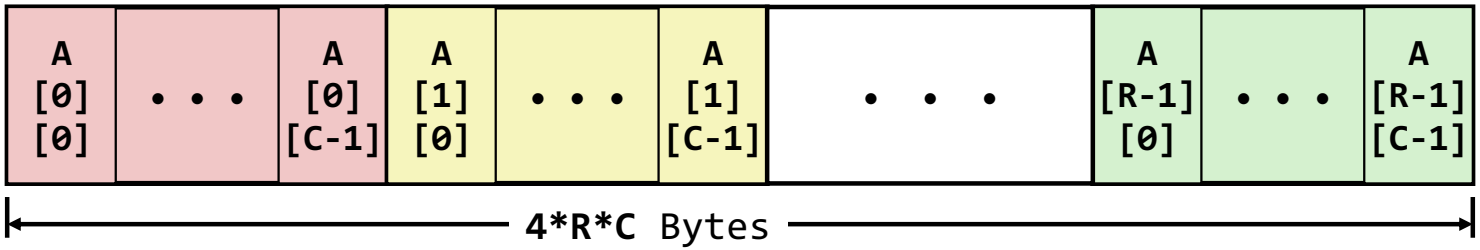
- 声明

  T **A**[*R*][*C*];

  - 数据类型为 *T* 的二维数组
  - *R* 行，*C* 列
- 数组大小
  - *R* * *C* * **sizeof**(*T*) bytes
- 排列
  - 按行优先排序



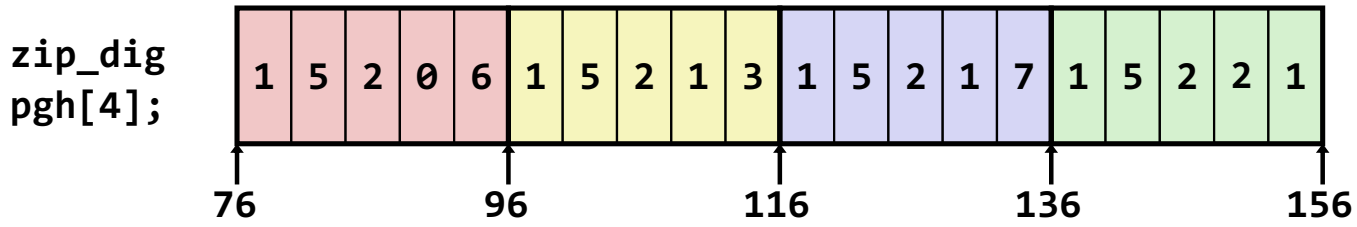
int A[R][C];



数组嵌套示例

```
#define PCOUNT 4
typedef int zip_dig[5];

zip_dig pgh[PCOUNT] =
  {{1, 5, 2, 0, 6},
   {1, 5, 2, 1, 3 },
   {1, 5, 2, 1, 7 },
   {1, 5, 2, 2, 1 }};
```

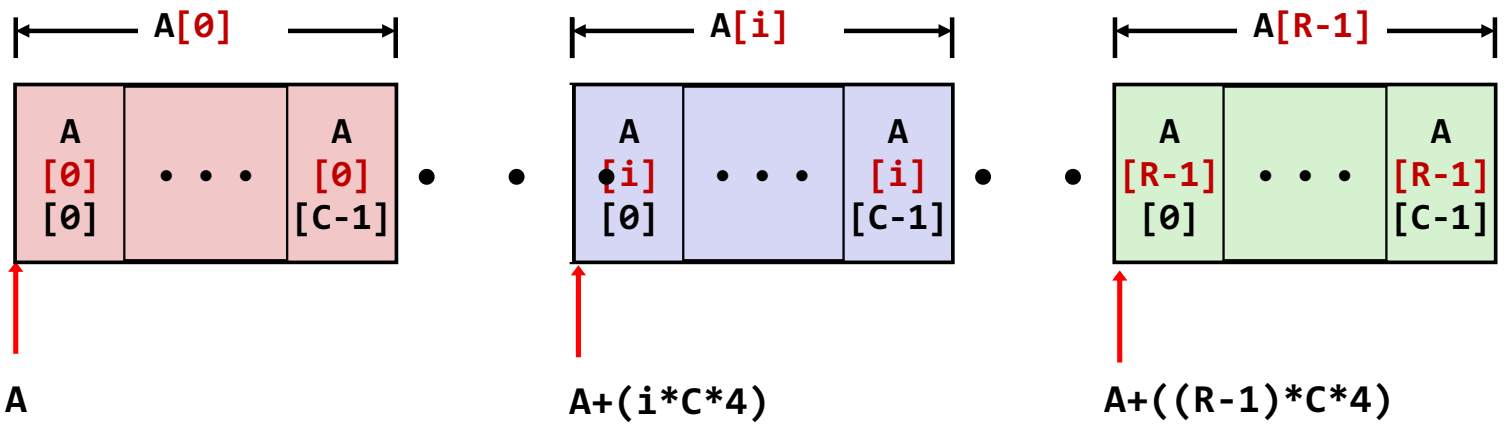


- “zip_dig pgh[4]” 等价于 “int pgh[4][5]”
  - 变量 pgh: 4个元素（20字节/元素）组成的数组，连续分配
  - 每个元素都是一个5个整型（4个字节/元素）的数组，连续分配
- 内存中所有的元素按“行优先”排序

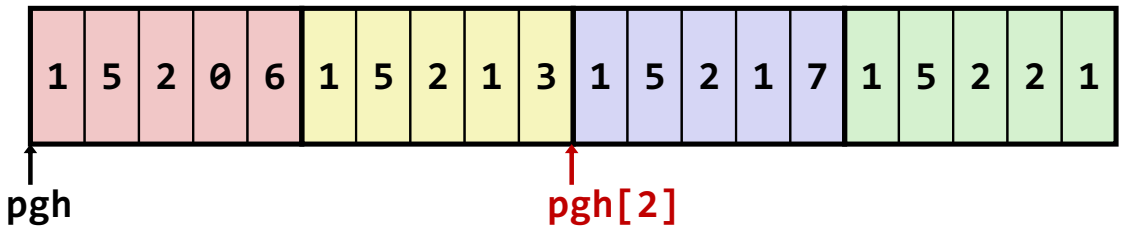
嵌套数组的行访问

- 多维数组中的行向量
  - **A[i]** 是类型为 *T* 的 *C* 个元素的数组
  - 起始地址为 **A + i * (C * sizeof(*T*))**

```
int A[R][C];
```



嵌套数组行访问代码




```
int *get_pgh_zip(int index)
{
  return pgh[index];
}
```

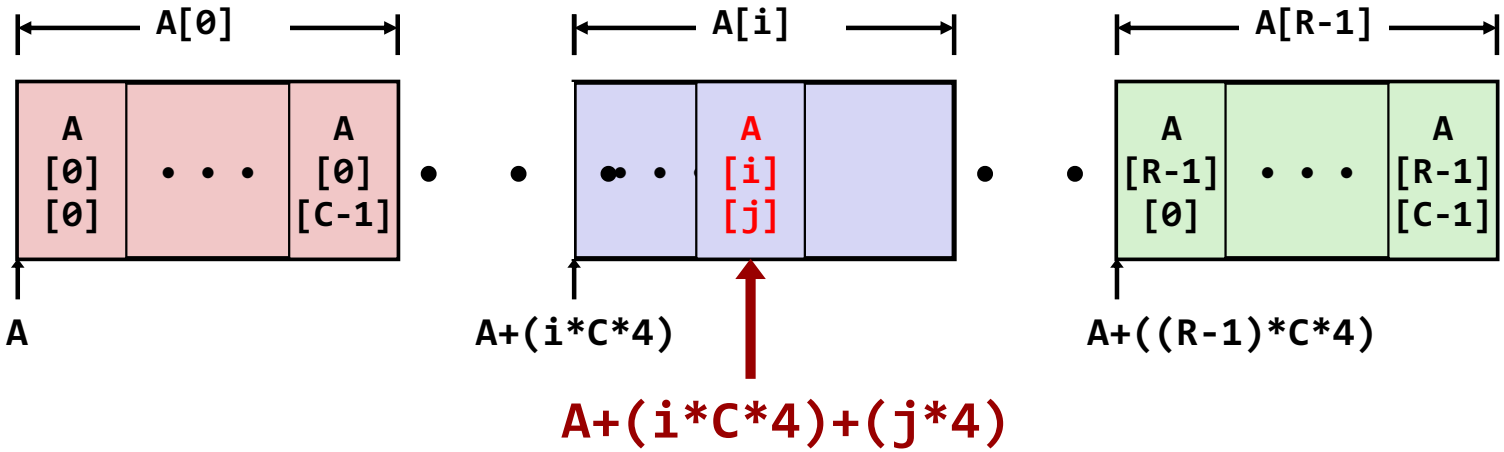
```
# %rdi = index
 leaq (%rdi,%rdi,4),%rax    # 5 * index
 leaq pgh(,%rax,4),%rax     # pgh + (20 * index)
```

- 行向量
  - **pgh[index]** 是一个 5 个整型数据的数组
  - 起始地址 **pgh+20*index**
- 机器码
  - 计算并返回地址
  - 计算 **pgh + 4*(index+4*index)**

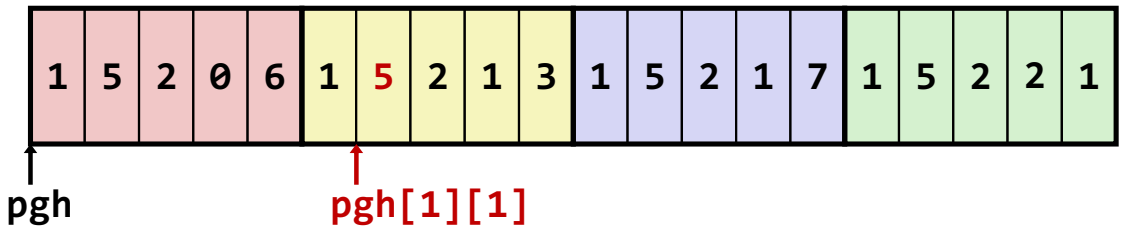
嵌套数组元素访问

- 数组元素
  - A[i][j] 是类型为 *T* 的元素，每个元素需要 *K* 个字节
  - A[i][j] 地址是 A + i * (C * K) + j * K
    = A + (i * C + j) * K

```
int A[R][C];
```



嵌套数组元素访问代码




```
int get_pgh_digit(int index, int dig)
{
  return pgh[index][dig];
}
```

```
leaq    (%rdi,%rdi,4), %rax     # 5*index
addl    %rax, %rsi              # 5*index+dig
movl    pgh(,%rsi,4), %eax      # M[pgh + 4*(5*index+dig)]
```

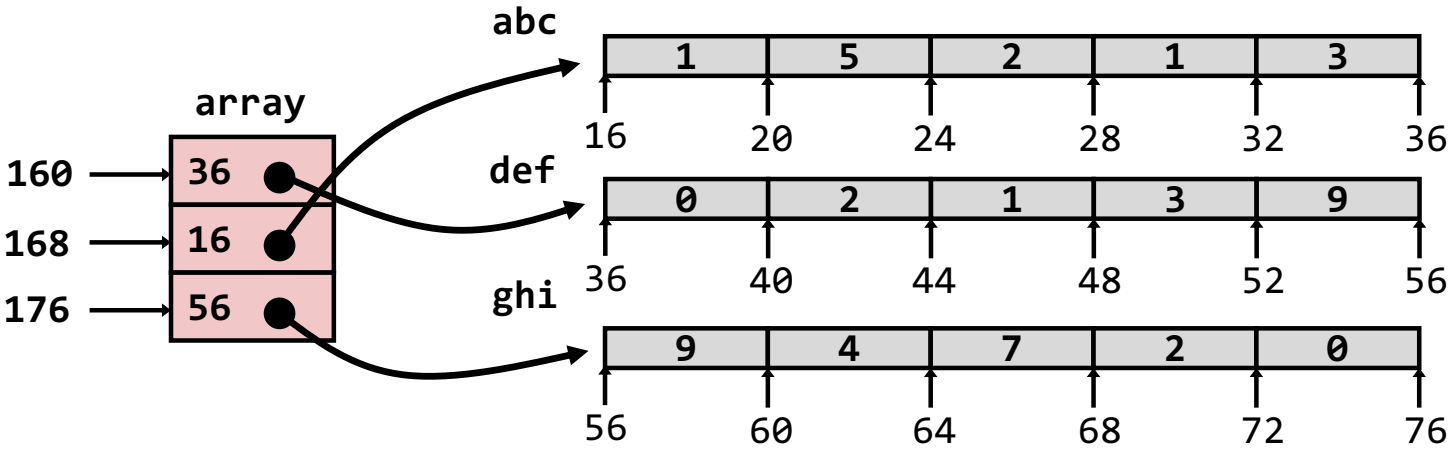
- 数组元素
  - **pgh[index][dig]** 是整型元素
  - 地址: **pgh + 20*index + 4*dig**
    = pgh + 4*(5*index + dig)

多级数组示例

```
zip_dig abc = { 1, 5, 2, 1, 3 };
zip_dig def = { 0, 2, 1, 3, 9 };
zip_dig ghi = { 9, 4, 7, 2, 0 };
```

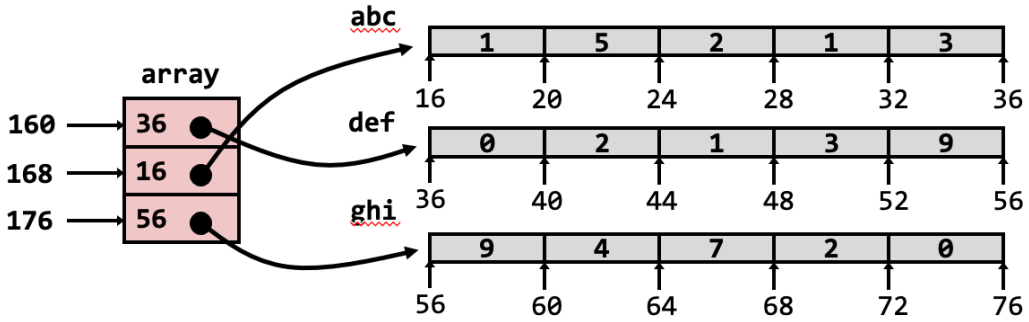
```
#define UCOUNT 3
int *array[UCOUNT] = {def, abc, ghi};
```

- 变量 **array** 表示有3个元素的数组
- 每个元素是指针类型
  - 8 个字节
- 每个指针指向一个整型数组



多级数组的元素访问

```
int get_array_digit
  (size_t index, size_t digit)
{
  return array[index][digit];
}
```




```
  salq    $2, %rsi              # 4*digit
  addq    array(,%rdi,8), %rsi # p = array[index] + 4*digit
  movl    (%rsi), %eax          # return *p
  ret
```

- 计算
  - 访问一个元素是要访问 **Mem[Mem[array+8*index]+4*digit]**
  - 必须读取两次内存
    - 第一次获取指向行数组的指针
    - 第二次访问数组中的元素

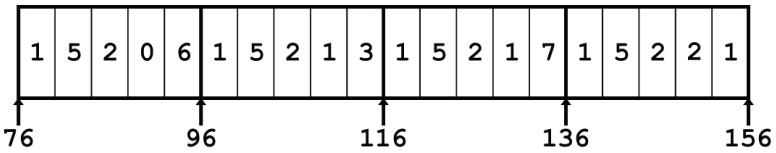
访问数组元素

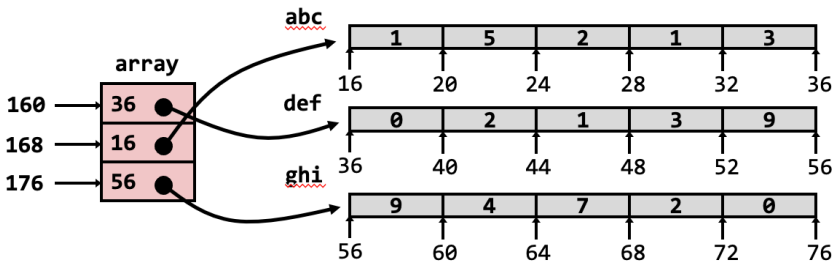
嵌套的数组

```
int get_pgh_digit
  (size_t index, size_t digit)
{
  return pgh[index][digit];
}
```

多级数组

```
int get_array_digit
  (size_t index, size_t digit)
{
  return array[index][digit];
}
```





在C语言中，访问看起来类似，但地址计算非常不同

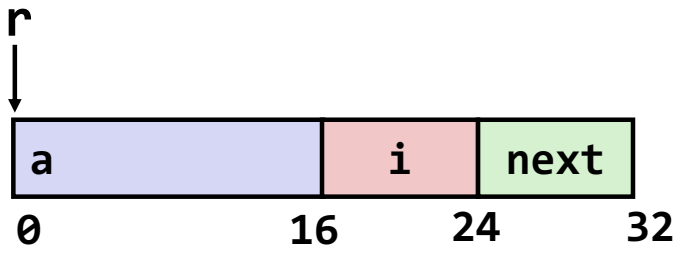
Mem[pgh+20*index+4*digit]　　**Mem[Mem[array+8*index]+4*digit]**

主要内容

- 数组
  - 一维数组
  - 多维数组（嵌套）
  - 变长数组
- 结构体
  - 分配
  - 存取
  - 数据对齐
- 联合体

结构描述

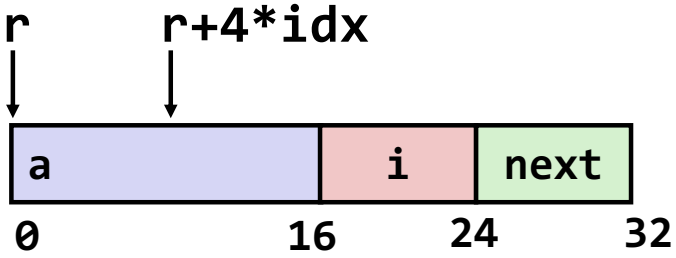
```
struct rec {
    int a[4];
    size_t i;
    struct rec *next;
};
```



- 结构是以一块连续的内存存储的
  - 大小可以容纳结构体所有的字段
- 字段按照声明的顺序排列
  - 即使另一种排列顺序可以产生更紧凑的表示
- 编译器决定字段的总体大小和位置
  - 机器级程序完全不知道源代码中的结构体

生成指向结构成员的指针

```
struct rec {
    int a[4];
    size_t i;
    struct rec *next;
};
```



- 生成指向数组元素的指针
  - 在编译时确定每个结构成员的偏移量
  - 计算 **r + 4*idx**

```
int *get_ap
 (struct rec *r, size_t idx)
{
  return &r->a[idx];
}
```

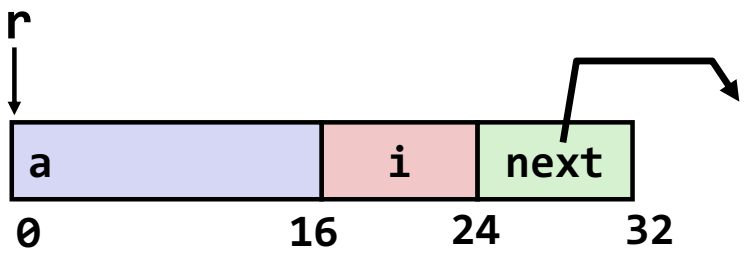
```
  # r in %rdi, idx in %rsi
  leaq  (%rdi,%rsi,4), %rax
  ret
```

遍历链表 #1

- C 代码

```
long length(struct rec*r) {
    long len = 0L;
    while (r) {
        len ++;
        r = r->next;
    }
    return len;
}
```

```
struct rec {
    int a[4];
    size_t i;
    struct rec *next;
};
```



- 循环汇编代码

```
.L11:                           # loop:
  addq    $1, %rax              #   len ++
  movq    24(%rdi), %rdi        #   r = Mem[r+24]
  testq   %rdi, %rdi            #   Test r
  jne     .L11                  #   If != 0, goto loop
```

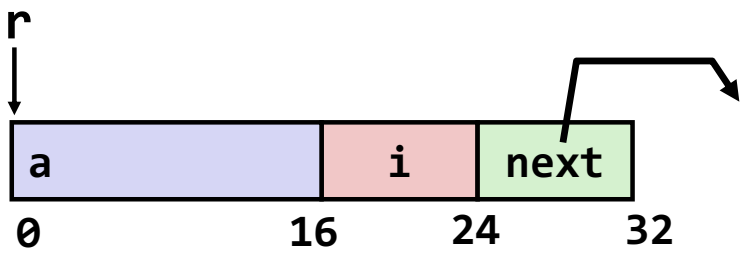
Register	Value
%rdi	r
%rax	len

遍历链表 #2

- C 代码

```
void set_val
  (struct rec *r, int val)
{
  while (r) {
    size_t i = r->i;
    // No bounds check
    r->a[i] = val;
    r = r->next;
  }
}
```

```
struct rec {
    int a[4];
    size_t i;
    struct rec *next;
};
```



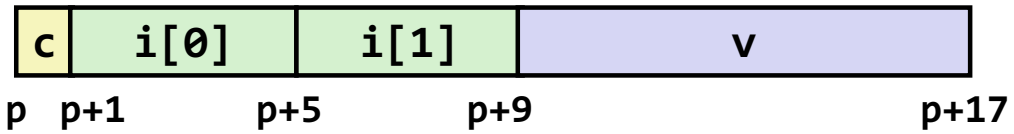
- 循环汇编代码

```
.L11:                           # loop:
  movq  16(%rdi), %rax          #   i = Mem[r+16]
  movl  %esi, (%rdi,%rax,4)     #   Mem[r+4*i] = val
  movq  24(%rdi), %rdi          #   r = Mem[r+24]
  testq %rdi, %rdi              #   Test r
  jne   .L11                    #   if !=0 goto loop
```

Register	Value
%rdi	r
%rax	len

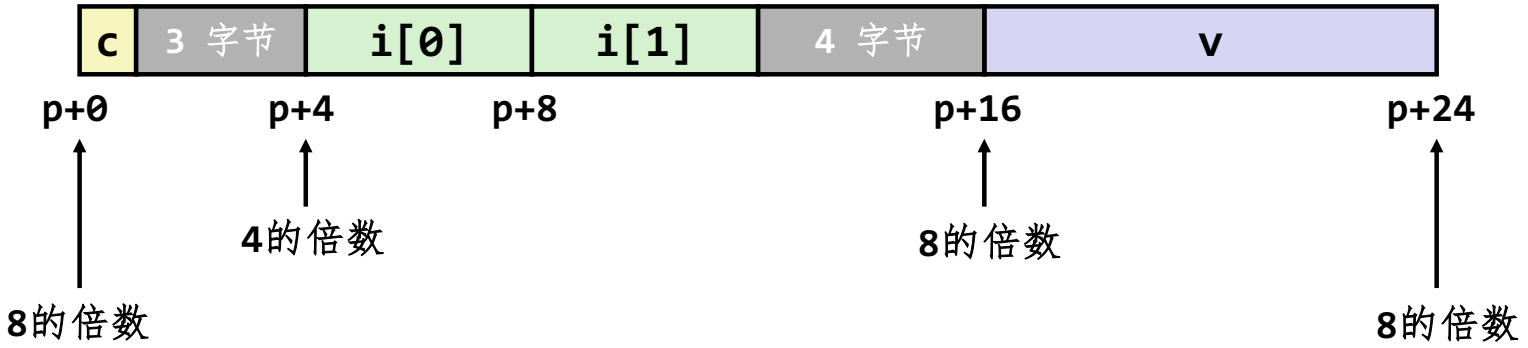
结构与对齐

- 非对齐(unaligned)的数据




```
struct S1 {
  char c;
  int i[2];
  double v;
} *p;
```

- 对齐(aligned)的数据
  - 基本数据类型需要 K 字节表示，则地址必须是 K 的倍数



对齐原则

- **对齐的数据**
  - 基本数据类型需要 K 个字节
  - 地址必须是 K 的倍数
  - 在某些机器上严格要求满足；在 x86-64 上建议满足
- **数据对齐的目的**
  - 内存是以（对齐的）4 或 8 字节（取决于系统）为单位存取内存块的
    - 加载或存储块界限的数据，效率低
    - 虚拟内存中，数据跨越 2 个页面会更棘手
- **编译**
  - 在结构中插入缝隙，以确保字段的正确对齐

对齐的具体案例（x86-64）

- **1 字节：char，…**
  - 对地址无要求
- **2 字节：short，…**
  - 最低1位地址必须是 0_2
- **4 字节：int，float，…**
  - 最低2位地址必须是 00_2
- **8 字节：double，long，char *，…**
  - 最低3位地址必须是 000_2

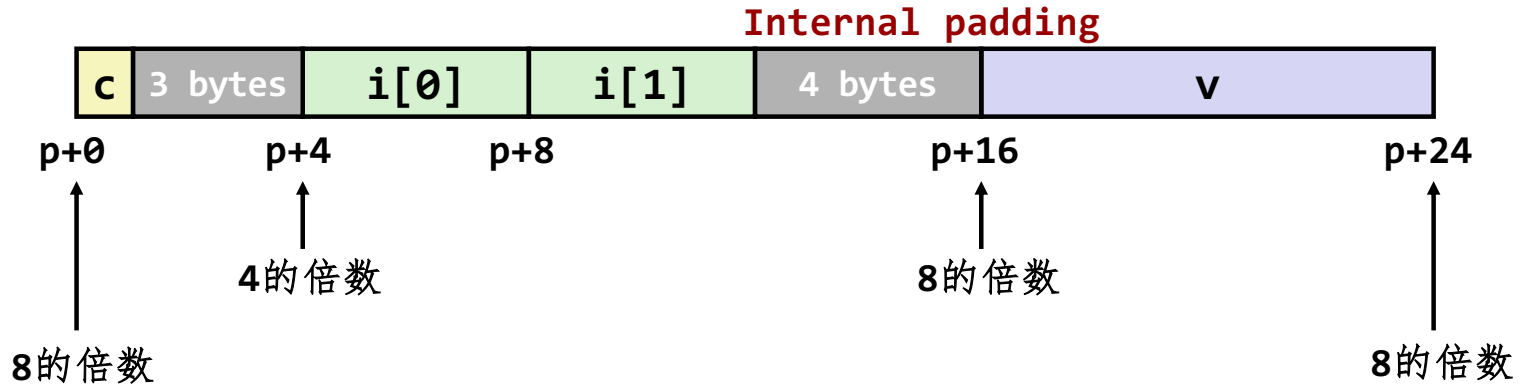
跳转表

```
.section   .rodata
  .align 8
.L4:
  .quad    .L8  # x = 0
  .quad    .L3  # x = 1
  .quad    .L5  # x = 2
  .quad    .L9  # x = 3
  .quad    .L8  # x = 4
  .quad    .L7  # x = 5
  .quad    .L7  # x = 6
```

结构体的对齐要求

- 结构体内部
  - 必须满足每个元素的对齐要求
- 总体结构放置
  - 每个结构都有对齐要求 K
    - K = 所有元素的最大对齐
  - 初始地址和结构长度必须是 K 的倍数
- 示例
  - K = 8, 因为是 double 类型元素

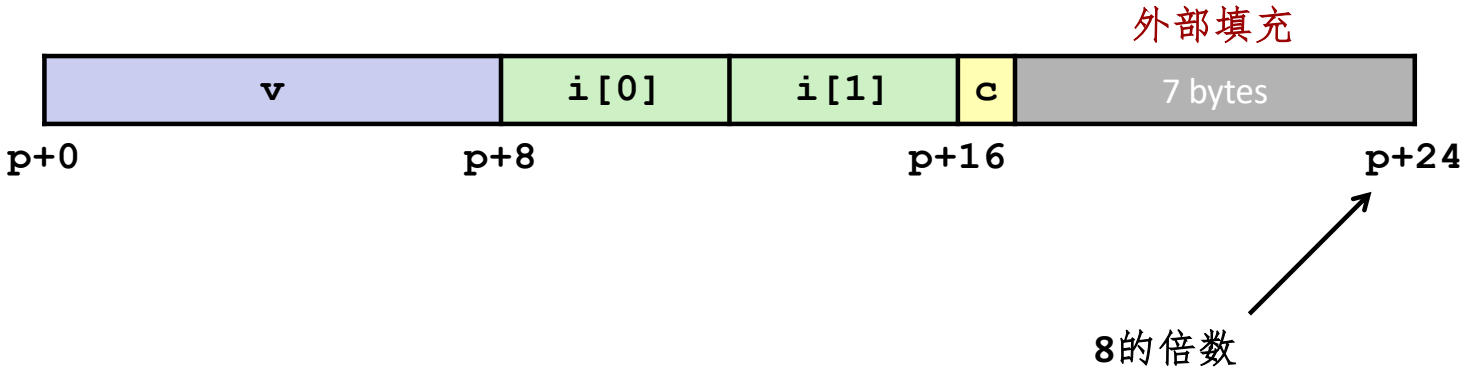
```
struct S1 {
  char c;
  int i[2];
  double v;
} *p;
```



满足整体对齐要求

- 最大的对齐要求 K
- 整体结构必须是 K 的倍数

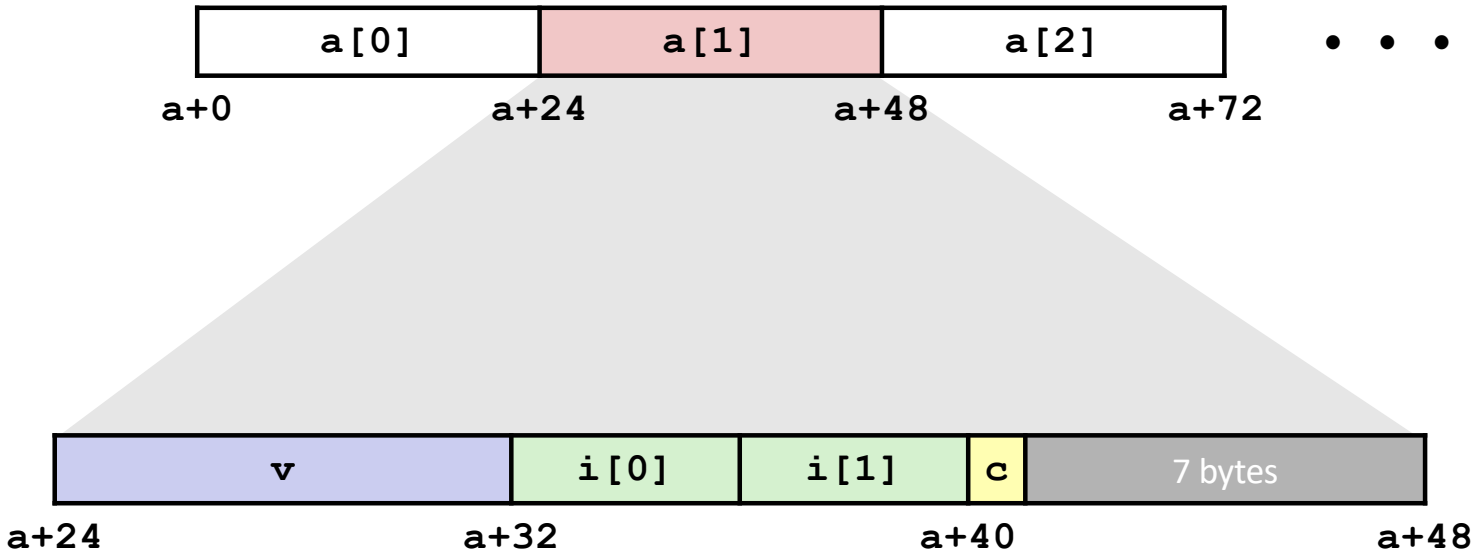
```
struct S2 {
  double v;
  int i[2];
  char c;
} *p;
```



元素是结构体的数组

- 整体结构长度是 K 的倍数
- 满足每个基本元素的对齐要求

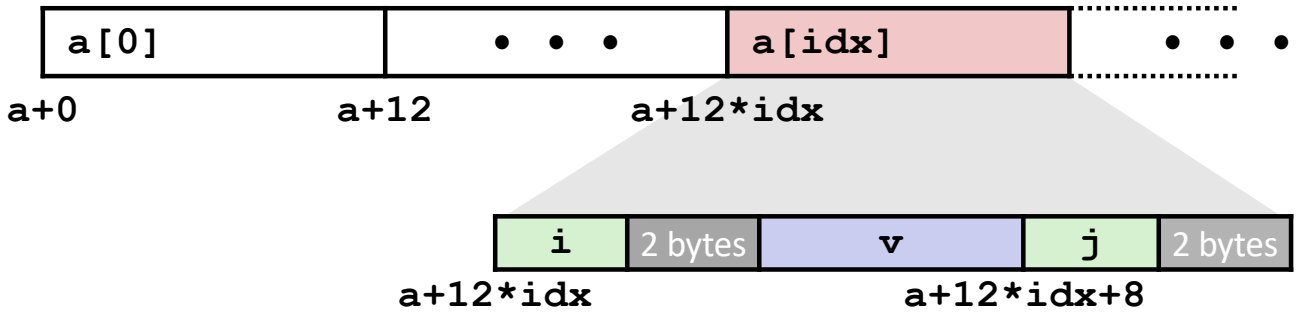
```
struct S2 {
  double v;
  int i[2];
  char c;
} a[10];
```



访问数组元素

- 计算数组偏移量 **12*idx**
  - sizeof(S3), 包括对齐所需的填充空间
- 元素 **j** 在结构中的偏移量为**8**
- 编译器给出偏移量 **a+8**

```
struct S3 {
  short i;
  float v;
  short j;
} a[10];
```




```
short get_j(int idx)
{
  return a[idx].j;
}
```

```
# %rdi = idx
leaq (%rdi,%rdi,2),%rax # 3*idx
movzwl a+8(,%rax,4),%eax
```

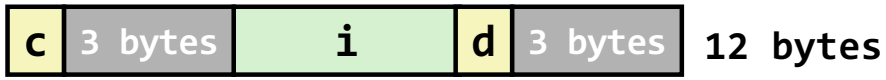
节省空间

- 把大型数据类型放在前面

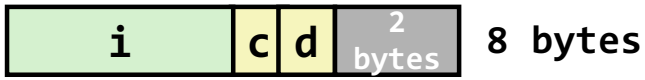
```
struct S4 {
  char c;
  int i;
  char d;
} *p;
```



```
struct S5 {
  int i;
  char c;
  char d;
} *p;
```



- 效果（最大对齐要求 K = 4）



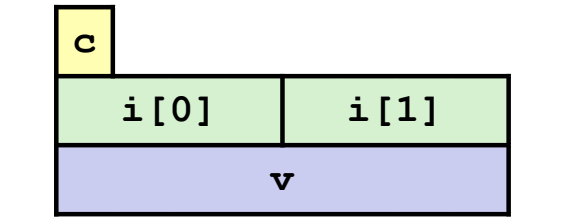
主要内容

- 数组
  - 一维数组
  - 多维数组（嵌套）
  - 变长数组
- 结构体
  - 分配
  - 存取
  - 数据对齐
- 联合体

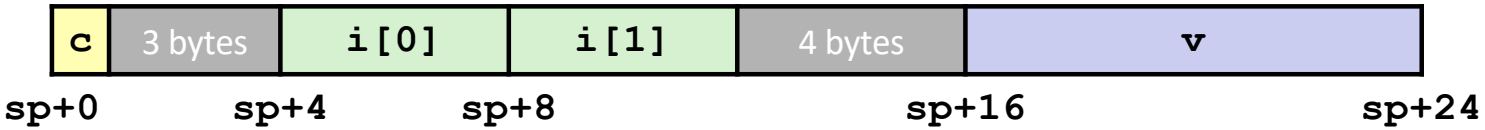
联合体分配

- 按最大元素分配
- 一次只能使用一个字段

```
union U1 {
  char c;
  int i[2];
  double v;
} *up;
```

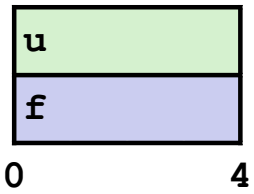



```
struct S1 {
  char c;
  int i[2];
  double v;
} *sp;
```



使用联合体访问位模式

```
typedef union {
    float f;
    unsigned u;
} bit_float_t;
```




```
float bit2float(unsigned u) {
    bit_float_t arg;
    arg.u = u;
    return arg.f;
}
```

Same as (float) u ?

```
unsigned float2bit(float f) {
    bit_float_t arg;
    arg.f = f;
    return arg.u;
}
```

Same as (unsigned) f ?

小结：C 语言中的复合数据结构

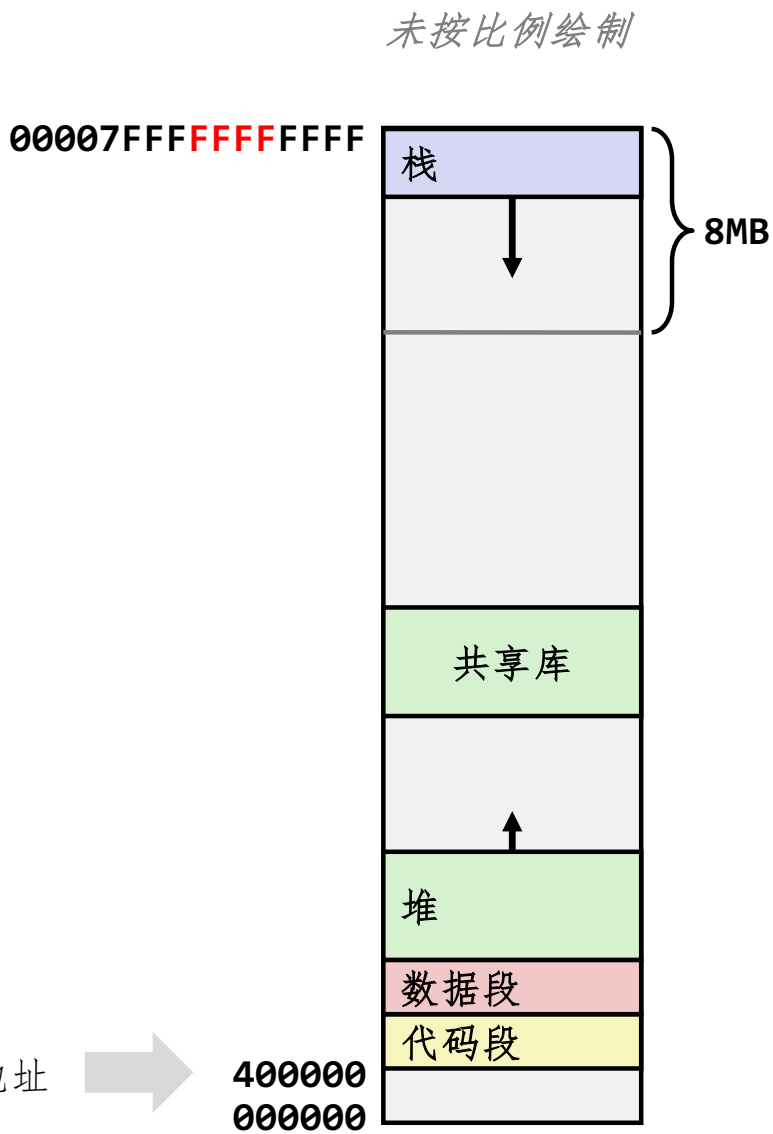
- 数组
  - 连续分配内存
  - 对齐以满足每个元素的对齐要求
  - 指向第一个元素的指针
  - 无边界检查
- 结构体
  - 按声明顺序分配字节
  - 在中间和末尾填充以满足对齐要求
- 联合体
  - 共享声明的内存区域
  - 一种绕过类型系统的方法

进阶话题：本节内容

- 内存布局
- 缓冲区溢出
  - 漏洞
  - 保护

x86-64 Linux 内存布局

- 栈
  - 运行时栈(8MB限制)
  - 例如，局部变量
- 堆
  - 根据需要动态分配
  - 调用函数malloc(), calloc(), new()时
- 数据
  - 静态分配的数据
  - 例如，全局变量，static变量，字符串常量
- 代码段/共享库
  - 可执行的机器指令
  - 只读



内存分配示例

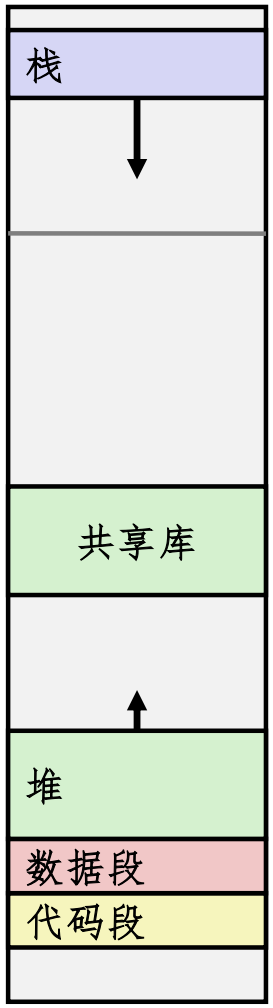
未按比例绘制

```
char big_array[1L<<24];  /* 16 MB */
char huge_array[1L<<31]; /*  2 GB */

int global = 0;

int useless() { return 0; }

int main ()
{
    void *p1, *p2, *p3, *p4;
    int local = 0;
    p1 = malloc(1L << 28); /* 256 MB */
    p2 = malloc(1L << 8);  /* 256  B */
    p3 = malloc(1L << 32); /*   4 GB */
    p4 = malloc(1L << 8);  /* 256  B */
 /* Some print statements ... */
}
```



都被分配到哪去了呢？

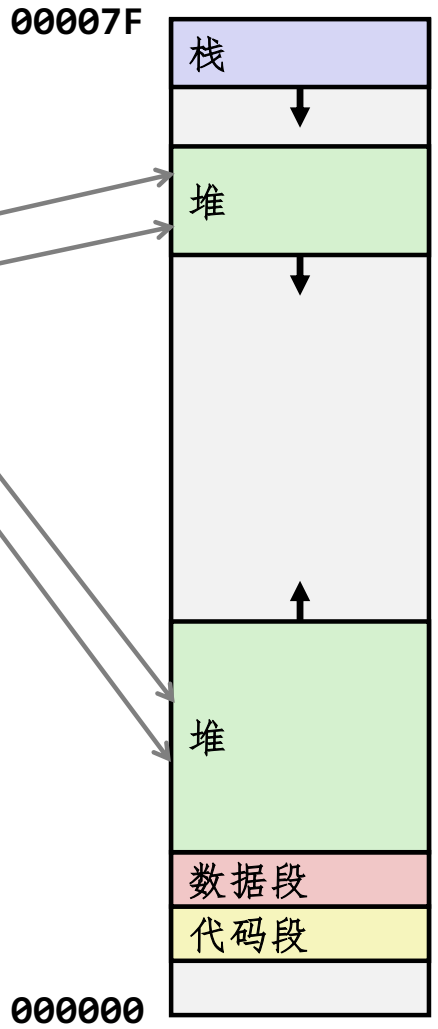
x86-64 地址示例

地址范围0~2^{47}

local	0x00007ffe4d3be87c
p1 (256MB)	0x00007f7262a1e010
p3 (4GB)	0x00007f7162a1d010
p4 (256B)	0x000000008359d120
p2 (256B)	0x000000008359d010
big_array	0x0000000080601060
huge_array	0x0000000000601060
main()	0x000000000040060c
useless()	0x0000000000400590

local	0x7ff7bca78884
p1(256MB)	0x7fccec700000
p3(4GB)	0x7fcbec700000
p4(256B)	0x600002c58100
p2(256B)	0x600002c58000
big_array	0x10348f020
huge_array	0x10448f020
main	0x10348add0
useless	0x10348adc0
global	0x10348f018

未按比例绘制



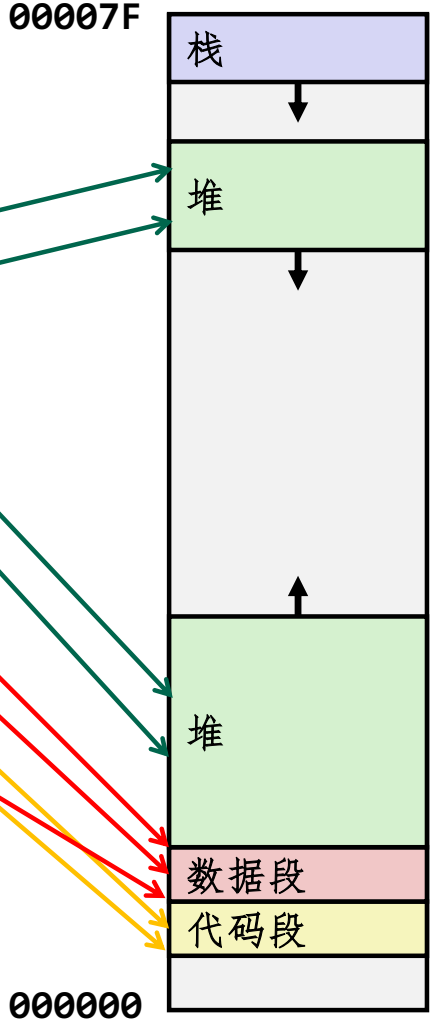
x86-64 地址示例

未按比例绘制

地址范围0~2^{47}

```
local           0x7ff7bca78884
p1(256MB)       0x7fccec700000
p3(4GB)         0x7fcbec700000
p4(256B)        0x600002c58100
p2(256B)        0x600002c58000
big_array       0x10348f020
huge_array      0x10448f020
main            0x10348add0
useless         0x10348adc0

global            0x10348f018
```



本节内容

- 内存布局
- 缓冲区溢出
  - 漏洞
  - 保护

回想：内存引用的 BUG

```
typedef struct {
  int a[2];
  double d;
} struct_t;

double fun(int i) {
  volatile struct_t s;
  s.d = 3.14;
  s.a[i] = 1073741824; /* Possibly out of bounds */
  return s.d;
}
```

```
fun(0)  →    3.14
fun(1)  →    3.14
fun(2)  →    3.1399998664856
fun(3)  →    2.00000061035156
fun(4)  →    3.14
fun(6)  →    Segmentation fault
```

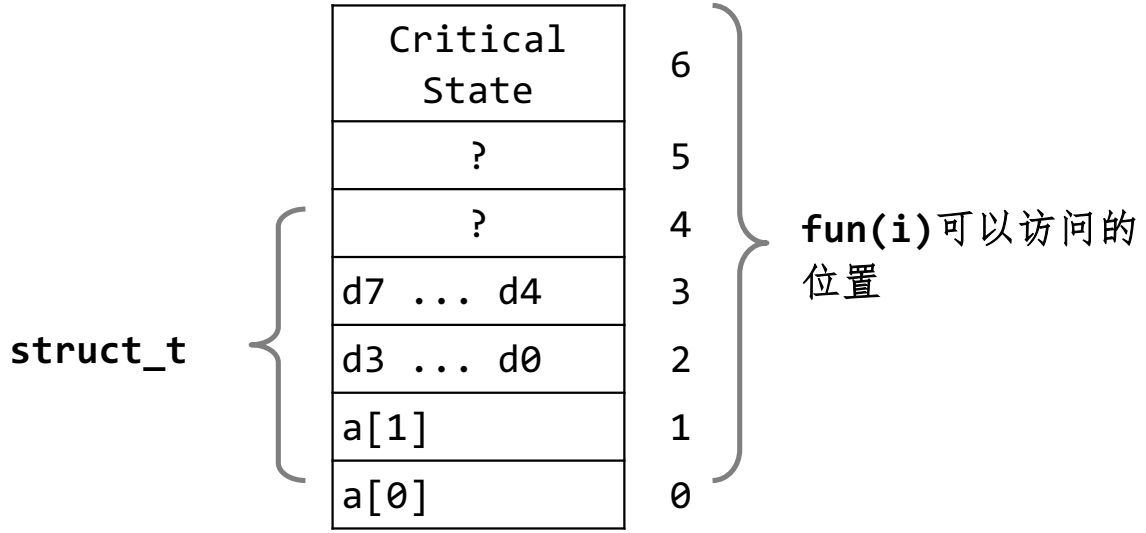
- 结果具有系统特异性

内存引用 BUG 示例

```
typedef struct {
  int a[2];
  double d;
} struct_t;
```

```
fun(0)  →  3.14
fun(1)  →  3.14
fun(2)  →  3.1399998664856
fun(3)  →  2.00000061035156
fun(4)  →  3.14
fun(6)  →  Segmentation fault
```

解释：



这些问题事关重大

- 通常被称作“缓冲区溢出”
  - 超出内存为数组所分配大小时发生
- 为什么事关重大？
  - 这是导致安全漏洞的 #1 技术原因
    - 所有的原因中 #1 的是social engineering / user ignorance
- 最常见的形式
  - 未检查输入字符串的长度
  - 特别是对于栈上有界的字符数组
    - 有时也被称为栈粉碎（stack smash）

String 库源代码

- Unix函数gets()的实现
  - 没有办法对读取的字符数做限制
- 其它库函数也有类似的问题
  - strcpy, strcat:复制任意长度的字符串
  - scanf, fscanf, sscanf: 当使用%s的转换规范时

```
/* Get string from stdin */
char *gets(char *dest)
{
    int c = getchar();
    char *p = dest;
    while (c != EOF && c != '\n') {
        *p++ = c;
        c = getchar();
    }
    *p = '\0';
    return dest;
}
```

缓冲区漏洞代码

```
/* Echo Line */
void echo()
{
    char buf[4];  /* Way too small! */
    gets(buf);
    puts(buf);
}
```

```
void call_echo() {
    echo();
}
```

```
unix>./bufdemo-nsp
Type a string:0123456789012345678901230123456789012345678901230
```

```
unix>./bufdemo-nsp
Type a string:012345678901234567890123
Segmentation Fault
```

那么，要多大才足够大呢？

缓冲区溢出汇编代码

echo:

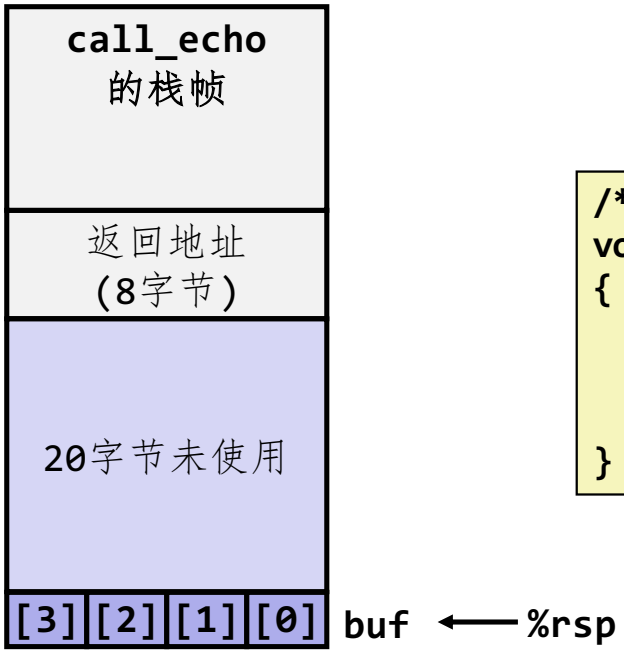
```
0000000004006cf <echo>:
 4006cf:   48 83 ec 18              sub    $0x18,%rsp
 4006d3:   48 89 e7                 mov    %rsp,%rdi
 4006d6:   e8 a5 ff ff ff           callq  400680 <gets>
 4006db:   48 89 e7                 mov    %rsp,%rdi
 4006de:   e8 3d fe ff ff           callq  400520 <puts@plt>
 4006e3:   48 83 c4 18              add    $0x18,%rsp
 4006e7:   c3                       retq
```

call_echo:

```
 4006e8:   48 83 ec 08              sub    $0x8,%rsp
 4006ec:   b8 00 00 00 00           mov    $0x0,%eax
 4006f1:   e8 d9 ff ff ff           callq  4006cf <echo>
 4006f6:   48 83 c4 08              add    $0x8,%rsp
 4006fa:   c3                       retq
```

缓冲区溢出的栈

调用gets前

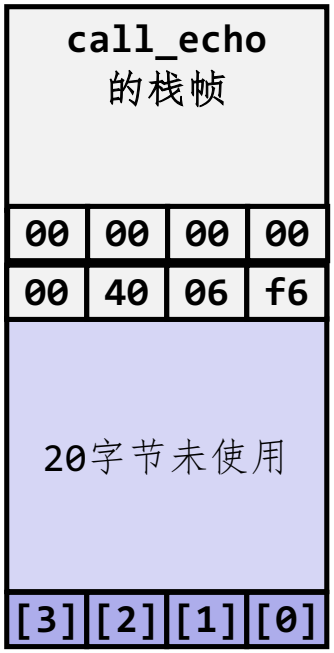



```
/* Echo Line */
void echo()
{
    char buf[4];  /* Way too small! */
    gets(buf);
    puts(buf);
}
```

```
echo:
  subq  $24, %rsp
  movq  %rsp, %rdi
  call  gets
  . . .
```

缓冲区溢出的栈示例

调用gets前




```
void echo()
{
    char buf[4];
    gets(buf);
    . . .
}
```

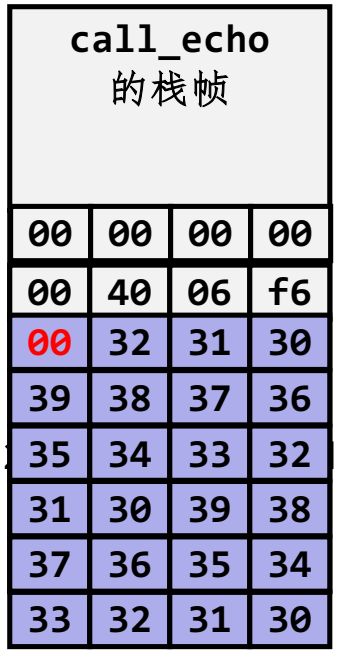
```
echo:
  subq  $24, %rsp
  movq  %rsp, %rdi
  call  gets
  . . .
```

call_echo:

```
  . . .
4006f1:   callq  4006cf <echo>
4006f6:   add    $0x8,%rsp
  . . .
```

缓冲区溢出的栈示例#1

调用gets后




```
void echo()
{
    char buf[4];
    gets(buf);
    . . .
}
```

```
echo:
  subq  $24, %rsp
  movq  %rsp, %rdi
  call  gets
  . . .
```

call_echo:

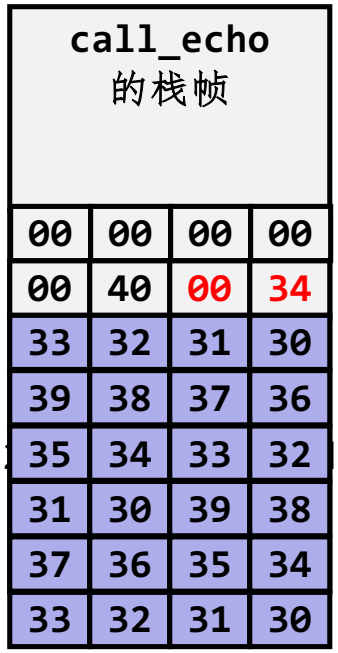
```
  . . .
  4006f1:   callq  4006cf <echo>
  4006f6:   add    $0x8,%rsp
  . . .
```

```
unix>./bufdemo-nsp
Type a string:0123456789012345678901 2
0123456789012345678901 2
```

缓冲区溢出了，但没有造成状态崩溃

缓冲区溢出的栈示例#2

调用gets后




```
void echo()
{
    char buf[4];
    gets(buf);
    . . .
}
```

```
echo:
  subq  $24, %rsp
  movq  %rsp, %rdi
  call  gets
  . . .
```

call_echo:

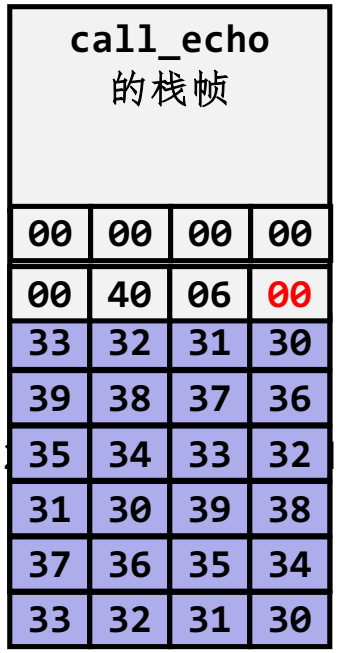
```
  . . .
  4006f1:   callq  4006cf <echo>
  4006f6:   add    $0x8,%rsp
  . . .
```

```
unix>./bufdemo-nsp
Type a string:0123456789012345678901234
Segmentation Fault
```

溢出的缓冲区，且破坏了返回指针

缓冲区溢出的栈示例#3

调用gets后




```
void echo()
{
    char buf[4];
    gets(buf);
    . . .
}
```

```
echo:
  subq  $24, %rsp
  movq  %rsp, %rdi
  call  gets
  . . .
```

call_echo:

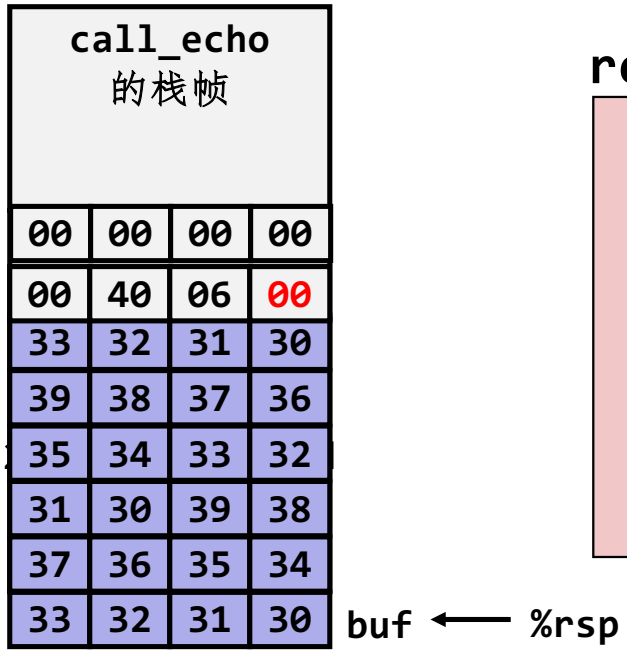
```
  . . .
  4006f1:   callq  4006cf <echo>
  4006f6:   add    $0x8,%rsp
  . . .
```

```
unix>./bufdemo-nsp
Type a string:012345678901234567890123
012345678901234567890123
```

缓冲区溢出，破坏了返回指针，但程序似乎可以运作！

缓冲区溢出的示例#3 解释

调用gets后



register_tm_clones:

```
. . .
400600:   mov    %rsp,%rbp
400603:   mov    %rax,%rdx
400606:   shr    $0x3f,%rdx
40060a:   add    %rdx,%rax
40060d:   sar    %rax
400610:   jne    400614
400612:   pop    %rbp
400613:   retq
```

“返回”不相关的代码

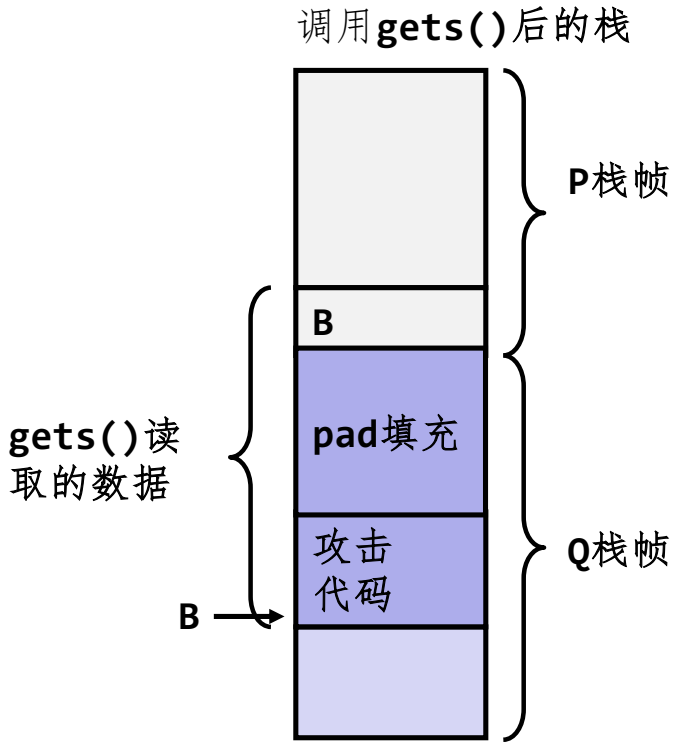
发生了很多事情，但没有修改关键状态，最终会执行 **retq** 返回到 **main**

代码注入攻击

```
void P(){
  Q();
  ...
}
```

返回地址A

```
int Q() {
  char buf[64];
  gets(buf);
  ...
  return ...;
}
```



- 输入的字符串包括了可执行代码的字节表示
- 用B的地址覆写返回地址A
- 当Q执行ret，就会跳转到攻击代码

利用缓冲区溢出

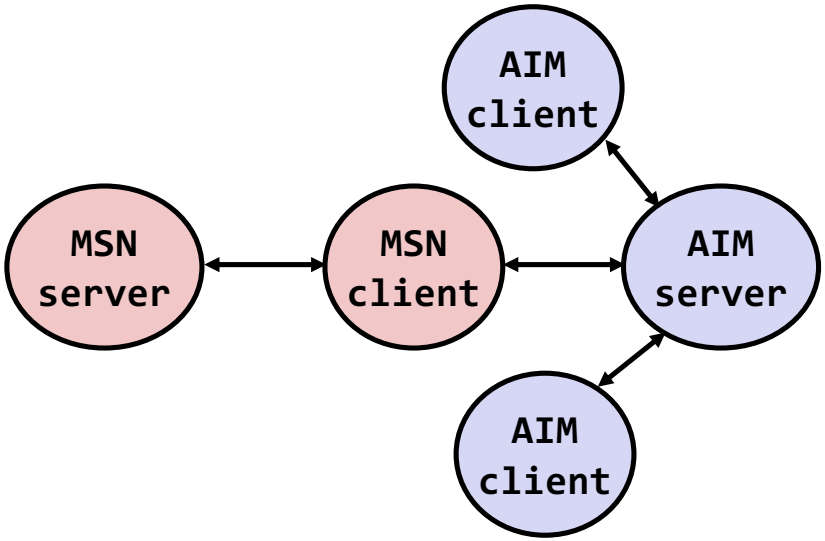
- *远程机器利用缓冲区溢出漏洞可以在受害机器上执行任意代码*
- 令人苦恼的是，这在实际程序中很常见
  - 程序员总是在范相同的错误☹
  - 最近一些措施让这些攻击变得更加困难
- 近几十年的例子
  - Original “Internet worm” (1988)
  - “IM wars” (1999)
  - Twilight hack on Wii (2000s)
  - …还有很多很多
- 你将在 **attacklab** 中学习部分技巧
  - 希望能帮助你不在自己的程序中留下这样的漏洞！！

示例1：最初的网络蠕虫(1988)

- 利用一些漏洞进行传播
  - 早期版本的手指服务器(fingerd)使用gets()方法读取客户端发送的参数：
    - **finger yiligong@whu.edu.cn**
  - 蠕虫通过发送伪造的参数进行攻击：
    - **finger *“exploit-code padding new-return-address”***
    - 攻击代码：executed a root shell on the victim machine with a direct TCP connection to the attacker.
- **Once on a machine, scanned for other machines to attack**
  - invaded ~6000 computers in hours (10% of the Internet ☺)
    - see June 1989 article in *Comm. of the ACM*
  - 蠕虫的年轻作者被起诉……

示例2：IM War

- **July, 1999**
  - Microsoft launches MSN Messenger (instant messaging system).
  - Messenger clients can access popular AOL Instant Messaging Service (AIM) servers



IM War（续）

- **August 1999**
  - 神奇的是，MSN 服务器的用户不再能使用 AIM 的服务器了
  - 微软和AOL开始了IM war
    - AOL 修改了服务器以拒绝MSN的客户端
    - 微软为了打败 AOL 对客户端程序做了修改
    - At least 13 such skirmishes
  - 事实上是怎么回事呢？
    - AOL 发现了自己 AIM 用户的缓冲区溢出漏洞
    - 他们利用这个漏洞探测和隔离微软：攻击代码会返回一个4字节的签名（在AIM用户的某些位置）给服务器

Date: Wed, 11 Aug 1999 11:30:57 -0700 (PDT)
From: Phil Bucking <philbucking@yahoo.com>
Subject: AOL exploiting buffer overrun bug in their own software!
To: rms@pharlap.com

Mr. Smith,

I am writing you because I have discovered something that I think you might find interesting because you are an Internet security expert with experience in this area. I have also tried to contact AOL but received no response.

I am a developer who has been working on a revolutionary new instant messaging client that should be released later this year.
...
It appears that the AIM client has a buffer overrun bug. By itself this might not be the end of the world, as MS surely has had its share. But AOL is now *exploiting their own buffer overrun bug* to help in its efforts to block MS Instant Messenger.
....
Since you have significant credibility with the press I hope that you can use this information to help inform people that behind AOL's friendly exterior they are nefariously compromising peoples' security.

Sincerely,
Phil Bucking
Founder, Bucking Consulting
philbucking@yahoo.com

后来确定这封邮件来自微软内部!

另：蠕虫和病毒

- 蠕虫是这样的一个程序：
  - 可以自己运行
  - 可以将自己的完整版传播到其它计算机中
- 病毒是这样的代码：
  - 将自己加在别的程序中
  - 并不能独立运行
- 两者(通常)都是为了在计算机之间传播和造成破坏而设计的

应对缓冲区溢出措施

- 避免溢出漏洞
- 使用系统级保护
- 编译器使用金丝雀
- 我们一一道来……

1.避免代码中的溢出漏洞

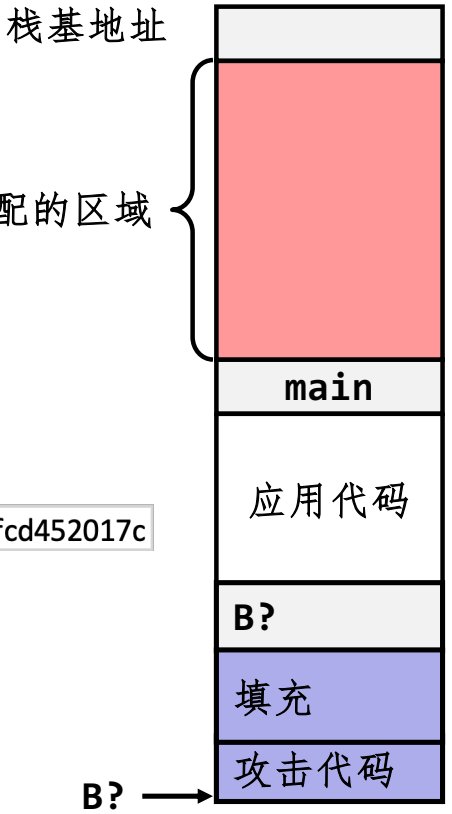
```
/* Echo Line */
void echo()
{
    char buf[4];  /* Way too small! */
    fgets(buf, 4, stdin);
    puts(buf);
}
```

- 例如，使用限制字符串长度的库例程
  - 使用 **fgets** 代替 **gets**
  - 使用 **strncpy** 代替 **strcpy**
  - 不要使用 **scanf** 的 **%s**
    - 使用 **fgets** 来读取字符串
    - 或使用 **%ns**，其中 **n** 是一个适当的整数

2.系统级的保护

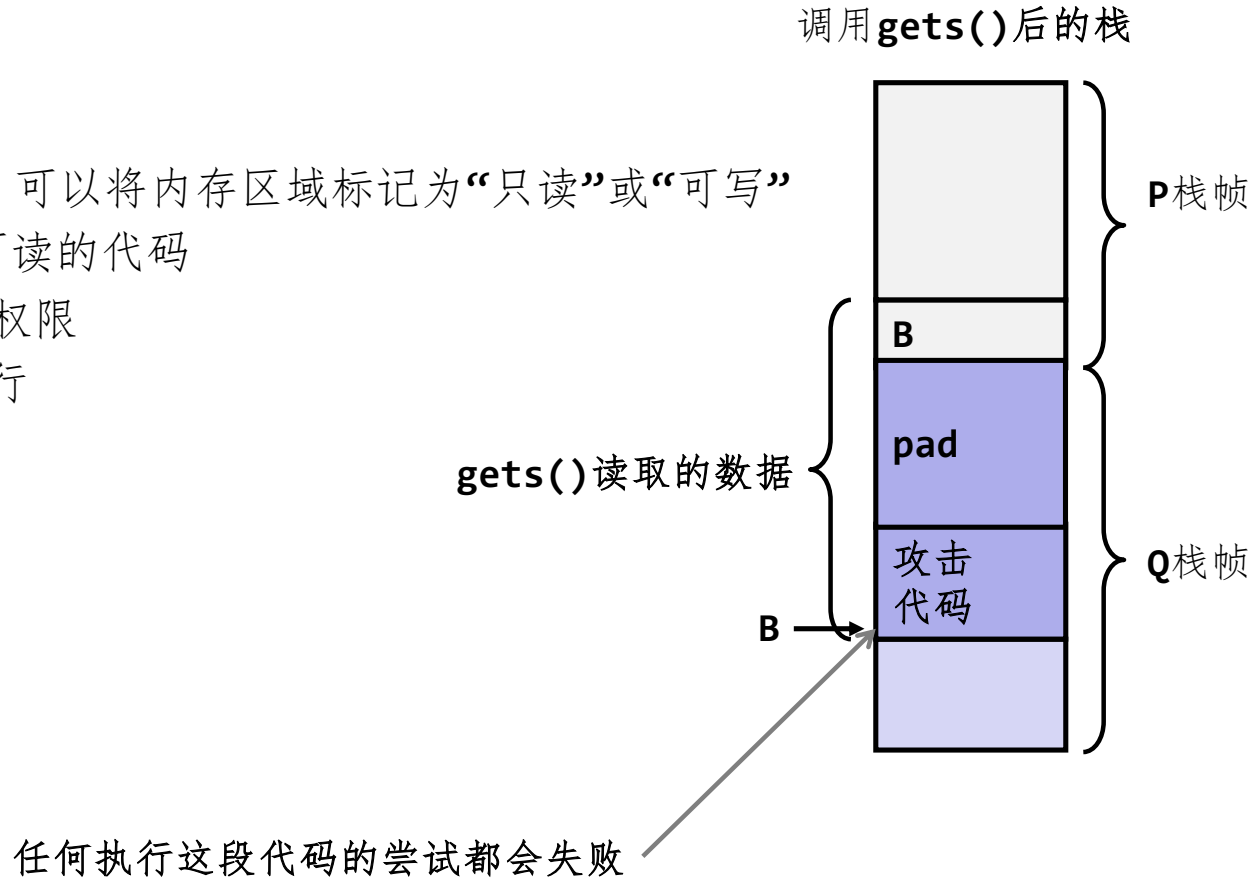
- 栈随机化
  - 程序开始时，在栈上分配一段随机大小的空间
  - 整个程序的地址发生位移
  - 让黑客难以预测插入代码的起始地址
  - E.g.: 5 次执行内存分配代码
    - 每次执行程序，栈的位置都发生改变

local	0x7ffe4d3be87c	0x7fff75a4f9fc	0x7ffeadb7c80c	0x7ffeaea2fdac	0x7ffcd452017c



2.系统级保护

- 不可执行代码段
  - 传统的x86体系中，可以将内存区域标记为“只读”或“可写”
    - 可以执行任何可读的代码
  - 增加了显式“执行”权限
  - 栈被标记为不可执行



3.栈金丝雀

- 想法
  - 在缓冲区附近放置一个特殊值（“金丝雀值”）
  - 退出函数前检查是否遭到破坏
- GCC实现
  - **-fstack-protector**
  - Now the default (disabled earlier)

```
unix>./bufdemo-sp
Type a string:0123456
0123456
```

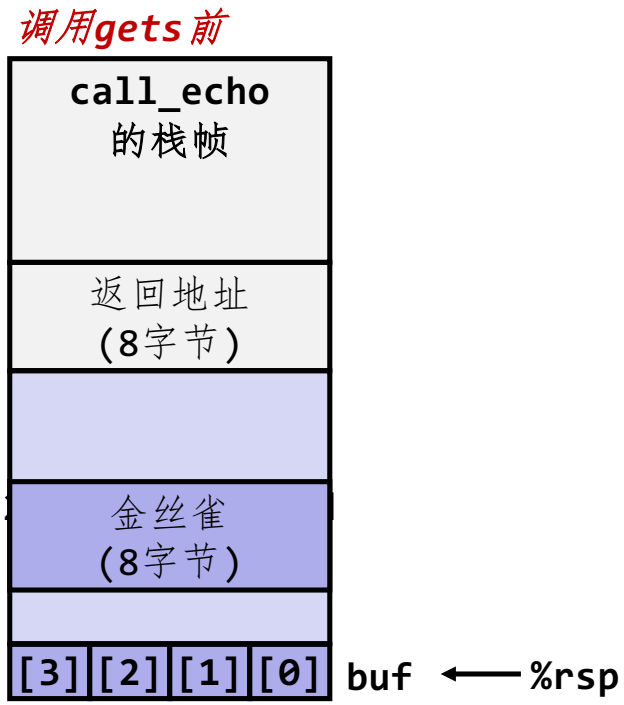
```
unix>./bufdemo-sp
Type a string:01234567
*** stack smashing detected ***
```

受保护缓冲区的汇编代码

echo:

```
40072f:   sub    $0x18,%rsp
400733:   mov    %fs:0x28,%rax
40073c:   mov    %rax,0x8(%rsp)
400741:   xor    %eax,%eax
400743:   mov    %rsp,%rdi
400746:   callq  4006e0 <gets>
40074b:   mov    %rsp,%rdi
40074e:   callq  400570 <puts@plt>
400753:   mov    0x8(%rsp),%rax
400758:   xor    %fs:0x28,%rax
400761:   je     400768 <echo+0x39>
400763:   callq  400580 <__stack_chk_fail@plt>
400768:   add    $0x18,%rsp
40076c:   retq
```

设置金丝雀

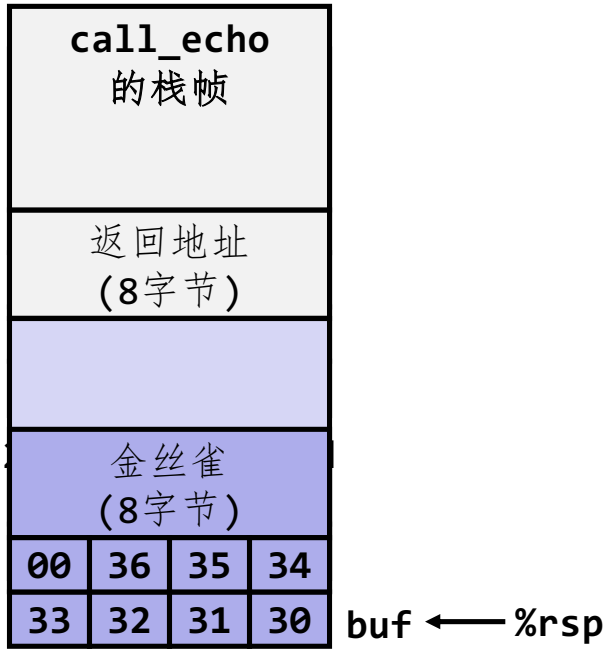



```
/* Echo Line */
void echo()
{
    char buf[4];  /* Way too small! */
    gets(buf);
    puts(buf);
}
```

```
echo:
    . . .
    movq    %fs:0x28, %rax  # Get canary
    movq    %rax, 8(%rsp) # Place on stack
    xorl    %eax, %eax    # Erase canary
    . . .
```

检查金丝雀

调用gets后




```
/* Echo Line */
void echo()
{
    char buf[4];  /* Way too small! */
    gets(buf);
    puts(buf);
}
```

输入*0123456*

```
echo:
    . . .
    movq    8(%rsp), %rax     # Retrieve from stack
    xorq    %fs:0x28, %rax    # Compare to canary
    je      .L6               # If same, OK
    call    __stack_chk_fail  # FAIL
.L6:. . .
```

面向返回的编程攻击

- **挑战（对黑客而言）**
  - 栈随机化使预测缓冲区的位置变得困难
  - 将栈标记为不可执行使插入二进制代码变得困难
- **可选择的策略**
  - 使用已经存在的代码
    - 例如，`stdlib` 的库代码
  - 字符串的片段组合在一起以达到希望的结果
  - *并不能“骗”过金丝雀*
- ***从 gadgets 中构建程序***
  - 以 **`ret` 结尾的指令序列**
    - 由单字节 **`0xc3` 编码**
  - 每次执行代码的位置是固定的
  - 代码是可执行的

Gadget示例#1

```
long ab_plus_c
  (long a, long b, long c) {
   return a*b + c;
}
```

```
00000000004004d0 <ab_plus_c>:
  4004d0:  48 0f af fe  imul %rsi,%rdi
  4004d4:  48 8d 04 17  lea (%rdi,%rdx,1),%rax
  4004d8:  c3           retq
```



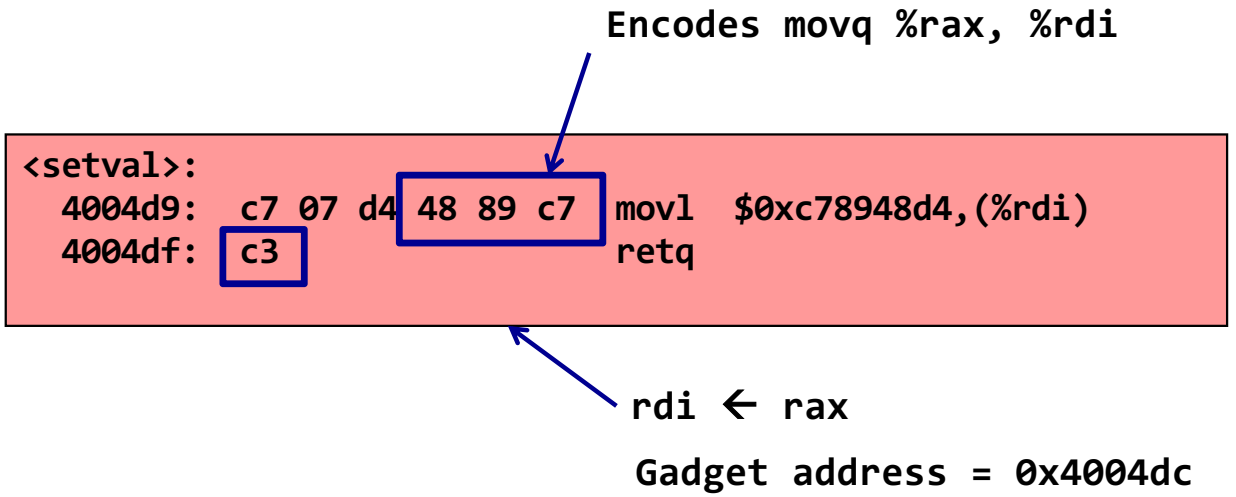
rax ← rdi + rdx

Gadget address = 0x4004d4

- 使用已经存在的函数的尾部

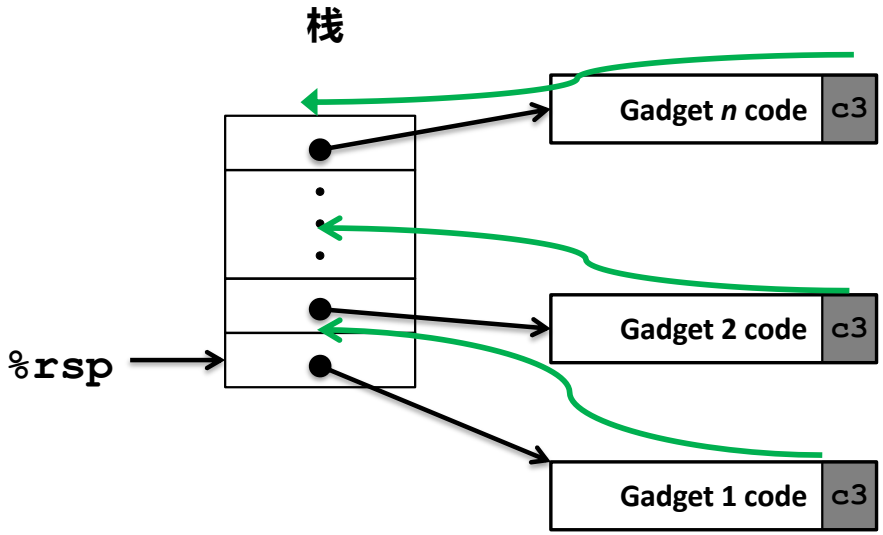
Gadget示例#2

```
void setval(unsigned *p) {
    *p = 3347663060u;
}
```



- 将字节码重新解读～～断章取义！！

ROP 执行



- 以 **ret** 指令触发
  - 会开始执行 gadget 1
- 每段 **gadget** 中最后的 **ret** 指令会启动下一段 **gadget**